CCF-CSP 前三题备考讲义

第一题：基础实现　第二题：工具与算法　第三题：综合模拟

2026 年 10 月 11 日

使用说明

本讲义面向已经学过变量、分支、循环、数组和函数，希望系统训练 CCF-CSP 认证前三题的学生。学习主线是先把短程序写准确，再学习减少重复计算的方法，最后把长题意拆成可靠的程序模块。

全书分成**第一题**、**第二题**、**第三题**三个大的篇章。第一篇依次是“程序基本功与做题技巧、真题速看、考点总结、练习题”；第二篇依次是“基础算法与 STL 使用、STL、真题速看、考点总结、练习题”；第三篇保留字符串解析、综合模拟案例及综合训练。各篇分别编号，提高专题放入附录。题号是一条学习路线，不表示每次考试的算法固定对应。

正文区分两类内容。**认证真题**标出题名、认证次数、年月、题号和做题链接，压缩转述完整规则，并给出解答、样例或关键代码；原资料 23 道真题及六道历年考点补充题均提供按所列题面编写的完整程序。**教学例题与自测**围绕资料知识点设计，完整展示规则、输入输出、样例、解答和参考实现；若旁边链接标为同类练习，平台原题的规则可能不同，提交前应重新核对。

主要在线练习平台：曙梦 OJ 题库；历次 CCF-CSP 认证训练总入口。各算法小节就近列出对应真题或练习题，不需要等到章末才能练习。原资料中的洛谷团队题单和专项训练链接在附录保留，缺少已核对曙梦入口的专项题使用洛谷补充。

题面、届次、数据范围和曙梦当前限制按 2026 年 10 月 11 日读取的页面核对。已核对原资料 23 个真题入口：第一篇五道速看与三道剩余题，第二篇五道速看与十道剩余题。原资料中的 AcWing 入口已通过浏览器题面及来源栏确认身份；部分旧题暂未读取到曙梦对应页面，保留原入口。历年考点补充题另行标明，不声称所有补充题均属于洛谷团队题单。

参考程序以 C 风格为主，使用 scanf/printf、普通数组、下标循环、结构体和独立函数；需要动态容器时再使用 STL。没有使用 auto、匿名函数或结构化绑定。STL 属于 C++，统一按 C++17 编译。主文件内嵌全部正文与代码，只下载本 TeX 文件也可以编译。代码高亮；行号保留显示，以空 ActualText 排除在复制文本之外（需要阅读器支持）；代码正文可选择、复制。输入输出样例分区装框，保留原始空格与换行。

建议先自己推演例题，再说明参考程序的关键不变量，最后遮住答案完成练习。每次写完都应检查最小规模、最大规模、重复值、端点、不可达状态，以及题面允许而样例没有出现的情况。

目录

第一部分　第一题：基础实现 **6**

1　程序基本功与做题技巧 **6**

1.1　程序基本功 6

1.2　读题重点 8

2　真题速看 **9**

2.1　打酱油 9

2.2　跳一跳 10

2.3　线性分类器 11

2.4　检测点查询 12

2.5　田地丈量 13

3　考点总结 **14**

3.1　基础枚举与小模拟 15

4　练习题 **17**

4.1　原题单剩余三道第一题 17

4.2　小中大 17

4.3　数组推导 18

4.4　如此编码 19

4.5　基础统计与枚举复做 20

4.6　例题统计一组整数 20

4.7　练习与解答 21

4.8　基础题训练路线 21

4.9　相反数与相邻数对：不重不漏 22

4.10　门禁系统与灰度直方图：同一种桶的两种输出 22

4.11　数列分段与分蛋糕：何时增加答案 23

4.12　卖菜：首尾与同步更新 23

4.13　报数：数字与轮次分开 24

4.14　归一化处理：均值、方差与浮点除法 24

4.15　重复局面：不用 STL 也能比较字符串 24

4.16　历年考点补充：数组与二维枚举 25

4.17　图像旋转 25

4.18 数位之和 . . . 26
4.19 折点计数 . . . 26
4.20 仓库规划 . . . 27

第二部分　第二题：基础算法与 STL　29

1　基础算法与 STL 使用　29

1.1 先选算法，再选工具 . . . 29
1.2 C 风格程序中怎样使用 STL . . . 30
1.3 前缀和与差分 . . . 30
1.4 二分与双指针 . . . 33
1.5 贪心与基础数学 . . . 37
1.6 搜索与剪枝 . . . 40
1.7 动态规划入门 . . . 43
1.8 基础背包与计数 . . . 47

2　STL　52

2.1 头文件、类型与最小程序 . . . 52
2.2 区间与迭代器：所有接口的共同基础 . . . 53
2.3 sort：普通数组排序与多关键字排序 . . . 53
2.4 例题按成绩和编号排序 . . . 54
2.5 unique：去相邻重复与修改逻辑长度 . . . 55
2.6 lower_bound 与 upper_bound：四种边界 . . . 56
2.7 离散化：值域大时保存相对大小 . . . 57
2.8 vector：可变长数组 . . . 57
2.9 string：与字符数组配合处理文本 . . . 59
2.10 pair：两个值的组合 . . . 61
2.11 stack：后进先出与括号匹配 . . . 61
2.12 queue：先进先出与处理顺序 . . . 63
2.13 priority_queue：堆与优先处理 . . . 64
2.14 set：自动去重的有序集合 . . . 65
2.15 multiset：重复值与删除一个副本 . . . 66
2.16 map：键到信息的映射 . . . 67
2.17 unordered_map 与 unordered_set：哈希查找 . . . 69
2.18 常用算法补充与容器选择 . . . 70

2.19 练习与解答 70

3 真题速看 **71**

3.1 买菜 71
3.2 邻域均值 72
3.3 垦田计划 73
3.4 相似度计算 75
3.5 机器人饲养指南 76

4 考点总结 **77**

5 练习题 **78**

5.1 梦境巡查 78
5.2 机器人宿管指南 79
5.3 字符串变换 81
5.4 机器人复健指南 82
5.5 回收站选址 83
5.6 风险人群筛查 85
5.7 非零段划分 86
5.8 出行计划 87
5.9 寻宝！大冒险！ 88
5.10 何以包邮？ 89
5.11 机器人饲养指南：完全背包复做 90
5.12 数字排序：STL 比较规则复做 91
5.13 其他算法专项训练 91
5.14 算法补充真题：稀疏存储与基础数学 92
5.15 稀疏向量 92
5.16 因子化简 93

第三部分 第三题：字符串解析与综合模拟 **95**

1 字符串处理与解析基础 **95**

1.1 常用操作与区间含义 95
1.2 例题带引号的单行分词 95
1.3 C 字符数组与 string 的区别 96
1.4 解析与业务处理分开 96

1.5 练习与解答 96

2 第三题综合模拟专题 97

2.1 长题意的实现流程 97
2.2 例题路径规范化 97
2.3 例题授权和撤销的状态管理 98
2.4 例题带括号的整数表达式 100
2.5 例题树形文档的后代选择查询 101
2.6 例题模板展开与循环依赖 103
2.7 练习与解答 105

3 综合训练与复盘 106

3.1 练习方式 106
3.2 例题训练 A 差值数对 106
3.3 例题训练 B 矩形增量的最大值 107
3.4 例题训练 C 非抢占任务调度 108
3.5 如何复盘一次错误 109
3.6 提交前检查表 110

A 提高专题与模型扩展 110

A.1 多重背包与二进制拆分 110
A.2 混合背包与多维费用 111
A.3 方案恢复 111
A.4 计数背包的撤销 111
A.5 LIS 优化与排列 LCS 112
A.6 实数二分与三分的前提 112
A.7 可删堆的延迟删除 112
A.8 字符串匹配的线性预处理 113
A.9 附录练习与解答 113

B 资料来源与继续练习 113

B.1 团队题单与专题入口 114
B.2 原资料真题做题入口 114

第一部分　第一题：基础实现

本篇以 C 语言基础、枚举、统计和小模拟为主。先读懂对象、规则与边界，再把它们翻译成变量、数组和循环；基础实现能够清楚表达规则时，优先使用基础实现。

1　程序基本功与做题技巧

1.1　程序基本功

程序基本功就是 C 语言的基础知识，包括格式化输入输出、数据的存储与运算（整型、浮点数）、函数的编写与调用、数组、结构体的使用等。下面按做题中实际会用到的顺序简单总结。

1.1.1　格式化输入输出

scanf 按给定格式读入变量，printf 按给定格式输出结果。读入普通变量时通常要写地址符 &；字符数组的名字已经可以表示首元素地址。格式符必须匹配变量类型。

类型	常见用途	scanf 格式	printf 格式
int	下标、计数、普通整数	`%d`	`%d`
long long	大整数、前缀和、乘积	`%lld`	`%lld`
double	实数、均值、方差	`%lf`	`%f`，如`%.6f`
char	单个字符、类别	`%c`	`%c`
char 数组	单词或一整行文本	`%100s` 或 fgets	`%s`

```
int n; long long total; double x; char type;
scanf("%d%lld%lf %c", &n, &total, &x, &type);
printf("%d %lld %.6f %c\n", n, total, x, type);
char word[101];
scanf("%100s", word);
```

`%c` 默认不跳过空白；格式串中在它前面加空格，表示先跳过空格与换行。`%s` 遇到空白停止，因此不能读取含空格的整行。fgets 读取整行时可能保留换行；数字输入后再读整行，要处理该行剩余字符。第二篇 STL 的 string 小节和字符串变换真题给出处理方法。输出空格和换行按题面要求写，不输出“请输入”等提示语。

1.1.2　整型、浮点数与运算

本讲义所用评测环境中 int 通常占 4 字节，范围为 -2147483648 到 2147483647；long long 占 8 字节，范围约为 $\pm 9.22 \times 10^{18}$。不要只看最后答案，**每个中间结果也必须装得下**。`long long s = a*b;` 仍可能先进行 int 乘法，应写 `1LL*a*b`。

整数除法舍去小数部分（向 0 截断），如 `5/2` 为 2；需要实数结果时写 `5.0/2` 或 `(double)a/b`。`%` 用于整数取余，不能用于 double。对非负整数 x，向上取整 x/d 可用 `x/d + (x%d != 0)`，避免 `x+d-1` 溢出。double 有舍入误差，通常不直接用 `==` 比较计算所得的实数；误差判定方式要与题目精度要求一致。比较均值与整数阈值时，若可以安全交叉相乘，优先使用整数比较。

```
int a = 1000000000, b = 3;
long long product = 1LL * a * b;
double mean = (double)a / b;
```

1.1.3 函数的编写与调用

把一个明确任务写成函数：参数说明输入，返回值说明结果。先定义函数再调用，或在前面声明。C 语言参数按值传递，修改函数内的普通参数不会自动修改调用方；需要修改调用方变量时传指针。数组作为参数时，函数收到首元素地址，必须另传长度。

```
long long sum_array(const int a[], int n) {
    long long s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}
void swap_int(int *x, int *y) {
    int t = *x; *x = *y; *y = t;
}
// Call: total = sum_array(a, n); swap_int(&a[0], &a[1]);
```

函数名应说明职责，例如判断、计数、更新；不要让同一个函数既读入又排序又输出所有状态。递归还必须有终止条件，搜索时另需撤销分支修改。

1.1.4 一维数组、二维数组与初始化

长度为 n 的数组用 0 下标时合法范围是 0 到 $n-1$；用 1 下标时至少申请 $n+1$ 格。字符数组还需要末尾的`'\0'`。二维数组 `a[i][j]` 通常让 i 表示行、j 表示列，循环边界分别对应行数和列数。

```
const int N = 1005;
int a[N], grid[N][N]; // Global arrays are initially zero.
int count[10] = {0};
for (int i = 0; i < n; i++) scanf("%d", &a[i]);
```

全局数组自动初始化为 0，局部普通数组未初始化时不能直接读取。大数组宜放全局，避免局部栈空间不足。最小值、最大值通常用第一个实际元素初始化；若全为正数，把最小值初始化成 0 会出错。频率数组只能在值域可控时按值索引，负数需要偏移，巨大值域不能直接开数组。

1.1.5 结构体与对象属性

一个对象有多个属性时，用结构体把它们放在一起。例如点有横坐标、纵坐标和类别，矩形有四个边界。多个对象用结构体数组存储，读题时的名词便有了明确的数据表示。

```
struct Point { int x, y; char type; };
struct Point p[1005];
scanf("%d%d %c", &p[0].x, &p[0].y, &p[0].type);
```

第一题真题“线性分类器”会直接使用这种写法。不需要因为使用结构体就引入复杂的类设计。

1.2 读题重点

1.2.1 理解基本题意

先用一句话说明“给什么，求什么”，再圈出“至少”“严格大于”“同时”“先……后……”和“不存在时……”等限制。明确输出是一个值、每个对象的结果，还是操作后的完整状态。自己手算一个小例子，并专门构造与样例不同的边界。读懂规则后才选择算法。

例如仓库规划比较的是**所有维度都严格更大**，输出的是**编号最小**的上级；图像旋转是**逆时针**，且行列数交换；折点计数只检查有两个邻居的内部位置。漏掉这些词，即使循环写得再熟练也会得到错误答案。

1.2.2 观察数据范围：两个首要检查

第一，数组不能开小。若 $n \le 1000$ 且使用 1 下标，数组不能只有 1000 格；二维边界、差分的 $n+1$ 位置、字符串末尾都要留出来。数组大小由**索引最大值**决定，不一定由输入个数决定：稀疏向量长度可达 10^9，却只有 5×10^5 个非零项，不能按完整长度开数组。

第二，运算过程中会不会超过 int 或 long long 的范围。若 $n = 2 \times 10^5$ 且每项绝对值不超过 10^9，总和上界为 2×10^{14}，需要 long long；两项乘积的上界应相乘估算。即便 long long 也可能溢出，例如 $10^{18} \times 5$ 还安全，而 $10^{18} \times 100$ 已经越界。机器人宿管会用商和余数拆开计算比例，避免大数直接乘 100。数组范围与数值范围要分别检查。

1.2.3 留意时间限制：时间复杂度与估算

时间复杂度描述基本操作次数随输入规模增长的数量级。$O(n)$ 表示数量随 n 线性增长，$O(n^2)$ 表示平方增长；忽略常数和低阶项，不表示精确秒数。独立的两重 n 循环通常为 $O(n^2)$；前后两次遍历是 $O(n) + O(n) = O(n)$；每步把候选区间减半通常为 $O(\log n)$；排序通常为 $O(n \log n)$。

估算时把**最大数据规模**代入操作次数，再看时限。例如 $n = 1000, m = 10$ 的三重仓库枚举至多约 10^7 次维度比较；$n = 2 \times 10^5$ 的两重枚举约 4×10^{10} 次，通常不可行。$2 \times 10^5 \log_2(2 \times 10^5)$ 约为 3.5×10^6，排序更值得考虑。机器、常数、输入输出和操作类型都会影响耗时，不能把某个次数阈值当作保证。

程序模式	数量级	估算重点
一次遍历 n 项	$O(n)$	每项处理是否有隐藏的大操作。
每次查询重新扫描 n 项	$O(nq)$	多次静态求和可改用前缀和。
枚举所有数对	$O(n^2)$	排序、桶、映射或双指针是否适用。
双指针各自只前进	$O(n)$	累计移动次数，而非只数循环层数。
枚举 n 个元素的子集	$O(2^n)$	$n = 20$ 有约百万个子集，还要算每个子集的检查成本。
DP	状态数 × 每状态转移数	不能只统计状态数。

1.2.4 留意空间限制：空间复杂度与估算

空间复杂度描述存储需求随规模增长的数量级。本书说明程序使用的主要数组，并把输入存储也计入；额外空间另有需要时会单独指出。一个 n 项数组为 $O(n)$，$n \times m$ 矩阵为 $O(nm)$，只保留若干计数变量为 $O(1)$。递归栈也占空间，深度 n 的递归可能占 $O(n)$ 栈空间。

实际估算为“元素个数 × 每个元素字节数”，再把所有数组、容器和递归开销相加。1000×1000 的 int 矩阵约 4×10^6 字节（约 3.8MiB），两个矩阵约 7.6MiB；10^5 项 long long 数组约 0.76MiB；$10^5 \times 10^5$ 的 int 矩阵约 37GiB，显然不可行。结构体可能有对齐填充，STL 容器还有管理开销，按实际 `sizeof` 和容量估算更准确。

1.2.5 写完后的检查顺序

先查格式与类型，再查最小规模、最大下标、重复值、负值、端点和更新顺序；最后分析算法为什么不重不漏。样例通过不代表所有数据正确。小规模时可写直接枚举作为对照，尤其适合检查前缀和、双指针、二分与 DP。

2 真题速看

本节的五题均出自原资料的第一题真题入口，训练枚举、状态更新、对象判定、多键比较与矩形求交。已有曙梦题目优先使用曙梦入口；其余旧题保留原资料链接。认证次数、样例与题面已逐项核对。

2.1 打酱油

第 11 次 CCF-CSP 认证，2017-09，第 1 题。打酱油（原资料入口）。**题目与输入输出**。带 N 元买每瓶 10 元的酱油，可选择买 3 瓶赠 1 瓶或买 5 瓶赠 2 瓶，促销组之间不重复使用已赠送的瓶。求最多得到多少瓶。输入 N，输出最大瓶数。N 是 10 的整数倍且不超过 300。**读题思路**。先把钱换成可付费的瓶数，赠品不消耗钱，也不能把同一份购买重复参加两种促销。付费瓶数至多 30，枚举促销组完全够用。**做题思路与解答**。枚举买 5 赠 2 的组数 f。剩余付费瓶数为 r 时，尽可能组成买 3 赠 1 的组，剩下的按单瓶购买，得到 $7f + 4\lfloor r/3 \rfloor + r \bmod 3$。每种方案都有一个 f；固定 f 时多组成三瓶组只会增加赠品，所以逐项取最大值必然包含最优方案。也可证明优先五瓶组的贪心，但本题的小规模枚举不需要额外假设。

输入输出样例

样例输入

```
40
```

样例输出

```
5
```

输入输出样例

样例输入

```
80
```

样例输出

```
11
```

复杂度与边界。时间 $O(N/50+1)$，额外空间 $O(1)$。0 元时答案 0；余下 4 个付费瓶应组成三瓶组加单瓶。

完整参考实现 49_soy.cpp

```cpp
#include <cstdio>
int main() {
    int money; scanf("%d", &money);
    int paid = money/10, answer = 0;
    for (int five = 0; five*5 <= paid; five++) {
        int rest = paid-five*5;
        int bottles = five*7+(rest/3)*4+rest%3;
        if (bottles > answer) answer = bottles;
    }
    printf("%d\n", answer);
    return 0;
}
```

2.2 跳一跳

第 13 次 CCF-CSP 认证，2018-03，第 1 题。跳一跳（原资料入口）。**题目与输入输出**。输入若干个 0、1、2，数量不超过 30；0 仅在末尾出现，表示游戏结束。1 表示跳到方块非中心，得 1 分；2 表示跳到中心，若是第一次跳或上次得 1 分，则得 2 分，否则比上次多 2 分。输出总得分。**读题思路**。输入没有 n，由 0 终止；连续中心得分为 2、4、6 等，遇到 1 后必须重置。末尾 0 不计分。**做题思路与解答**。last 表示上一跳实际得分，初值 0。读到 1 令 last=1；读到 2 且 last 不超过 1 时令 last=2，否则加 2。然后累加 last。每次更新恰好实现规则，因此处理完前缀后总分始终正确。

输入输出样例

样例输入

```
1 1 2 2 2 1 1 2 2 0
```

样例输出

```
22
```

复杂度与边界。时间与输入数量成正比，空间 $O(1)$。输入首项为 0 时总分为 0；不要把 last 当作上一次输入的 1 或 2。

完整参考实现 50_jump.cpp

```cpp
#include <cstdio>
int main() {
    int x, last = 0, answer = 0;
    while (scanf("%d", &x) == 1 && x != 0) {
        if (x == 1) last = 1;
        else if (last <= 1) last = 2;
        else last += 2;
        answer += last;
    }
    printf("%d\n", answer);
    return 0;
}
```

2.3 线性分类器

第 19 次 CCF-CSP 认证，2020-06，第 1 题。线性分类器（CSP202006A）。**题目与输入输出。**给出 n 个点，每点含整数坐标 x、y 以及类别 A 或 B，两类都存在。随后给 m 条直线 $\theta_0+\theta_1 x+\theta_2 y=0$；保证没有给定点恰在直线上。若同类点全部位于同一侧、异类点位于另一侧，输出 Yes，否则 No，每条直线一行。输入首行 n、m，随后 n 行点、m 行三个系数。$1 \le n \le 1000$，$1 \le m \le 20$，坐标和系数绝对值不超过 10^4。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**必须把一条线与所有点比较；不是检查两类各找一个代表就结束。Yes 和 No 的大小写要精确。两侧由表达式符号区分，无需求斜率，因而竖直线也适用。**做题思路与解答。**用第一个点确定“正侧应该是哪一类”，再检查每个点的符号是否与类别相符。如果有反例，这条线不满足。代入式使用 long long 计算；原范围下 int 也能容纳，但显式提升类型便于保持安全的乘法习惯。正确性来自直线两侧的表达式符号相反，以及逐点检查完整覆盖了分类条件。

输入输出样例

样例输入

```
9 3
1 1 A
1 0 A
1 -1 A
2 2 B
2 3 B
0 1 A
3 1 B
1 3 B
2 0 A
0 2 -3
-3 0 2
-3 1 1
```

样例输出

```
No
No
Yes
```

复杂度与边界。时间 $O(nm)$，空间 $O(n)$。结构体记录三项属性；读类别用格式串中的空格跳过之前的换行。

完整参考实现 38_classifier.cpp

```cpp
#include <cstdio>
struct Point { int x, y; char type; } p[1005];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        scanf("%d%d %c", &p[i].x, &p[i].y, &p[i].type);
    while (m--) {
        int c, a, b; scanf("%d%d%d", &c, &a, &b);
        long long first = c + 1LL*a*p[0].x + 1LL*b*p[0].y;
        char positive = first > 0 ? p[0].type :
                        (p[0].type == 'A' ? 'B' : 'A');
        bool ok = true;
        for (int i = 0; i < n; i++) {
            long long v = c + 1LL*a*p[i].x + 1LL*b*p[i].y;
            if (v == 0 || ((v > 0) != (p[i].type == positive)))
                ok = false;
        }
        puts(ok ? "Yes" : "No");
    }
    return 0;
}
```

2.4 检测点查询

第 20 次 CCF-CSP 认证，2020-09，第 1 题。检测点查询（原资料入口）。**题目与输入输出**。输入 n、居民坐标 X、Y，随后 n 行检测点坐标，编号按输入顺序从 1 开始。输出最近的三个点编号，每个一行；同距离时编号小者在前。$3 \le n \le 200$，坐标绝对值不超过 1000。**读题思路**。要按距离、编号两层比较。无需真的开方，平方距离与距离具有相同顺序。只取三个点，三轮扫描就够用。**做题思路与解答**。计算各点平方距离。每轮从未选点中找最小距离者，按编号升序扫描且只在距离严格减小时替换，就自动保留同距离最小编号。输出并标记已选，重复三轮。归纳可知每轮得到剩余点中的第一名，三轮恰好得到所需前三名。

输入输出样例

样例输入

```
3 2 2
2 2
2 3
2 4
```

样例输出

```
1
2
3
```

输入输出样例

样例输入

```
5 0 1
-1 0
0 0
1 0
0 2
-1 2
```

样例输出

```
2
4
1
```

复杂度与边界。时间 $O(3n) = O(n)$，数组空间 $O(n)$。平方距离最大为 8×10^6，此范围 int 可装下；代码用 long long 并先提升差值类型。

完整参考实现 `51_nearest.cpp`

```cpp
#include <cstdio>
long long dist[205];
bool used[205];
int main() {
    int n, X, Y; scanf("%d%d%d", &n, &X, &Y);
    for (int i = 1; i <= n; i++) {
        int x, y; scanf("%d%d", &x, &y);
        long long dx = 1LL*x-X, dy = 1LL*y-Y;
        dist[i] = dx*dx+dy*dy;
    }
    for (int round = 0; round < 3; round++) {
        int best = 0;
        for (int i = 1; i <= n; i++)
            if (!used[i] && (best == 0 || dist[i] < dist[best])) best = i;
        printf("%d\n", best); used[best] = true;
    }
    return 0;
}
```

2.5 田地丈量

第 29 次 CCF-CSP 认证，2023-03，第 1 题。田地丈量（CSP202303A）。**题目与输入输出**。输入 n、a、b，随后 n 行现有田地矩形的左下与右上坐标。现有田地互不重叠（交集面积为 0），目标区域为 $[0, a] \times [0, b]$。求目标区域内已有田地面积。$n \le 100$，所有坐标绝对值不超过 10^4，a、b 为正。输出面积。**读题思路**。统计连续面积，边接触贡献 0，不能加 1。现有田地互不重叠，所以各块与目标的交面积可以直接相加；若取消这一保证，则不能简单累加。**做题思路与解答**。每块田地裁剪到目标：横向左端取 $\max(x_1, 0)$、右端取 $\min(x_2, a)$，纵向同理。有正宽、正高才累加其乘积。两个一维交区间的笛卡尔积正好是矩形交集；原田地互不重叠，因此求和无重复覆盖。

输入输出样例

样例输入

```
4 10 10
0 0 5 5
5 -2 15 3
8 8 15 15
-2 10 3 15
```

样例输出

```
44
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$，可边读边算。面积与累加值用 long long，乘法前提升类型。

完整参考实现 52_land.cpp

```cpp
#include <cstdio>
int main() {
    int n, a, b; scanf("%d%d%d", &n, &a, &b);
    long long answer = 0;
    while (n--) {
        int x1, y1, x2, y2; scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
        int left = x1 > 0 ? x1 : 0, down = y1 > 0 ? y1 : 0;
        int right = x2 < a ? x2 : a, up = y2 < b ? y2 : b;
        if (left < right && down < up) answer += 1LL*(right-left)*(up-down);
    }
    printf("%lld\n", answer);
    return 0;
}
```

3 考点总结

考点	实现方法	对应真题与常见错误
拆数位与基本循环	取余、整除、条件分支	数位之和、报数；检查 0、跳过计数与轮次。
统计与值域数组	次数桶、边读边更新	门禁系统、灰度直方图；容量由值域决定。
二维数组与坐标映射	按目标输出顺序访问原数组	图像旋转；逆时针方向和行列交换。
局部关系判断	比较相邻项、首尾单独处理	折点计数、卖菜；端点不能访问越界。
对象与多条件判断	结构体、逐项代入	线性分类器；先确定符号与类别的对应。
枚举候选与最小编号	按编号升序，遇到首个合法者停止	仓库规划；严格比较，不能只比较部分维度。

考点	实现方法	对应真题与常见错误
小模拟	定义状态时刻和更新顺序	网格同步更新、日期计算；不要混用新旧状态。
浮点数	均值、方差、数学函数	归一化处理；整数除法和输出精度。

学习后的自测：能否用一句话说明变量的含义？数组最后一个合法下标是什么？最坏情况总共比较多少次？哪个中间值最大？相等、0、负值或最小规模时程序仍按定义工作吗？

3.1 基础枚举与小模拟

3.1.1 模拟的核心是状态时刻

模拟题要求按规则执行，并不意味着随意堆条件。先定义变量表示“操作之前”还是“操作之后”，再明确更新顺序。若同一轮的所有变化依赖旧状态，应把旧状态和新状态分开。若操作依次发生，则后面的操作通常应看到前面操作的结果。

枚举题还需说明为什么不重不漏。例如枚举无序数对，使用 $1 \le i < j \le n$，每对只出现一次；若使用 $i \ne j$，则每对通常出现两次。判断条件正确和枚举范围正确同样重要。

3.1.2 例题网格的一轮同步更新

题目。给定一个 $n \times m$ 的 0/1 网格。一次更新后，每个格子的新值由其旧状态的上下左右四邻格决定：值为 1 的邻格至少有两个，则新值为 1，否则为 0。越界邻格不计入。只进行一轮更新，$1 \le n, m \le 500$。

输入。第一行 n, m，随后每行 m 个 0 或 1。**输出**。更新后的网格。

输入输出样例

样例输入

```
2 3
1 0 1
0 1 0
```

样例输出

```
0 1 0
1 0 1
```

推演。左上格的两个有效邻格都是 0，所以变为 0；上方中间格的三个邻格都是 1，所以变为 1。所有计数都使用原数组 a，把答案放进 b。若直接覆盖 a，后面的格子会读到已经变化的数据。

正确性。程序对每格恰好检查四个方向，只有合法坐标参与计数，因此统计值与题意一致；写入 b 不改变 a，满足同步更新。时间和空间均为 $O(nm)$。

完整参考实现 02_grid.cpp

```cpp
#include <cstdio>
const int N = 505;
int a[N][N], b[N][N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int count = 0;
            for (int k = 0; k < 4; k++) {
                int x = i + dx[k], y = j + dy[k];
                if (x >= 0 && x < n && y >= 0 && y < m)
                    count += a[x][y];
            }
            b[i][j] = (count >= 2);
        }
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            printf("%d%c", b[i][j], j + 1 == m ? '\n' : ' ');
    return 0;
}
```

3.1.3 矩形相交的边界

真题练习：田地丈量：第 29 次，2023-03，第 1 题；把每块田与固定区域求交后累加（CSP202303A）。

两个与坐标轴平行的矩形分别为 $[x_1, x_2] \times [y_1, y_2]$ 和 $[u_1, u_2] \times [v_1, v_2]$，且各端点已经有序。相交宽度与高度为

$$w = \max(0, \min(x_2, u_2) - \max(x_1, u_1)), \qquad h = \max(0, \min(y_2, v_2) - \max(y_1, v_1)).$$

面积为 wh。例如 $[0,3] \times [0,2]$ 与 $[2,5] \times [1,4]$ 的交集宽、高都为 1，面积为 1。仅接触边界时面积为 0。若题目统计整数格点而非连续面积，端点规则不同，不能擅自加 1 或减 1。

3.1.4 日期模拟

真题练习：日期计算：第 5 次，2015-09，第 2 题；用月份天数表消耗年内日序号（CSP201509B）。

日期问题先确定历法。公历闰年的判定是：能被 400 整除，或者能被 4 整除但不能被 100 整除。月份长度用表保存，二月按闰年调整。把日期加一天时，先增加日；若超过当月天数，日改为 1 并增加月；若月变成 13，再进入下一年。每步只有一个明确定义的状态变化。

```cpp
bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}
int days(int y, int m) {
    int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    return d[m] + (m == 2 && leap(y));
}
```

3.1.5 自测与解答

练习 1。对只有一个格子的网格，原值为 1，更新后是多少？**解答**。为 0，因为没有有效邻格。它检查越界和最小规模。

练习 2。2000 年和 2100 年是否闰年？**解答**。2000 能被 400 整除，是闰年；2100 能被 100 整除但不能被 400 整除，不是闰年。因此两年的二月分别有 29 天和 28 天。

练习 3。矩形 $[0,2]\times[0,2]$ 与 $[2,4]\times[0,2]$ 的相交面积是多少？**解答**。宽度为 0，面积为 0；边接触不产生正面积。

练习 4。若网格更新两轮，是否可以一直只计算 b？**解答**。不可以。第一轮结束后应交换 a、b，使第二轮从第一轮结果出发；每轮仍必须完整覆盖 b 的全部位置。

4 练习题

4.1 原题单剩余三道第一题

先做原题单，再做知识点补充。真题速看未展开的三题如下，都保留认证次数、完整规则、样例与可提交程序。

4.2 小中大

第 16 次 CCF-CSP 认证，2019-03，第 1 题。小中大（原资料入口）。**题目与输入输出**。输入 $n \le 10^5$ 以及 n 个已按升序或降序排列的整数，可重复，绝对值不超过 10^7。输出最大值、中位数、最小值，空格分隔；整数不带小数，非整数保留 1 位小数。**读题思路**。已经有序，无需重排；升序和降序都能由首尾取得极值。奇数长度中位数是一项，偶数长度是中间两项均值。**做题思路与解答**。首尾取 min、max，中间位置按奇偶取值。先计算中位数的两倍 twice；偶数时直接输出 twice/2，否则输出 twice/2.0 保留一位。这使负半整数也正确，避免整数除法丢掉 0.5。

输入输出样例

样例输入

```
3
-1 2 4
```

样例输出

```
4 2 -1
```

输入输出样例

样例输入

```
4
-2 -1 3 4
```

样例输出

```
4 1 -2
```

复杂度与边界。读入时间 $O(n)$、数组空间 $O(n)$。可以只保留首尾及中间项把额外空间降为 $O(1)$。n=1 时三个输出值相同。

完整参考实现 53_small_middle_large.cpp

```cpp
#include <cstdio>
int a[100005];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int low = a[0] < a[n-1] ? a[0] : a[n-1];
    int high = a[0] > a[n-1] ? a[0] : a[n-1];
    long long twice = n%2 ? 2LL*a[n/2] : 1LL*a[n/2-1]+a[n/2];
    printf("%d ", high);
    if (twice % 2 == 0) printf("%lld", twice/2);
    else printf("%.1f", twice/2.0);
    printf(" %d\n", low);
    return 0;
}
```

4.3 数组推导

第 23 次 CCF-CSP 认证，2021-09，第 1 题。数组推导（原资料入口）。**题目与输入输出**。非负整数数组 A 的前缀最大值为 $B_i = \max(A_1, \ldots, A_i)$。已知单调不降的 B，求 A 的元素和最大值及最小值，各输出一行，最大值在前。输入 n、n 项 B；$1 \le n \le 100$，$0 \le B_i \le 10^5$。**读题思路**。B 是前缀最大值，不是前缀和。B 上升时当前 A 被迫等于新值；B 不变时 A 可小到 0。**做题思路与解答**。最大和为 $\sum B_i$，因为各项都可选为 B 对应值，且不能超过该值。最小和只保留第一项及严格上升处的 B 值，其他 A 取 0。上升时此前元素不可能提供新最大值，必须由当前 A 提供；这些强制项已足以实现全部 B，所以该和最小。

输入输出样例

样例输入

```
6
0 0 5 5 10 10
```

样例输出

```
30
15
```

输入输出样例

样例输入

```
7
10 20 30 40 50 60 75
```

样例输出

```
285
285
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$。B 首项可为 0，全 0 时两个答案都是 0。

完整参考实现 54_derive.cpp

```cpp
#include <cstdio>
int main() {
    int n, previous = 0; scanf("%d", &n);
    long long largest = 0, smallest = 0;
    for (int i = 0; i < n; i++) {
        int b; scanf("%d", &b);
        largest += b;
        if (i == 0 || b > previous) smallest += b;
        previous = b;
    }
    printf("%lld\n%lld\n", largest, smallest);
    return 0;
}
```

4.4 如此编码

第 27 次 CCF-CSP 认证，2022-09，第 1 题。如此编码（原资料入口）。**题目与输入输出。**每题有 $a_i \geq 2$ 个选项，正确选项 $0 \leq b_i < a_i$。设 $c_0 = 1, c_i = \prod_{j=1}^{i} a_j$，编码 $m = \sum_{i=1}^{n} c_{i-1} b_i$。输入 n、m 及 n 项 a，恢复并输出 n 项 b，空格分隔。$n \leq 20, c_n \leq 10^9, 0 \leq m < c_n$。**读题思路。**这是不同位使用不同进制的编码。题面给出长公式，但可以从最低位逐次取余，避免反复计算前缀积。**做题思路与解答。**对 a_1 取余，后面所有项都是 a_1 的倍数，所以 $b_1 = m \bmod a_1$。再令 $m = \lfloor m/a_1 \rfloor$，得到余下位的同类编码。依次取余和整除即可恢复全部 b。每步都唯一确定一位，最终得到原编码的唯一表示。

输入输出样例

样例输入

```
4 0
2 3 2 5
```

样例输出

```
0 0 0 0
```

输入输出样例

样例输入

```
7 23333
3 5 20 10 4 3 10
```

样例输出

```
2 2 15 7 3 1 0
```

复杂度与边界。时间 $O(n)$，额外空间 $O(1)$，可边读 a 边输出。m 为 0 时每位都为 0；余数不是输出的逆序。

完整参考实现 55_encoding.cpp

```cpp
#include <cstdio>
int main() {
    int n; long long m; scanf("%d%lld", &n, &m);
    for (int i = 0; i < n; i++) {
        int base; scanf("%d", &base);
        printf("%lld%c", m%base, i+1 == n ? '\n' : ' ');
        m /= base;
    }
    return 0;
}
```

4.5 基础统计与枚举复做

以下练习补足频率统计、相邻关系、贪心分段、浮点计算和重复状态。每道在线题附提交入口；教学例题与概念问答没有独立提交页面。

4.6 例题统计一组整数

题目。输入 n 个整数，输出总和、最小值、最大值，以及出现次数最多的值及其次数。若多个值并列，输出最小的值。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入。第一行 n，第二行 n 个整数。**输出**。第一行依次输出总和、最小值、最大值；第二行输出众数及出现次数。

输入输出样例

样例输入

```
6
3 -1 3 2 -1 5
```

样例输出

```
11 -1 5
-1 2
```

解答。总和与极值可边读边更新；最小值不能初始化为 0，否则全正数时会出错。众数作为本例的进阶要求：先调用 C 标准库 qsort 排序，再扫描每个相等值段。按值升序扫描且只在次数严格增加时更新答案，就保留了并列时较小的值。qsort 的比较函数返回负数、0 或正数，表示小于、等于或大于；不要用可能溢出的相减判断。扫描为 $O(n)$，空间为 $O(n)$，总时间由排序主导。C 标准未规定 qsort 的复杂度保证；需要明确的 $O(n \log n)$ 比较次数保证时，可使用第二篇讲解的 sort。初学时先掌握总和与极值，再学习众数部分。

完整参考实现 01_stats.cpp

```cpp
#include <cstdio>
#include <cstdlib>
#include <climits>
long long a[200005];
int compare(const void *p, const void *q) {
    long long x = *(const long long *)p;
    long long y = *(const long long *)q;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}
int main() {
    int n;
    scanf("%d", &n);
    long long sum = 0, mn = LLONG_MAX, mx = LLONG_MIN;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]); sum += a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
    }
    qsort(a, n, sizeof(a[0]), compare);
    long long mode = a[0];
    int best = 0, i = 0;
    while (i < n) {
        int j = i;
        while (j < n && a[j] == a[i]) j++;
        if (j - i > best) { best = j - i; mode = a[i]; }
        i = j;
    }
    printf("%lld %lld %lld\n%lld %d\n", sum, mn, mx, mode, best);
    return 0;
}
```

4.7 练习与解答

练习 1。数组长度为 10^5，每项不超过 10^9，总和是否可以用 int？**解答**。最大总和为 10^{14}，应使用 long long。若单项也可能相乘，还要检查乘法是否提前提升类型。

练习 2。对 $[4,4,4]$，上题输出什么？**解答**。第一行是 `12 4 4`，第二行是 `4 3`。这检查重复值与极值更新。

练习 3。有 10^5 次查询，每次扫描 10^5 个数求和，工作量约是多少？**解答**。约 10^{10} 次元素访问。若数组不变，应预处理前缀和；若存在修改，则先判断能否离线处理，不直接套静态前缀和。

4.8 基础题训练路线

按顺序打开做题链接，先根据本书的简述实现；输入细节和平台限制以原题为准。后面的逐题解答提供关键代码，并说明为什么这样统计正确。

真题与做题链接	认证届次	任务和训练目标
相反数（CSP201403A）	第 1 次，2014-03	统计互为相反数的数对；值域与不重复计数。

真题与做题链接	认证届次	任务和训练目标
相邻数对（CSP201409A）	第 2 次，2014-09	统计差的绝对值为 1 的数对；枚举无序对。
门禁系统（CSP201412A）	第 3 次，2014-12	依次输出当前编号是第几次出现；频率数组。
数列分段（CSP201509A）	第 5 次，2015-09	相邻相等的连续段数量；边界变化计数。
分蛋糕（CSP201703A）	第 10 次，2017-03	按顺序累积到重量阈值就分出；末段不足仍分出。
卖菜（CSP201809A）	第 14 次，2018-09	内部位置取三项均值，首尾取两项均值，整数截断。
报数（CSP201912A）	第 18 次，2019-12	四人轮流报数，7 的倍数或含 7 者跳过，统计各人跳过次数。
灰度直方图（CSP202104A）	第 22 次，2021-04	按灰度值计频率，输出完整值域（包括频率 0）。
归一化处理（CSP202206A）	第 26 次，2022-06	计算均值、总体方差及归一化值；double 与精度。
重复局面（CSP202305A）	第 30 次，2023-05	对每个棋盘局面输出此前累计出现次数；二维字符比较。

4.9 相反数与相邻数对：不重不漏

做题：相反数（第 1 次）（CSP201403A）、相邻数对（第 2 次）（CSP201409A）。**解答**。这两题数据小，读入数组后枚举 $i < j$ 即可。相反数检查 $a_i + a_j = 0$；相邻数对检查差为 1 或-1。每个无序对只枚举一次。题面保证输入值互不相同；若改成可重复，按值存在性计数便不等于按位置数对计数。

```
int answer = 0;
for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
        if (1LL*a[i]+a[j] == 0) answer++; // Opposite numbers.
        // Neighbor pairs: replace the condition above by
        // 1LL*a[i]-a[j] == 1 || 1LL*a[i]-a[j] == -1
    }
}
```

时间 $O(n^2)$，数组空间 $O(n)$。值域小且不重复时也可用存在性桶：相反数只检查正数 x 与-x，相邻数对只检查 x 与 x+1，这同样只计一次。

4.10 门禁系统与灰度直方图：同一种桶的两种输出

做题：门禁系统（第 3 次）（CSP201412A）、灰度直方图（第 22 次）（CSP202104A）。门禁题输入 n 及 n 次记录，每条是编号，输出每条编号在当前前缀中的出现次数；原题 $1 \le n \le 1000$，编

号在 1 到 n 之间。直方图题输入 n、m、L 和 $n \times m$ 个灰度值，灰度在 0 到 $L-1$ 之间，输出 L 个计数。**解答**。数组 cnt 全 0，每读入一个 x 就 `cnt[x]++`。门禁在更新后立即输出，直方图在全部更新后按 0 到 L-1 输出。

```
int cnt[1005] = {0};
for (int i = 0; i < n; i++) {
    int id; scanf("%d", &id);
    printf("%d%c", ++cnt[id], i+1 == n ? '\n' : ' ');
}
// Histogram core: read all pixels and do cnt[gray]++.
// Then print cnt[0], cnt[1], ..., cnt[L-1], including zeros.
```

每条记录只访问一次，时间线性；存储取决于值域，不是记录数量。多个测试实例时必须重新清零。

4.11 数列分段与分蛋糕：何时增加答案

做题：数列分段（第 5 次）（CSP201509A）、分蛋糕（第 10 次）（CSP201703A）。数列分段把相邻相等项放在同一段，输入 n 及数组，输出段数；n 至少为 1。**解答**。先有一段，每个 $a_i \neq a_{i-1}$ 的位置开启新段，答案为 1 加变化次数。

```
int segments = 1;
for (int i = 1; i < n; i++) if (a[i] != a[i-1]) segments++;
```

分蛋糕按输入次序累积正重量，达到 k 就交给一个人并从 0 重新累积；最后不足 k 但仍有蛋糕也交给一个人。输入 n、k 和各块重量，输出人数。

```
long long sum = 0; int people = 0;
for (int i = 0; i < n; i++) {
    sum += weight[i];
    if (sum >= k) { people++; sum = 0; }
}
if (sum > 0) people++;
```

达到阈值就分出，符合题目指定流程；若从最大化可分人数理解，正重量条件下延后分出只会消耗更多块，不能增加后续可分段数。两题时间 $O(n)$；读入可流式处理，额外空间 $O(1)$。

4.12 卖菜：首尾与同步更新

做题：卖菜（第 14 次）（CSP201809A）。输入至少 2 个原价格；内部新价格是左、中、右原价格之和除以 3 向下取整，两端是两个相邻原价格之和除以 2。输出新价格序列。**解答与关键代码**。全部原值先读入 a，再按规则输出，或者写入新数组 b；不要修改 a 后再算后面的均值。价格非负，整数除法正好完成向下取整。

```
printf("%d", (a[0]+a[1])/2);
for (int i = 1; i+1 < n; i++) printf(" %d", (a[i-1]+a[i]+a[i+1])/3);
printf(" %d\n", (a[n-2]+a[n-1])/2);
```

时间 $O(n)$，空间 $O(n)$。n=2 时内部循环不执行，两个答案都是两项均值。

4.13 报数：数字与轮次分开

做题：报数（第 18 次）（CSP201912A）。四人依次轮流报自然数，从 1 开始；7 的倍数或十进制含 7 时跳过并记录当前人的跳过次数。直到成功报出 n 个数，输出四人的跳过次数（每人一行）。**解答**。变量 x 记录当前自然数，done 只统计成功次数；每处理一个 x，轮次都会推进，包括被跳过的数。当前人是 $(x-1) \bmod 4$。

```
bool skip(int x) {
    if (x % 7 == 0) return true;
    while (x) { if (x % 10 == 7) return true; x /= 10; }
    return false;
}
// int cnt[4]={0}, done=0;
// for (int x=1; done<n; x++)
//     if (skip(x)) cnt[(x-1)%4]++; else done++;
```

不要在跳过时停留在同一个人，也不要把 x 直接限制为 n。若实际处理 T 个数，时间 $O(T \log T)$，空间 $O(1)$。

4.14 归一化处理：均值、方差与浮点除法

做题：归一化处理（第 26 次）（CSP202206A）。给定 n 个整数，计算均值 $\mu = \frac{1}{n}\sum a_i$、总体方差 $D = \frac{1}{n}\sum(a_i-\mu)^2$，再逐行输出 $(a_i-\mu)/\sqrt{D}$；原题保证方差非零。**解答与关键代码**。使用 double 累加，先求均值，再求平方偏差和。不要误用样本方差的分母 n-1，也不要把平方根漏掉。

```
double sum = 0, variance = 0;
for (int i = 0; i < n; i++) sum += a[i];
double mean = sum/n;
for (int i = 0; i < n; i++) {
    double d = a[i]-mean; variance += d*d;
}
variance /= n;
for (int i = 0; i < n; i++) printf("%.10f\n", (a[i]-mean)/sqrt(variance));
```

需包含 <cmath> 并调用相应 sqrt。时间 $O(n)$，数组空间 $O(n)$。自编变式若允许全部相等，必须规定 D=0 时的输出，不能直接除以 0。

4.15 重复局面：不用 STL 也能比较字符串

做题：重复局面（第 30 次）（CSP202305A）。输入 n 个 8×8 字符棋盘，按输入顺序输出当前棋盘在截至此刻的全部棋盘中出现了几次。原题 $n \le 100$。**解答**。每个棋盘压成一个长度 64 的字符数组，末尾加 0；枚举此前及当前所有棋盘，用 strcmp 判断完全相同。n 很小，$O(64n^2)$ 足够，无需把 map 作为第一题的先修条件。

```
char board[105][65];
// Read each row into char row[9] with scanf("%8s", row),
// then copy its 8 characters to board[i]+8*r.
// Set board[i][64]=0 before strcmp.
for (int i = 0; i < n; i++) {
    int count = 0;
    for (int j = 0; j <= i; j++)
        if (strcmp(board[i], board[j]) == 0) count++;
    printf("%d\n", count);
}
```

包含 <cstring>。比较必须覆盖全部 64 格，不能只用某一个格子或棋子数量作为身份。第二篇学完 map 后，可把完整棋盘串作为键重新实现。

4.16 历年考点补充：数组与二维枚举

下列四题依据原资料的历年考点表补充，不冒充原题单中的八个入口。

4.17 图像旋转

第 4 次 CCF-CSP 认证，2015-03，第 1 题。图像旋转（CSP201503A）。**题目与输入输出。**输入一个 $n \times m$ 矩阵，将它逆时针旋转 90° 后输出。第一行 n、m，随后 n 行每行 m 个整数；输出 m 行，每行 n 个整数。$1 \le n, m \le 1000$，元素在 0 到 1000 之间。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**圈出“逆时针”和“输出 m 行 n 列”。矩阵不一定是正方形，不能把行列长度混用。百万个 int 约 3.8MiB，可以存整个原矩阵。**做题思路与解答。**原矩阵第 j 列从上到下，会成为新矩阵第 $m-1-j$ 行（0 下标）。因此按列从右到左、每列从上到下输出即可；无需再开一张新矩阵。映射式为 $b[m-1-j][i] = a[i][j]$。每个输入元素恰好输出一次，所以既没有漏项，也没有重复。

输入输出样例

样例输入

```
2 3
1 5 3
3 2 4
```

样例输出

```
3 4
5 2
1 3
```

复杂度与边界。时间 $O(nm)$，空间 $O(nm)$。用 1×3 和 3×1 检查方向；每行最后一个值后输出换行。

完整参考实现 35_rotate.cpp

```cpp
#include <cstdio>
int a[1005][1005];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
    for (int j = m - 1; j >= 0; j--) {
        for (int i = 0; i < n; i++)
            printf("%d%c", a[i][j], i + 1 == n ? '\n' : ' ');
    }
    return 0;
}
```

4.18 数位之和

第 6 次 CCF-CSP 认证，2015-12，第 1 题。数位之和（CSP201512A）。**题目与输入输出。**输入一个整数 n，输出其十进制各位数字之和。$0 \le n \le 10^9$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**任务是求数字之和，不是数位数量，也不是把各位拼成新数。输入允许 0；不必为了拆位把整数转换为字符串。**做题思路与解答。**每轮用 $n \bmod 10$ 取个位，再用整数除法 $n/10$ 删除个位；不断累加。每位恰好取一次，得到所求总和。do-while 会对 0 执行一次，结果仍为 0；普通 while 配合总和初值 0 也正确。

输入输出样例

样例输入

```
20151220
```

样例输出

```
13
```

复杂度与边界。若十进制位数为 d，时间 $O(d)$，空间 $O(1)$。10^9 只有 10 位，和最多 81（10^9 本身为 1）；int 足够。

完整参考实现 36_digits.cpp

```cpp
#include <cstdio>
int main() {
    int n, answer = 0; scanf("%d", &n);
    do { answer += n % 10; n /= 10; } while (n != 0);
    printf("%d\n", answer);
    return 0;
}
```

4.19 折点计数

第 7 次 CCF-CSP 认证，2016-04，第 1 题。折点计数（CSP201604A）。**题目与输入输出。**有 n 天销售量 $a_1, \ldots, a_n$，相邻两天的销售量不同。一个内部位置的值严格大于左右两项，或严格小于左右两项，称为折点。输入 n 及 n 项销售量，输出折点数。$1 \le n \le 1000$，销售量在 0 到 10000 之间。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路。**折点表示趋势从升变降或从降变升。第一天和最后一天没有两侧邻居，不能计入；不相邻的位置可以取相同值。**做题思路与解答。**从第二项到倒数第二项检查“同时大于左右”或“同时小于左右”。每个内部位置独立判断一次，恰好覆盖全部折点。也可检查相邻差的符号是否不同，但直接写大小关系更直观，且没有乘法溢出的隐患。

输入输出样例

样例输入

```
7
5 4 1 2 3 6 4
```

样例输出

```
2
```

复杂度与边界。时间 $O(n)$，空间 $O(n)$。n 为 1 或 2 时循环不执行，答案为 0。严格单调序列也没有折点。

完整参考实现 37_turns.cpp

```cpp
#include <cstdio>
int a[1005];
int main() {
    int n, answer = 0; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 1; i + 1 < n; i++) {
        if ((a[i] > a[i-1] && a[i] > a[i+1]) ||
            (a[i] < a[i-1] && a[i] < a[i+1])) answer++;
    }
    printf("%d\n", answer);
    return 0;
}
```

4.20 仓库规划

第 32 次 CCF-CSP 认证，2023-12，第 1 题。仓库规划（CSP202312A）。**题目与输入输出**。n 个仓库各有 m 维坐标。只有 j 的每一维都严格大于 i，j 才是 i 的上级；若有多个，取编号最小的 j，否则为 0。输入 n、m，再输入 n 行坐标；按编号顺序输出各仓库上级，一行一个。$1 \le n \le 1000$，$1 \le m \le 10$，各坐标绝对值不超过 10^6。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。关注“每一维”“严格大于”“编号最小”。输出不是最近仓库，也不是坐标最小仓库。$n^2 m \le 10^7$，直接枚举候选可行，不必急着排序或使用 STL。**做题思路与解答**。对每个 i，按编号递增枚举 j；逐维检查，一旦出现 $a[j][k] \le a[i][k]$ 便拒绝。第一个全部通过的 j 就是编号最小答案，可以退出候选循环。自身必然无法通过严格比较。遍历所有候选保证无遗漏，升序与首次接受保证最小编号。

输入输出样例

样例输入

```
4 2
0 0
-1 -1
1 2
0 -1
```

样例输出

```
3
1
0
3
```

复杂度与边界。时间 $O(n^2 m)$，空间 $O(nm)$。相等坐标不是上级；无上级时保留初值 0。

完整参考实现 39_warehouses.cpp

```cpp
#include <cstdio>
int a[1005][15];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++)
        for (int k = 0; k < m; k++) scanf("%d", &a[i][k]);
    for (int i = 1; i <= n; i++) {
        int answer = 0;
        for (int j = 1; j <= n; j++) {
            bool upper = true;
            for (int k = 0; k < m; k++) {
                if (a[j][k] <= a[i][k]) { upper = false; break; }
            }
            if (upper) { answer = j; break; }
        }
        printf("%d\n", answer);
    }
    return 0;
}
```

第二部分　第二题：基础算法与 STL

本篇学习如何利用结构减少重复计算。第一节按算法模型展开，第二节详细讲解 STL 工具，第三节用五道认证真题检验建模，最后汇总考点并完成专题练习。即使暂时不用 STL，也应理解前缀和、双指针、二分和 DP 的原理。

1　基础算法与 STL 使用

1.1　先选算法，再选工具

数据结构负责保存和查询状态，算法负责说明为什么这样处理正确。不要只根据题名猜容器：已有排序的两组数据可以双指针扫描，不一定需要 map；静态区间和优先用前缀和，不必上复杂树结构。普通数组、结构体和独立函数仍是本篇实现的主体。

需求	算法或工具	在本篇就近练习的题目
反复查询静态区域总和	前缀和、二维容斥	邻域均值（CSP202104B）；梦境巡查（CSP202412B）。
批量修改区间或矩形	差分、最后恢复	画图（CSP201409B）；第二节后练习。
答案有单调可行性	二分答案、判定函数	机器人宿管（CSP202605B）；垦田计划（CSP202303B）。
两组已排序序列	双指针归并、区间扫描	稀疏向量（CSP202006B）；买菜（CSP201809B）。
重复选择，资源恰好用完	完全背包	机器人饲养指南（CSP202503B）。
可达性或最短跳数	DFS、BFS、状态标记	机器人复健指南（CSP202506B）。
按频率和编号排序	次数桶、结构体、sort	数字排序（CSP201503B）。
维护单词集合与交并	set、map、规范化	相似度计算（CSP202403B）。

已有原题能在曙梦 OJ 练习时优先使用该入口；原资料中的全排列、最长上升子序列、01 背包等专项题保留洛谷补充链接。专题题单目录集中放在附录，各算法学习时可以直接打开旁边的具体题目。

1.2 C 风格程序中怎样使用 STL

STL 是 C++ 标准库的一部分，可以与 scanf/printf、普通数组和结构体一起使用。按 C++17 编译，写清需要的头文件。数组排序可直接使用 `std::sort(a,a+n)`；不需要把已有数组先复制成 vector。比较函数写成独立函数；迭代器写明类型。本篇用到的接口会在第二节逐个说明。

```cpp
#include <cstdio>
#include <algorithm>
struct Student { int id, score; };
bool less_by_score(const Student &a, const Student &b) {
    if (a.score != b.score) return a.score > b.score;
    return a.id < b.id;
}
// After declaring array students:
// std::sort(students, students+n, less_by_score);
```

以上是比较函数片段，Student 必须先定义，例如 `struct Student{int id,score;};`。同分时的比较规则来自题面；不能把 `<=` 写进严格比较函数。

1.3 前缀和与差分

真题练习：邻域均值：第 22 次，2021-04，第 2 题；裁剪边界后查二维矩形和；第三节给完整解答（CSP202104B）。

真题练习：梦境巡查：第 36 次，2024-12，第 2 题；前缀、后缀最大需求，第五节给完整实现（CSP202412B）。

真题练习：画图：第 2 次，2014-09，第 2 题；可用二维差分求矩形覆盖后统计非零格子（CSP201409B）。

1.3.1 一维前缀和的推导

定义 $pre[0]=0$，$pre[i]=a_1+\cdots+a_i$。则

$$pre[i]=pre[i-1]+a_i,\qquad \sum_{i=l}^{r}a_i=pre[r]-pre[l-1].$$

原因是前一个总和包含 1 到 r，减去 1 到 $l-1$ 后只剩所需区间。预处理 $O(n)$，每次查询 $O(1)$。该方法适合**原数组不变**的多次查询。

1.3.2 例题静态区间求和

同类练习（在线做题）：P8218 【深进 1. 例 1】求区间和。

使用一维前缀和；每次查询计算右端前缀减去左端前一项。

输入长度为 n 的数组和 q 次闭区间查询，每次输出区间和。$1\le n,q\le 2\times 10^5$，$|a_i|\le 10^9$，保证 $1\le l\le r\le n$。第一行 n,q，第二行数组，随后 q 行各给出 l,r。

输入输出样例

样例输入

```
5 3
2 -1 3 4 5
1 3
2 4
5 5
```

样例输出

```
4
6
5
```

前缀数组为 $[0,2,1,4,8,13]$，查询 $[2,4]$ 得 $8-2=6$。0 号位置让 $l=1$ 的查询无需特殊判断。使用 long long 存前缀和。总复杂度 $O(n+q)$，空间 $O(n)$。

完整参考实现 05_prefix.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long pre[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        long long x;
        scanf("%lld", &x);
        pre[i] = pre[i - 1] + x;
    }
    while (q--) {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%lld\n", pre[r] - pre[l - 1]);
    }
    return 0;
}
```

1.3.3 二维前缀和与容斥

设 $S[i][j]$ 为从 $(1,1)$ 到 (i,j) 的矩形和，0 行和 0 列为 0：

$$S[i][j]=S[i-1][j]+S[i][j-1]-S[i-1][j-1]+a[i][j].$$

左上角被前两项算了两次，因此减一次。查询左上 (x_1,y_1)、右下 (x_2,y_2) 的矩形：

$$S[x_2][y_2]-S[x_1-1][y_2]-S[x_2][y_1-1]+S[x_1-1][y_1-1].$$

例如矩阵 $\begin{pmatrix}1&2&3\\4&5&6\end{pmatrix}$ 的第 2–3 列总和是 $2+3+5+6=16$；用公式得到 $21-0-5+0=16$。预处理 $O(nm)$，每个查询 $O(1)$。

1.3.4 差分把区间修改变成端点修改

定义 $d[1]=a[1]$，$d[i]=a[i]-a[i-1]$。原数组是差分数组的前缀和。给 $[l,r]$ 每个元素加 v，只需

$$d[l]\ +\!=v,\qquad d[r+1]\ -\!=v.$$

前一个修改让从 l 开始的前缀和都增加 v，后一个修改从 r+1 起抵消。必须为 n+1 位置预留空间。该做法适合执行全部修改后再求最终结果。

1.3.5 例题批量区间加法

同类练习（在线做题）：P2367 语文成绩。

差分恢复全部成绩后，原题还要取最小值，而不是输出整个数组。

输入长度为 n 的数组，执行 q 次操作，每次给闭区间 $[l,r]$ 加 v，最后输出整个数组。$1\le n,q\le 2\times 10^5$，初始值和修改值绝对值不超过 10^9。

输入输出样例

样例输入

```
5 2
1 2 3 4 5
2 4 3
1 5 -1
```

样例输出

```
0 4 5 6 4
```

先用初始数组构造差分。所有修改都可叠加，最后做一次前缀累加。总时间 $O(n+q)$，空间 $O(n)$。上例第一轮得到 $[1,5,6,7,5]$，第二轮整体减 1。

完整参考实现 06_difference.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long a[N], d[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &a[i]);
        d[i] = a[i] - a[i - 1];
    }
    while (q--) {
        int l, r; long long v;
        scanf("%d%d%lld", &l, &r, &v);
        d[l] += v; d[r + 1] -= v;
    }
    long long x = 0;
    for (int i = 1; i <= n; i++) {
        x += d[i];
        printf("%lld%c", x, i == n ? '\n' : ' ');
    }
    return 0;
}
```

1.3.6 二维差分的四个角

给矩形 $[x_1, x_2] \times [y_1, y_2]$ 加 v：

```
d[x1][y1] += v;
d[x2+1][y1] -= v;
d[x1][y2+1] -= v;
d[x2+1][y2+1] += v;
```

随后对 d 求二维前缀和，就得到每个格子的增量。左上角启动影响，下方和右方分别取消，右下角因取消两次再补一次。若有初始矩阵，可先恢复增量再加到初值上，或按 $d[i][j] = a[i][j] - a[i-1][j] - a[i][j-1] + a[i-1][j-1]$ 建初始差分。

1.3.7 自测与解答

练习 1。数组 $[3, -2, 7]$ 的前缀和与差分是什么？**解答**。前缀和含 0 号为 $[0, 3, 1, 8]$；差分为 $[3, -5, 9]$。注意差分可以为负。

练习 2。在全 0 的 3×3 网格中，给左上 2×2 加 2，再给右下 2×2 加 3，结果是什么？**解答**。

$$\begin{pmatrix} 2 & 2 & 0 \\ 2 & 5 & 3 \\ 0 & 3 & 3 \end{pmatrix}.$$

中心格被覆盖两次，得到 5。按四角更新再恢复，应得到同样矩阵。

练习 3。修改和查询交替出现，是否仍能用上述普通差分每次 $O(1)$ 回答区间和？**解答**。不能。普通差分方便累计修改，但恢复全部值需要一次扫描。需要重新分析是否可离线，或使用树状数组等动态结构，不能隐瞒恢复成本。

1.3.8 原题单差分应用：阈值与检测时刻

非零段划分（第 23 次，2021-09）把“一个位置何时成为段开头”转成阈值区间贡献；出行计划（第 25 次，2022-03）把“计划允许的检测时刻”转成整数区间贡献。本篇练习给出公式、样例和完整代码。两题都先推导区间端点，再做差分，不能从题名直接套模板。

1.4 二分与双指针

真题练习：机器人宿管：第 42 次，2026-05，第 2 题；人数越多越难满足天数，二分最后可行人数；第五节给解答（CSP202605B）。

1.4.1 二分依赖单调性

真题练习：垦田计划：第 29 次，2023-03，第 2 题；天数越大所需资源越少，找第一个可行天数，真题速看给解答（CSP202303B）。

二分的对象可以是有序数组，也可以是答案。核心是判定结果沿某个方向只变化一次。例如长度 x 能切出至少 k 段木头，那么更短的正整数长度也能做到；可行区间是前缀。先写出“真、真、假、假”的顺序，再选择搜索第一个真或最后一个真。

在闭区间 $[lo, hi]$ 搜索最后一个可行值时，取上中位数 $mid = lo + (hi - lo + 1)/2$。若 mid 可行，令 lo=mid；否则 hi=mid-1。上中位数保证剩下两个元素时仍能缩小区间。每步至少排除一个候选，区间最终只剩答案。

1.4.2 例题切木条的最大整数长度

同类练习（在线做题）：P2440 木材加工。

用“能切出的段数不少于所需数量”作判定，找最后一个可行长度。

有 n 根木条，第 i 根长度为 a_i。每根可切成若干长度相同的整段，剩余部分丢弃，不能拼接不同木条。至少要得到 k 段，求最大正整数段长；若连长度 1 都无法满足，输出 0。$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^9$，$1 \le k \le 10^{12}$。输入第一行 n、k，第二行各长度。

输入输出样例

样例输入

```
4 7
20 15 10 17
```

样例输出

```
7
```

长度 7 时可得 $2 + 2 + 1 + 2 = 7$ 段；长度 8 时只有 $2 + 1 + 1 + 2 = 6$ 段。判定函数为 $\sum_i \lfloor a_i/x \rfloor \ge k$，随 x 增大计数不增。0 作为“无正整数解”的哨兵，不传入判定函数，因此不会除以 0。复杂度 $O(n \log \max a_i)$，空间 $O(n)$。

完整参考实现 07_wood.cpp

```cpp
#include <cstdio>
const int N = 200005;
int n;
long long k, a[N];
int check(long long len) {
    long long count = 0;
    for (int i = 0; i < n; i++) {
        count += a[i] / len;
        if (count >= k) return 1;
    }
    return 0;
}
int main() {
    scanf("%d%lld", &n, &k);
    long long lo = 0, hi = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]);
        if (a[i] > hi) hi = a[i];
    }
    while (lo < hi) {
        long long mid = lo + (hi - lo + 1) / 2;
        if (check(mid)) lo = mid;
        else hi = mid - 1;
    }
    printf("%lld\n", lo);
    return 0;
}
```

1.4.3 跳石头中的判定建模

同类练习（在线做题）：P2678 跳石头。

检查两岸都被保留；可行性判断必须覆盖最后一段距离。

若目标是让相邻保留石头的最小距离尽量大，可以二分距离 x。判定时从起点向后扫描，若当前石头与上次保留位置距离小于 x，则至少需要移走其中一块。通常贪心优先移走当前中间石头，以保留更靠前的落点，为后续留下空间。计数的含义应是**达到距离要求至少需移走的数量**，不是最多能移走多少。

终点不可移走，需要单独验证末段，或者采用已证明等价的计数处理。不要只在中间石头上检查距离。二分答案的困难往往在判定函数的正确性，而不是二分循环本身。

1.4.4 双指针如何减少重复枚举

双指针有两种常见模型。**两个有序序列的归并**：比较两组当前位置，下标或结束时间较小的一方先移动；它不可能再和对方后续更大的位置匹配。**同一数组上的滑动窗口**：右端逐次扩大，违反条件时左端向右收缩。两者都需要证明“已丢弃的部分不可能再影响后续答案”，不能仅因为写了两个下标就称为线性算法。两个指针不后退时，总移动至多 $2n$ 次；内层 while 的各次执行累计起来是 n 级别，因此嵌套循环仍可能是 $O(n)$。每次移动和条件检查还应是 $O(1)$，若检查内部重新扫描窗口，则不再是线性。

```
int i = 0, j = 0;
while (i < na && j < nb) {
    if (a[i].index < b[j].index) i++;
    else if (a[i].index > b[j].index) j++;
    else { /* process this matching index */ i++; j++; }
}
```

上面要求每组下标严格递增；若同一下标可以重复，必须先合并或按等值组计算贡献。稀疏向量题面已经保证严格递增，直接逐项处理即可。

真题练习：买菜：第 14 次；较早结束的区间先移动（CSP201809B）。

真题练习：稀疏向量：第 19 次；下标小的一项先移动，相等时累加并同时移动（CSP202006B）。

设右端点逐渐增加，若满足条件的最小左端点不会向左移动，就可保存它而不重复扫描。两个指针分别最多移动 n 次，总移动次数为 $O(n)$。有两层循环不代表一定是 $O(n^2)$，要统计指针在全程移动多少次。

1.4.5 例题乘积小于 k 的连续区间个数

给定 n 个**正整数**，统计乘积严格小于 k 的非空连续子数组数量。$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^6$，$0 \le k \le 10^9$。输入第一行 n、k，第二行数组。

输入输出样例

样例输入

```
4 100
10 5 2 6
```

样例输出

```
8
```

合法区间包括 4 个单元素、$[10,5]$、$[5,2]$、$[2,6]$、$[5,2,6]$，共 8 个。维护窗口乘积，每加入右端点，就在乘积过大时删除左端点。缩小后，以当前右端点结尾的合法区间有 $right - left + 1$ 个。

所有数至少为 1，因此缩短区间不会使乘积增大，合法左端点形成后缀。若允许负数，这个性质会失效；若允许 0，除法维护也需另外处理。k 不大于 1 时没有合法非空区间。按本题约束，乘入新元素前乘积小于 10^9，乘后小于 10^{15}，long long 足够。时间 $O(n)$，空间 $O(n)$。

完整参考实现 08_window.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long a[N];
int main() {
    int n; long long k;
    scanf("%d%lld", &n, &k);
    for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
    if (k <= 1) { puts("0"); return 0; }
    long long product = 1, answer = 0;
    int left = 0;
    for (int right = 0; right < n; right++) {
        product *= a[right];
        while (left <= right && product >= k) product /= a[left++];
        answer += right - left + 1;
    }
    printf("%lld\n", answer);
    return 0;
}
```

1.4.6 差值数对与重复元素

同类练习（在线做题）：P1102 A-B 数对：排序后按相同值分组，注意重复元素贡献。

给定正整数差 c，若已排序，可对每个值 x 查找 x+c 的出现次数。使用 upper_bound 与 lower_bound 之差可自然处理重复。若 x 有 p 个，x+c 有 q 个，贡献是 pq，而不是 1。例如 $[1,1,3,3,3]$ 且 c=2，答案是 $2\times 3=6$。

当 c=0 时需先确认题目问有序数对还是无序数对，是否允许同一位置使用两次。无序且不同位置的同值组贡献为 $p(p-1)/2$，不能沿用 c>0 的计数公式。

1.4.7 自测与解答

练习 1。木条 $[1,1]$，k=3，答案是什么？**解答**。0。最大段长哨兵必须覆盖无解情况。

练习 2。窗口题数组 $[1,1,1]$，k=2，答案是多少？**解答**。全部 6 个非空连续区间都合法，分别贡献 1、2、3。k=1 时答案为 0。

练习 3。闭区间最后可行值搜索中，lo=2、hi=3，若使用下中位数且判定为真会怎样？**解答**。mid=2，更新 lo=2 后区间不变，可能死循环。应取上中位数，或采用另一套能证明缩小区间的写法。

1.5 贪心与基础数学

真题练习：分蛋糕：第 10 次，2017-03，第 1 题；正重量累积到阈值后立即分出，第一篇练习给解答（CSP201703A）。

真题练习：因子化简：第 32 次，2023-12，第 2 题；反复试除并记录完整指数；第三节给解答（CSP202312B）。

真题练习：矩阵运算：第 30 次，2023-05，第 2 题；用结合律先算小矩阵，降低乘法次数（CSP202305B）。

1.5.1 贪心需要证明

贪心是在当前状态下确定一个选择并且不回退。它成立需要证明：至少存在一个最优解包含这次选择。常用方法是交换论证，即把某个最优方案换成包含该选择的方案，而目标值不变差。找到几个正确样例不是证明。

例如按价值最大的物品优先装背包并不总是正确：容量 4，物品的（体积，价值）分别为 $(3,5),(2,4),(2,4)$，先拿价值 5 的物品只能得到 5，拿后两件可得 8。基础 01 背包通常需要 DP，不能仅因“选大的”直观就使用贪心。

1.5.2 例题合并石堆的最小代价

有 n 堆石子，每次可以任意选两堆合并，代价等于两堆石子数之和。求合成一堆的最小总代价。本题**没有相邻限制**。$1 \le n \le 2 \times 10^5$，每堆石子数在 1 到 10^9 之间。输入第一行 n，第二行各堆大小。

输入输出样例

样例输入

```
4
1 2 3 4
```

样例输出

```
19
```

先合并 1 和 2，代价 3；再合并 3 和 3，代价 6；最后合并 4 和 6，代价 10。总计 19。

为什么取最小两堆。合并过程可以看成二叉树，每个原始堆是叶子；总代价等于每个叶子权值乘以其深度之和。在固定树形中，较小权值放到较深位置不会使代价增大。选择一对最深的兄弟叶子，可通过交换把最小的两个权值放到那里。因此存在一个最优方案先合并最小两堆；把它们缩成一堆后，剩下同类子问题。重复此选择即可。

用小根堆维护所有堆，每次取两个最小值再插入它们的和。时间 $O(n \log n)$，空间 $O(n)$。只有一堆时无需合并，答案为 0。

完整参考实现 09_merge.cpp

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <functional>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    priority_queue<long long, vector<long long>, greater<long long> > q;
    for (int i = 0; i < n; i++) {
        long long x;
        scanf("%lld", &x); q.push(x);
    }
    long long answer = 0;
    while (q.size() > 1) {
        long long x = q.top(); q.pop();
        long long y = q.top(); q.pop();
        answer += x + y; q.push(x + y);
    }
    printf("%lld\n", answer);
    return 0;
}
```

1.5.3 中位数最小化绝对距离和

若在数轴上选位置 x，使 $\sum_i |a_i - x|$ 最小，排序后取中位数即可。把最左和最右配对：对任何位于两者之间的 x，两项距离和都是两端距离；离开该区间只会增大。对称配对后，所有区间的公共部分包含中位数。偶数个元素时，中间两数之间的任意位置都可达到最小值。

例如 $[1, 2, 9]$ 取 x=2，距离和为 8；取平均数 4，距离和为 10。平均数与中位数解决的目标不同。

1.5.4 质因数分解与溢出检查

对正整数 x，从 2 开始试除。若 d 整除 x，反复除去并统计指数。循环条件可写 `d <= x/d`，避免计算 d^2 溢出。循环结束后若 x>1，它是最后一个质因子。1 没有质因子。

```cpp
// prime[] and exponent[] each need at least 64 elements.
int factorize(long long x, long long prime[], int exponent[]) {
    int total = 0;
    for (long long d = 2; d <= x / d; d++) {
        if (x % d != 0) continue;
        int count = 0;
        do { x /= d; count++; } while (x % d == 0);
        prime[total] = d; exponent[total++] = count;
    }
    if (x > 1) { prime[total] = x; exponent[total++] = 1; }
    return total;
}
```

最坏试除次数为 $O(\sqrt{x})$。如果要分解很多较大的数，应重新估计总复杂度，而不是只看一次分解。

1.5.5 矩阵运算先看维度

若 A 是 $n\times k$，B 是 $k\times m$，则 $C=AB$ 是 $n\times m$，有 $C_{ij}=\sum_{t=1}^{k}A_{it}B_{tj}$，朴素复杂度 $O(nkm)$。矩阵乘法一般不可交换。若题意出现转置或连乘，应先写出各中间矩阵的维度与计算量，再决定结合顺序。计算量可以因结合顺序变化而显著不同，但结果在精确算术下符合结合律。

1.5.6 自测与解答

练习 1。三堆 $[2,2,8]$ 的最优代价是多少？**解答**。先合并 2 和 2 得到 4，代价 4；再与 8 合并代价 12，总计 16。若先合并 2 和 8 则为 22。

练习 2。分解 360。**解答**。$360=2^3\cdot 3^2\cdot 5$。反复整除才能记录完整指数。

练习 3。A 为 10×100，B 为 100×5，C 为 5×50，先算哪一对更省？**解答**。$(AB)C$ 的乘加规模为 $10\cdot 100\cdot 5+10\cdot 5\cdot 50=7500$；$A(BC)$ 为 $100\cdot 5\cdot 50+10\cdot 100\cdot 50=75000$，前者更省。

1.6 搜索与剪枝

真题练习：机器人复健指南：第 38 次，2025-06，第 2 题；马步八方向、最短步数与去重，第五节给解答（CSP202506B）。

1.6.1 搜索状态和递归职责

写递归前先明确：一次调用负责哪一步、已经作出哪些选择、接下来有哪些合法选择、什么时候记录答案。回溯是撤销本次选择，让下一条分支从同一个父状态开始。常见错误是撤销不完整，导致不同分支互相污染。

1.6.2 例题按字典序输出全排列

同类练习（在线做题）：P1706　全排列问题。

原题要求每个数占 5 个字符宽度，应使用 `printf("%5d", x)`，不能直接沿用单空格输出。

输入 $1\le n\le 8$，输出 1 到 n 的所有排列，每行一个，按字典序升序。

输入输出样例

样例输入

```
3
```

样例输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

path[0..depth-1] 保存已选的前缀，used[x] 标记 x 是否已在前缀中。每层从小到大尝试尚未使用的数，输出顺序自然为字典序。每个排列对应唯一一条长度 n 的路径，因此不重不漏。输出本身有 $n \cdot n!$ 个数字量级，时间 $O(n \cdot n!)$，递归与标记空间 $O(n)$。

完整参考实现 10_permutations.cpp

```cpp
#include <cstdio>
int n, path[10], used[10];
void dfs(int depth) {
    if (depth == n) {
        for (int i = 0; i < n; i++)
            printf("%d%c", path[i], i + 1 == n ? '\n' : ' ');
        return;
    }
    for (int x = 1; x <= n; x++) {
        if (used[x]) continue;
        used[x] = 1; path[depth] = x;
        dfs(depth + 1);
        used[x] = 0;
    }
}
int main() {
    scanf("%d", &n);
    dfs(0);
    return 0;
}
```

1.6.3 剪枝必须有排除理由

同类练习（在线做题）：P1219 八皇后 Checker Challenge。

逐行搜索并标记列与两条对角线；原题输出前三个方案以及方案总数。

剪枝是提前排除不可能产生合法答案或不可能改善答案的分支。可行性剪枝例如皇后问题中已占用的列与对角线；最优性剪枝例如当前代价已不小于已知最优值，且所有后续代价非负。若后续允许负代价，后一判断就可能失效。

n 皇后按行放置时，记录列 c、对角线 $r+c$、另一方向 $r-c+n$ 是否已使用。每行只放一个皇后，所以行冲突由递归结构自动排除。剪枝减少搜索量，但不意味着最坏复杂度必然变成多项式。

1.6.4 例题迷宫最短步数

同类练习（在线做题）：P1443 马的遍历。

将四方向改成马的八种跳法，按原题输出整个距离矩阵；不是只输出终点距离。

输入 $n \times m$ 字符网格，点号为可走格，井号为障碍。每步上下左右移动一格，求左上角到右下角的最少步数；不可达输出-1。$1 \le n, m \le 500$，起终点也可能是障碍。第一行 n、m，随后 n 行长度为 m 的字符串。

输入输出样例

样例输入

```
3 4
....
.##.
....
```

样例输出

```
5
```

解答。所有移动代价相等，使用 BFS。起点距离为 0，邻居距离为当前距离加 1；入队时就记录距离，确保同一格最多入队一次。队列按距离层次扩展，因此首次到达就是最短距离。注意这里的“步数”不包含起点，单格可走迷宫答案为 0。时间与空间均为 $O(nm)$。

完整参考实现 11_bfs.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 505;
char g[N][N];
int d[N][N], qx[N * N], qy[N * N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) {
        scanf("%504s", g[i]);
        fill(d[i], d[i] + m, -1);
    }
    int head = 0, tail = 0;
    if (g[0][0] == '.') {
        d[0][0] = 0; qx[tail] = 0; qy[tail++] = 0;
    }
    while (head < tail) {
        int x = qx[head], y = qy[head]; head++;
        for (int k = 0; k < 4; k++) {
            int a = x + dx[k], b = y + dy[k];
            if (a < 0 || a >= n || b < 0 || b >= m) continue;
            if (g[a][b] == '#' || d[a][b] != -1) continue;
            d[a][b] = d[x][y] + 1;
            qx[tail] = a; qy[tail++] = b;
        }
    }
    printf("%d\n", d[n - 1][m - 1]);
    return 0;
}
```

1.6.5 访问标记的三种含义

同类练习（在线做题）：P3956 棋盘。

这道提高题包含颜色与施法规则，转移费用不全相同；要按完整状态求最小费用，不能直接提交本章普通 BFS 代码。

全排列中的 used 是当前路径内是否使用，返回时必须清除。普通 BFS 的 visited 表示已首次到达，通常全程不清除。记忆化的标记表示这个**完整状态的答案已算出**，应与具体到达路径无关。把

这三种标记混用，会造成漏解、死循环或重复计算。

若走到同一位置时“钥匙是否拿到”影响后续，则状态必须是（位置，钥匙状态）。只用位置作为 visited 会错误合并不同状态。若边权不同，普通 BFS 不再保证最小代价；应根据权值性质考虑 0–1 BFS 或最短路算法，不直接套网格模板。

1.6.6 自测与解答

练习 1。数组版全排列返回后，需要清空 path[depth] 吗？**解答**。不必。有效前缀长度由 depth 给出，下次选择会覆盖这一格，只有 depth 等于 n 时才输出。used[x] 必须恢复为 0，否则后续分支会错误地禁止使用 x。若用可变长 vector 保存路径，则应配对 push_back 与 pop_back，不能把两种实现的恢复规则混为一谈。

练习 2。BFS 为何在入队时标记，而不是出队时？**解答**。同一格可能被多个同层前驱发现，若出队才标记会重复入队。入队时记录距离既防重，也固定首次最短距离。

练习 3。只有两个格子，边代价分别可能为 1 或 100，按经过格子数最少就一定费用最小吗？**解答**。不一定。BFS 最小化边数，只有每条边的代价相等时才等价于最小费用。选择算法前必须看目标函数。

1.7 动态规划入门

真题练习：机器人饲养指南：第 37 次，2025-03，第 2 题；定义恰好消耗 v 个苹果的最大收益；第三节给解答（CSP202503B）。

1.7.1 先定义状态再写转移

动态规划把问题拆成可重复利用的子问题。每道题都要写出五项：状态含义、转移关系、初始值、计算顺序、答案位置。只记转移公式，容易在初值和边界处出错。

所谓无后效性，是状态已经包含决定后续所需的信息。给定状态后，不必再区分到达它的不同历史。这不表示当前选择不会影响未来，当前选择当然会改变后续状态。若过去某项信息会改变可选操作，就应把它加进状态。

例如爬楼梯，每次走 1、2 或 3 阶，令 $f[i]$ 为从 0 阶走到 i 阶的方案数。按最后一步分类，有 $f[i] = f[i-1] + f[i-2] + f[i-3]$。令 $f[0] = 1$，负下标状态为 0。这里的 1 表示“什么也不走”的一种空方案。按 i 从小到大计算，最终答案是 f[n]。

1.7.2 例题最长严格上升子序列

同类练习（在线做题）：B3637　最长上升子序列。

先掌握以当前位置结尾的状态，再根据数据范围选择二次或对数优化。

输入 n 个整数，选择一些位置并保持原有顺序，使选出的值严格递增，求最大长度。$1 \le n \le 5000$，值的绝对值不超过 10^9。子序列可以不连续。第一行 n，第二行数组。

输入输出样例

样例输入

```
6
3 1 2 2 5 4
```

样例输出

```
3
```

可取 $[1,2,5]$ 或 $[1,2,4]$。设 f[i] 为**以第 i 个元素结尾**的最长长度，

$$f[i] = 1 + \max\{f[j] : j < i,\ a_j < a_i\},$$

没有合法 j 时 f[i]=1。任何以 i 结尾的较长方案，倒数第二个位置必然是一个合法 j；反过来，把 a[i] 接到该 j 的方案后面也合法。因此枚举所有前驱就覆盖全部可能。答案是所有 f[i] 的最大值，不一定是 f[n]。时间 $O(n^2)$，空间 $O(n)$。

完整参考实现 `12_lis.cpp`

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 5005;
int a[N], f[N];
int main() {
    int n, answer = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < n; i++) {
        f[i] = 1;
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) f[i] = max(f[i], f[j] + 1);
        answer = max(answer, f[i]);
    }
    printf("%d\n", answer);
    return 0;
}
```

1.7.3 例题非空最大子段和

同类练习（在线做题）：P1115 最大子段和。

必须选择非空连续区间，检查全负数组。

输入 n 个整数，求一个**连续且非空**区间的最大和。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入输出样例

样例输入

```
5
-2 3 -1 4 -5
```

样例输出

```
6
```

设 ending 为以当前位置结尾的非空区间最大和。若使用当前数 x，则最优区间要么从 x 重新开

始，要么延长前一位置的最优区间：$ending = \max(x, ending + x)$。全局答案记录所有位置的最大 ending。由于要求非空，初始化为第一个元素，不能初始化为 0。时间 $O(n)$，额外空间 $O(1)$。

完整参考实现 `13_maxsum.cpp`

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int main() {
    int n; long long x;
    scanf("%d%lld", &n, &x);
    long long ending = x, answer = x;
    for (int i = 1; i < n; i++) {
        scanf("%lld", &x);
        ending = max(x, ending + x);
        answer = max(answer, ending);
    }
    printf("%lld\n", answer);
    return 0;
}
```

1.7.4 例题数字三角形的最大路径和

输入 n 行数字，第 i 行有 i 个数。从顶点出发，每次走到下一行的正下方或右下方，求到最后一行的最大路径和。$1 \le n \le 1000$，每个数的绝对值不超过 10^6。第一行 n，随后各行给出三角形。

输入输出样例

样例输入

```
3
2
-1 3
4 -2 1
```

样例输出

```
6
```

二维状态 f[i][j] 表示到第 i 行第 j 个位置的最优值，合法前驱只有 $(i-1, j-1)$ 与 $(i-1, j)$。初值为顶点，非法位置为负无穷。转移

$$f[i][j] = a[i][j] + \max(f[i-1][j-1], f[i-1][j]).$$

上例路径 $2, 3, 1$ 得 6。因为每行只依赖前一行，可压缩成一维。**从右向左更新**才能确保 f[j-1] 尚未被本行覆盖。代码设置 f[1]=0 作为顶点前的虚拟起点，其他位置为负无穷。时间 $O(n^2)$，额外空间 $O(n)$。

完整参考实现 `14_triangle.cpp`

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 1005;
const long long NEG = -(1LL << 60);
long long f[N], row[N];
int main() {
    int n;
    scanf("%d", &n);
    fill(f, f + n + 1, NEG); f[1] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) scanf("%lld", &row[j]);
        for (int j = i; j >= 1; j--)
            f[j] = max(f[j], f[j - 1]) + row[j];
    }
    printf("%lld\n", *max_element(f + 1, f + n + 1));
    return 0;
}
```

1.7.5 二维 DP 与最长公共子序列

给两个长度分别为 n、m 的序列，令 $f[i][j]$ 为两个前缀的最长公共子序列长度。若最后两个元素相等，候选为 $f[i-1][j-1]+1$；还可舍弃任意一边末元素。统一写为

$$f[i][j]=\max(f[i-1][j],f[i][j-1],\ [a_i=b_j]\ \text{时的}f[i-1][j-1]+1).$$

空前缀长度为 0，按行列递增计算。时间 $O(nm)$，滚动数组可把空间压缩到 $O(m)$，但仍不减少时间。若 n、m 都到 10^5，仅压缩空间仍不能通过，需要利用题目的额外性质。

1.7.6 记忆化搜索

记忆化按需求递归计算状态，求完后缓存。它与按顺序递推的主要区别是访问顺序。缓存键必须包含完整状态；若结果还取决于一个没有放进键里的参数，缓存会错误复用。

不要简单地以“结果为 0”判断是否计算过，因为 0 可能是合法答案。可以单设 done 数组。一般 DP 要求依赖关系可以按某种顺序求值；有环图上的最短路不能靠“加个缓存”就自动解决。第 11 章模板展开会同时使用“正在计算”和“已完成”两种标记识别环。

1.7.7 自测与解答

练习 1。数组 $[5,4,3]$ 的严格 LIS 长度是多少？**解答**。1。每个位置的 f 都为 1；若题目要求不下降序列，应把比较条件改为不大于，而不是仍用严格小于。

练习 2。数组 $[-5,-2,-7]$ 的非空最大子段和是多少？**解答**。-2。初始化为 0 会错误地选择空区间。

练习 3。三阶楼梯、每步走 1、2 或 3 阶，共多少方案？**解答**。4 种：111、12、21、3。由 $f[0]=1, f[1]=1, f[2]=2$ 得 $f[3]=4$。

练习 4。滚动数组能否把 LCS 的 $O(nm)$ 时间降为 $O(m)$？**解答**。不能。它只减少保存的历史行数，仍需要计算 n 行各 m 个状态。

1.8 基础背包与计数

真题练习：机器人饲养指南：第 37 次，2025-03，第 2 题；每天的喂法可重复，资源必须恰好用完；对照本节初始化和循环方向（CSP202503B）。

1.8.1 先确定问题问的是什么

背包的“容量”可以是时间、预算或其他资源，“价值”可以是收益、数量或得分。建模前必须区分：资源是最多使用还是恰好使用；每件只能选一次还是可无限选择；目标是最大价值、最小代价、能否达到还是方案数量。

状态含义	初始化	典型答案
容量不超过 v 的最大价值	全 0，允许什么都不选	f[V]。
恰好用 v 的最大价值	f[0]=0，其余负无穷	f[V]；不可达需另行输出。
恰好用 v 的最小代价	f[0]=0，其余正无穷	f[V]。
恰好凑出 v 的方案数	f[0]=1，其余 0	f[V]，必要时取模。

1.8.2 例题每件物品最多选一次

同类练习（在线做题）：P1048　采药。

时间是容量，药材价值是收益；一维转移按容量逆序。

有 n 件物品，每件体积 c、价值 w，容量为 V，求总体积不超过 V 的最大总价值。$1 \le n \le 200$，$0 \le V \le 5000$，$1 \le c \le 5000$，$0 \le w \le 10^9$。输入第一行 n、V，后面每行 c、w。

输入输出样例

样例输入

```
3 5
2 3
3 4
4 5
```

样例输出

```
7
```

令 f[i][v] 为只考虑前 i 件物品、容量不超过 v 的最大价值。第 i 件选或不选：

$$f[i][v] = \max(f[i-1][v],\ f[i-1][v-c_i] + w_i),$$

第二项只在 $v \geq c_i$ 时存在。若压缩掉 i 维，容量必须从大到小更新，使 f[v-c] 仍是上一层状态，避免同一件被重复选。复杂度 $O(nV)$，空间 $O(V)$。

完整参考实现 15_knapsack01.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = capacity; v >= cost; v--)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

1.8.3 例题每种物品可以无限选择

同类练习（在线做题）：P1616 疯狂的采药。

每种可重复选取，按容量正序转移；同时核对原题的较大容量和答案范围。

输入格式与上一题相同，但现在 n 表示物品种类，每种可以选任意非负整数件。仍要求体积不超过 V、最大化价值，且体积必须为正。采用同样的约束。

输入输出样例

样例输入

```
2 7
2 3
3 4
```

样例输出

```
10
```

选择体积 2 的两件和体积 3 的一件，价值 10。二维转移的选取项变为 $f[i][v-c_i]+w_i$，它允许当前种类继续使用。因此一维容量循环从小到大，使 f[v-c] 已经包含本种物品。时间 $O(nV)$，空间 $O(V)$。

完整参考实现 16_knapsack_complete.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = cost; v <= capacity; v++)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

1.8.4 用最小反例检查循环方向

只有一件体积 2、价值 3 的物品，容量 4。01 背包答案为 3；若正序更新，计算 f[4] 时使用了本轮刚更新的 f[2]=3，得到 6，实际上重复选择了同一件。完全背包允许重复，答案才是 6。记住“逆序 01、正序完全”之外，更要能解释读到的是哪一层。

1.8.5 例题无限硬币的组合数

有 n 种**互不相同的正整数面额**，每种无限个，问恰好凑出 target 的组合数，硬币选择顺序不算不同方案。输出对 10^9+7 取模的结果。$1 \le n \le 100$，$0 \le target \le 10^4$，面额不超过 10^4。输入第一行 n、target，第二行所有面额。

输入输出样例

样例输入

```
3 5
1 2 5
```

样例输出

```
4
```

四种为 $1+1+1+1+1$、$1+1+1+2$、$1+2+2$、5。令 f[v] 为已处理面额能凑出的组合数。先枚举面额，再正序枚举金额，确保每种组合按固定面额顺序生成一次。f[0]=1 是构造非空方案的起点。时间 $O(n \cdot target)$，空间 $O(target)$。

完整参考实现 17_coin_count.cpp

```cpp
#include <cstdio>
const int MOD = 1000000007;
int f[10005];
int main() {
    int n, target;
    scanf("%d%d", &n, &target);
    f[0] = 1;
    for (int i = 0; i < n; i++) {
        int c;
        scanf("%d", &c);
        for (int v = c; v <= target; v++) f[v] = (f[v] + f[v - c]) % MOD;
    }
    printf("%d\n", f[target]);
    return 0;
}
```

若改成先枚举金额、再枚举最后一个硬币，就会按排列计数。例如面额 1 和 2 凑 3，组合只有 $1+1+1$ 与 $1+2$ 两种，排列则还区分 $1+2$ 和 $2+1$，共三种。循环顺序改变了状态的计数含义。

1.8.6 例题每组最多选一件

同类练习（在线做题）：P1757 通天之分组背包。

原题输入包含物品的组号，应先归组，再按整组转移。

有 g 组物品，每组最多选一件，也可以不选。求容量 V 内的最大价值。$1 \le g \le 100, 0 \le V \le 2000$，每组 1 到 20 件，体积为正、价值非负。输入第一行 g、V，随后每组先给物品数 k，再给 k 对体积、价值。

输入输出样例

样例输入

```
2 5
2
2 3
3 5
2
2 4
4 7
```

样例输出

```
9
```

第一组选体积 3 价值 5，第二组选体积 2 价值 4。对每组保留旧数组 f，所有选择都从旧状态转移到 next_f，next_f 初始复制 f 表示不选。这种写法直接保证同一组至多贡献一件，比混用循环顺序更容易检查。若总物品数为 M，时间 $O(MV)$，空间 $O(V)$。

完整参考实现 18_group.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[2005], next_f[2005], value[25];
int cost[25];
int main() {
    int groups, capacity;
    scanf("%d%d", &groups, &capacity);
    for (int g = 0; g < groups; g++) {
        int k;
        scanf("%d", &k);
        for (int i = 0; i < k; i++) scanf("%d%lld", &cost[i], &value[i]);
        for (int v = 0; v <= capacity; v++) {
            next_f[v] = f[v];
            for (int i = 0; i < k; i++)
                if (v >= cost[i])
                    next_f[v] = max(next_f[v], f[v - cost[i]] + value[i]);
        }
        for (int v = 0; v <= capacity; v++) f[v] = next_f[v];
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

1.8.7 恰好装满时如何处理不可达

将所有状态设为负无穷，仅 f[0]=0。在转移前检查前驱可达，再进行加法。代码可使用约 -4×10^{18} 的哨兵，但必须证明合法值不会接近它；也可用额外布尔数组表示可达。不能把不可达初始化为 0，否则相当于凭空允许每个体积都有一个合法空方案。

1.8.8 自测与解答

练习 1。只有体积 2 价值 3 的一件物品，容量 3 且要求恰好装满，答案是什么？**解答**。不可达；若要求不超过容量，答案为 3。两种问法的初始化和答案解释不同。

练习 2。面额 2、3 凑 0 的组合数是多少？**解答**。1，即不取任何硬币。这也是 f[0]=1 的组合含义。

练习 3。只有一组，包含体积 2 价值 3、体积 3 价值 5，容量 5。答案是多少？**解答**。5，不能同组选两件得到 8。使用旧数组转移可防止这个错误。

练习 4。若完全背包出现体积 0 且价值正的物品，答案还有限吗？**解答**。没有有限上界。本章题目明确要求体积为正，这是算法与问题有意义的重要前提。

1.8.9 原题单 01 背包练习

何以包邮？（第 27 次，2022-09，第 2 题）每本书只能购买一次，先做恰好总价可达性，再找大于等于阈值的首个可达总价。不能照搬“容量以内最大价值”的答案位置；本篇练习给完整实现。

2 STL

真题练习：数字排序：第 4 次，2015-03，第 2 题；计数后按次数降序、数值升序排序（CSP201503B）。

真题练习：相似度计算：第 33 次，2024-03，第 2 题；统一字母大小写后用 set 计算交集和并集（CSP202403B）。

本节是第二篇的独立 STL 专章。STL 通常指标准模板库；本讲义将常用的标准库容器、算法、字符串和数对放在一起讲解。它们仍然属于 C++，程序需要用 C++ 编译器。**采用 C 风格写法**是指优先使用 scanf/printf、普通数组、下标循环、结构体和独立函数；使用 STL 时保留必要的模板类型和成员函数调用。

学习时先掌握 sort、二分查找、vector 与基本队列，再学 set、map 和堆。string、嵌套容器与自定义比较器会在第三篇继续用到。每种工具都要回答：数据如何存放、能做什么操作、一次操作花多少时间、什么情况下不能调用。

2.1 头文件、类型与最小程序

C++ 可以调用 C 标准库。`<cstdio>` 提供 scanf/printf，`<cstring>` 提供 strlen/strcmp，`<algorithm>` 提供 sort、unique 等算法。容器各有头文件；只包含 algorithm 并不能保证 queue 可用。

头文件	工具	适合解决的问题
algorithm	sort、unique、二分、reverse	排序、去重、静态有序查找。
vector	vector	元素数变化，需要按下标访问。
string	string	保存、比较和加工文本。
utility	pair、make_pair	把两个有关联的值放在一起。
stack	stack	后进先出，撤销、括号匹配。
queue	queue、priority_queue	先来先处理，或优先处理极值。
set	set、multiset	动态有序集合，前驱、后继。
map	map	从名字、编号映射到信息或次数。
functional	greater	常用于声明小根堆。

本章短代码默认已经包含对应头文件，且写在 main 或指定函数中；给出的完整参考程序可以直接编译。`using namespace std;` 允许省略 std:: 前缀。`vector<int>` 表示元素为 int 的动态数组，尖括号里的类型不是容量。

```cpp
#include <cstdio>
#include <vector>
using namespace std;
int main() {
    vector<int> a;
    a.push_back(7);
    printf("%d\n", a[0]);
    return 0;
}
```

读法。a 是容器；点号后是它的成员操作；push_back 在末尾添加一个元素。普通数组也可以配合 STL 算法，不必把所有数组改成 vector。

2.2 区间与迭代器：所有接口的共同基础

2.2.1 左闭右开区间

算法通常接收 $[first, last)$：包含 first，不包含 last。数组前 n 项是 `[a,a+n)`；使用 1 下标时则是 `[a+1,a+n+1)`。空区间满足 first 等于 last，长度为 0。

```cpp
int a[5] = {9, 4, 7, 1, 3};
sort(a + 1, a + 4); // Sort indices 1, 2, 3 only.
// a is now {9, 1, 4, 7, 3}.
```

2.2.2 迭代器可以移动，但不一定能相减

`begin()` 指向首元素，`end()` 指向最后一个元素之后。`*it` 读取当前元素，`++it` 移动到下一个。空容器的 begin 等于 end；end 不能解引用。

```cpp
vector<int> a(3, 8);
vector<int>::iterator it;
for (it = a.begin(); it != a.end(); ++it)
    printf("%d\n", *it);
```

vector 迭代器支持 `it+3` 和相减；set/map 迭代器只支持逐步前后移动，不能写 `it+3`、`s[0]` 或用迭代器相减求排名。迭代器也不能跨两个不同容器比较或相减。

2.2.3 什么时候需要重新取得迭代器

vector 扩容会使原来的指针、引用和迭代器失效；即使没有扩容，在中间插入、删除后，受影响位置及后面的迭代器也不能继续照用。set/map 插入通常不影响已有迭代器，删除则使指向被删元素的迭代器失效。最稳妥的初学者习惯是：修改容器后重新查找，删除时使用 erase 返回的下一个位置。

2.3 sort：普通数组排序与多关键字排序

同类练习（在线做题）：P1177 【模板】排序。

原题只要求整数升序；本节的多关键字比较是进一步扩展。

2.3.1 升序、降序和局部排序

默认 sort 按小于关系升序排列。下例里的 cmp 是**普通函数**，不是匿名函数；返回 true 表示 x 应排在 y 前面。

```
bool descending(int x, int y) { return x > y; }
// Inside main:
int a[5] = {3, 1, 4, 1, 5};
sort(a, a + 5);              // 1 1 3 4 5
sort(a, a + 5, descending); // 5 4 3 1 1
```

对 n 个元素，sort 保证 $O(n \log n)$ 量级的比较次数。不要把它固定理解为某一种朴素快速排序实现。sort 不保证等价元素的原相对顺序；需要保留时用 stable_sort，或将原下标加入次关键字。

2.3.2 比较器必须严格

`cmp(x,x)` 必须为 false；x 排在 y 前时，y 不能也排在 x 前；比较关系还要传递。`x>=y` 不是合格比较器。不能在比较过程中随机改变规则，也不应通过 `return x-y;` 返回布尔结果：非零都会变成 true，而且减法可能溢出。字符数组按内容比较应使用 strcmp，不能直接比较两个数组地址。

2.4 例题按成绩和编号排序

题目。输入 n 名学生的编号和成绩，成绩降序，相同成绩按编号字典序升序。编号是不含空格的字符串，长度 1 到 30 且互不相同；$1 \le n \le 2 \times 10^5$，成绩 0 到 100。首行 n，随后每行编号和成绩，输出排序后的记录。

输入输出样例

样例输入

```
4
b02 90
a01 95
a02 90
a03 90
```

样例输出

```
a01 95
a02 90
a03 90
b02 90
```

解答。结构体数组保存记录，cmp 先判断成绩，再用 strcmp 判断编号。编号数组容量 31，多留一格保存字符串结束符。`%30s` 限制一次最多读入 30 个字符。比较次数为 $O(n \log n)$，每次字符串比较最多扫描 30 个字符，记录存储为 $O(n)$。

完整参考实现 03_rank.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const int N = 200005;
struct Student { char id[31]; int score; };
Student a[N];
bool cmp(Student x, Student y) {
    if (x.score != y.score) return x.score > y.score;
    return strcmp(x.id, y.id) < 0;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%30s%d", a[i].id, &a[i].score);
    sort(a, a + n, cmp);
    for (int i = 0; i < n; i++) printf("%s %d\n", a[i].id, a[i].score);
    return 0;
}
```

自测。若编号为 a2 和 a10，字典序中 a10 在 a2 之前；题目若要求按数字编号排序，应先把数字部分解析为数值。

2.5 unique：去相邻重复与修改逻辑长度

unique 把相邻等价元素压缩到区间前部，返回**新的逻辑末尾**，不改变普通数组容量，也不会自动缩短 vector。新末尾后的元素不属于去重结果，不要依赖其具体值。

```cpp
int a[6] = {3, 1, 3, 2, 2, 1};
sort(a, a + 6);                 // 1 1 2 2 3 3
int m = unique(a, a + 6) - a;   // m = 3
for (int i = 0; i < m; i++) printf("%d ", a[i]);
// Output: 1 2 3
```

如果不排序，[1,2,1] 中两个 1 不相邻，unique 不会删掉后面的 1。若要求保留首次出现顺序，排序去重会改变顺序，应该用集合判断是否见过，同时另存答案。vector 全局去重的常见顺序是 sort、unique、erase：

```cpp
vector<int> v;
v.push_back(3); v.push_back(1); v.push_back(3);
sort(v.begin(), v.end());
vector<int>::iterator last = unique(v.begin(), v.end());
v.erase(last, v.end());
```

练习与完整解答：有序去重。输入 n 个整数，$1 \le n \le 2 \times 10^5$、$|a_i| \le 10^9$。第一行输出不同值数量，第二行按升序输出这些值。

输入输出样例

样例输入

```
6
3 1 3 2 2 1
```

样例输出

```
3
1 2 3
```

排序后每个等值段保留一个即可，时间 $O(n \log n)$。

同类练习（在线做题）：P1059　明明的随机数。

该原题同样要求去重后升序输出，提交时按原题范围与格式读写。

完整参考实现　27_distinct.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int a[200005];
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    sort(a, a + n);
    int m = unique(a, a + n) - a;
    printf("%d\n", m);
    for (int i = 0; i < m; i++)
        printf("%d%c", a[i], i + 1 == m ? '\n' : ' ');
    return 0;
}
```

2.6　lower_bound 与 upper_bound：四种边界

同类练习（在线做题）：P2249　【深基 13. 例 1】查找。

原题要求首次出现的位置；将迭代器转成题面要求的下标，并单独处理查找失败。

本节默认序列按升序排列；不要在无序数组上直接使用二分查找。lower_bound 找第一个不小于 x 的位置，upper_bound 找第一个大于 x 的位置；找不到时都返回右端点。

```cpp
int a[4] = {2, 2, 5, 7};
int n = 4, x = 2;
int l = lower_bound(a, a + n, x) - a;
int r = upper_bound(a, a + n, x) - a;
printf("%d %d %d\n", l, r, r - l); // 0 2 2
if (l < n && a[l] == x) puts("FOUND");
```

需求	下标公式	必须检查
第一个 $\geq x$	lower_bound 减起点	下标小于 n。
第一个 $> x$	upper_bound 减起点	下标小于 n。
最后一个 $< x$	lower_bound 下标减 1	结果不小于 0。

需求	下标公式	必须检查
最后一个 $\le x$	upper_bound 下标减 1	结果不小于 0。
x 出现次数	upper 下标减 lower 下标	两次使用同一有序区间。

数组或 vector 上的一次查询为 $O(\log n)$。例如 x=8 时两端均为 4，表示不存在 8；`a[4]` 不是有效元素。**set/map 上优先使用成员函数**，如 `s.lower_bound(x)`；通用算法作用于树迭代器时，比较次数虽少，迭代器移动却可能达到线性数量。

2.7 离散化：值域大时保存相对大小

离散化把实际值变成在去重有序表中的排名。原序列 $[100, 5, 100, 20]$ 的有序不同值为 $[5, 20, 100]$，1 起始排名为 $[3, 1, 3, 2]$。**保留相等关系与大小关系，不保留坐标距离。**

```
int a[4] = {100, 5, 100, 20}, keys[4], rank_id[4];
int n = 4;
for (int i = 0; i < n; i++) keys[i] = a[i];
sort(keys, keys + n);
int m = unique(keys, keys + n) - keys;
for (int i = 0; i < n; i++)
    rank_id[i] = lower_bound(keys, keys + m, a[i]) - keys + 1;
```

排序 $O(n \log n)$，逐项查询 $O(n \log n)$，空间 $O(n)$。必须复制一份数据再排序，否则原顺序被破坏。涉及线段长度或面积时，用排名定位，用 keys 恢复原坐标计算距离。在线出现的新值不能随意追加到末尾并假定排名仍正确。

2.8 vector：可变长数组

2.8.1 建立、赋值与下标访问

```
vector<int> a;              // Empty, size 0.
vector<int> b(5);           // Five zeros.
vector<int> c(5, -1);       // Five copies of -1.
a.push_back(10); a.push_back(20);
printf("%d %d\n", (int)a.size(), a[1]); // 2 20
b = a;                      // Copies the elements.
for (int i = 0; i < (int)b.size(); i++) printf("%d ", b[i]);
```

合法下标为 0 到 size 减 1。`a[5]=1` 不会自动扩容。size 返回无符号类型；本讲义数据量可用 int 表示，循环前转为 int，避免空容器上 `size()-1` 发生无符号下溢。`front()`、`back()`、`pop_back()` 均应先确认非空。

2.8.2 size、capacity、resize、reserve 的区别

size 是已有元素数，capacity 是无需重新分配时能容纳的数量。resize 改变元素数；reserve 只预留容量。`vector<int> a; a.reserve(100);` 之后 a 仍然为空，不能写 a[0]，应该 push_back 或先 resize。`a.resize(100,7)` 只把新增元素填成 7，原有元素不会全部改写；要整体重置可用 `a.assign(100,7)`。

2.8.3 插入、删除和二维 vector

```cpp
vector<int> a(3, 5);        // 5 5 5
a.insert(a.begin() + 1, 9); // 5 9 5 5
a.erase(a.begin() + 2);     // 5 9 5
a.erase(a.begin(), a.begin() + 2); // 5
a.clear();                  // size becomes 0.
vector<vector<int> > grid(3, vector<int>(4, 0));
grid[1][2] = 8;
```

下标访问为 $O(1)$，末尾 push_back 均摊 $O(1)$；单次扩容可能搬移全部元素。中间插入删除通常为 $O(n)$，连续在开头删除 n 次可能变成 $O(n^2)$。clear 清空元素，不承诺立刻归还所有容量。二维 vector 的每一行是独立 vector，不能假定跨行内存连续；规模已知的普通矩阵通常直接使用二维数组。

练习与解答：**追加和撤销**。首行 q，$1 \le q \le 10^5$；随后每条命令为 ADD x、UNDO 或 PRINT，$|x| \le 10^9$。ADD 末尾追加，UNDO 删除末尾一项，空时忽略；PRINT 输出当前序列，空时输出 EMPTY。保证全部 PRINT 实际输出的元素数不超过 2×10^5。

输入输出样例

样例输入

```
7
UNDO
ADD 4
ADD 9
PRINT
UNDO
PRINT
UNDO
```

样例输出

```
4 9
4
```

每次修改仅涉及末尾，vector 适合保存操作结果。时间为 $O(q+S)$ 的均摊量级，S 为输出元素总数；没有对 S 的约束时，反复 PRINT 的输出本身也可能很大。

完整参考实现 33_undo.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    vector<int> a;
    while (q--) {
        char op[10]; scanf("%9s", op);
        if (strcmp(op, "ADD") == 0) {
            int x; scanf("%d", &x); a.push_back(x);
        } else if (strcmp(op, "UNDO") == 0) {
            if (!a.empty()) a.pop_back();
        } else {
            if (a.empty()) puts("EMPTY");
            else {
                for (int i = 0; i < (int)a.size(); i++)
                    printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
            }
        }
    }
    return 0;
}
```

2.9 string：与字符数组配合处理文本

2.9.1 读写、复制和字典序

string 自动管理存储空间，可赋值、拼接和按内容比较。字符数组用 `strcmp(a,b)`，string 可以直接用 `a==b`、`a<b`。字符串`"10"` 按字典序小于`"2"`，这不等于数值比较。char 指针不应直接作为按文本内容比较的 map 键，本章统一使用 string 键。

```cpp
char buf[101];
scanf("%100s", buf);
string s = buf;
printf("%s\n", s.c_str());
string t = s; t += "!";
for (int i = 0; i < (int)t.size(); i++) printf("%c", t[i]);
```

不能写 `scanf("%s", s)` 或 `printf("%s", s)`：格式化函数不会自动把 string 转换成字符指针。输出使用 c_str；输入先读字符数组再赋给 string。`c_str()` 返回的指针只宜用于即时读取，修改 string 后不要继续依赖旧指针。

2.9.2 整行输入与空格

```cpp
char line[1005];
if (fgets(line, sizeof(line), stdin) != NULL) {
    line[strcspn(line, "\r\n")] = '\0';
    string s = line;
    printf("[%s]\n", s.c_str());
}
```

fgets 最多读入容量减 1 个字符，可能保留换行。例中用 strcspn 找到第一个回车或换行并截断。容量必须覆盖题目允许的最大行长，否则一次 fgets 不一定读完一行。先 scanf 读取数字再 fgets 时，

应把该行余下字符读到换行；只调用一次 getchar 仅在确知剩余恰好一个换行时才足够。不要用跳过所有空白的方法处理允许空行或前导空格的题目。

2.9.3 截取、查找、插入、删除与替换

```
string s = "abcdef";
string t = s.substr(2, 3);  // "cde": start, count.
s.erase(1, 2);              // "adef": remove b,c.
s.insert(1, "XY");          // "aXYdef".
s.replace(1, 2, "-");       // "a-def".
string::size_type p = s.find("de");
if (p != string::npos) printf("%d\n", (int)p); // 2
```

substr 第二参数是长度，不是终点。find 失败返回 string::npos，先判断成功再转成 int 或拿来访问。insert/erase/replace 可能移动后缀，反复从头修改长字符串会很慢；可以逐段追加到新字符串。string 不能用 memset 清零，清空用 clear。

练习与解答：**从左向右替换**。输入三行 s、p、t，各行长度不超过 1000，允许空格；s 和 t 可空，p 非空。按原串 s 从左到右查找 p，每次选取最靠左的不重叠出现，将其替换为 t。新插入的内容不再次参与匹配，输出结果。例如 s 为 aaaa、p 为 aa、t 为 b，结果 bb；s 为 aaa 时结果 ba。

输入输出样例

样例输入

```
one two one
one
X
```

样例输出

```
X two X
```

用 start 标记原串尚未处理的位置。找到后先追加未匹配片段，再追加 t，随后前进 p 的长度。始终在原串上 find，可避免 t 包含 p 时重复替换甚至死循环。答案最长为约 10^6 字符；朴素匹配可用 $O(|s||p| + |answer|)$ 估计，具体 string 查找实现可能优化，本题规模不依赖这种优化。

完整参考实现 34_replace.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
using namespace std;
char line[1005], pattern[1005], replacement[1005];
void read_line(char s[]) {
    if (fgets(s, 1005, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
}
int main() {
    read_line(line); read_line(pattern); read_line(replacement);
    string s = line, p = pattern, t = replacement, answer;
    string::size_type start = 0;
    while (true) {
        string::size_type pos = s.find(p, start);
        if (pos == string::npos) break;
        answer += s.substr(start, pos - start);
        answer += t;
        start = pos + p.size();
    }
    answer += s.substr(start);
    printf("%s\n", answer.c_str());
    return 0;
}
```

2.10 pair：两个值的组合

pair 是辅助类型，不是可增删元素的容器。`pair<int,int>` 可以存坐标、数值与下标等；成员名为 first、second。为了保持简单写法，本章用 make_pair 创建数对，不用结构化绑定。

```cpp
pair<int, int> p = make_pair(3, 8);
printf("%d %d\n", p.first, p.second);
pair<int, int> a[3];
a[0] = make_pair(2, 1);
a[1] = make_pair(1, 9);
a[2] = make_pair(1, 4);
sort(a, a + 3); // (1,4), (1,9), (2,1)
```

默认按 first 比较，first 相同才比较 second，即字典序。字段有明确业务含义时可改用结构体并命名为 x、y、score 等，提高可读性；pair 的比较不会自动满足“第一项降序、第二项升序”，这种规则仍需自定义比较函数。

2.11 stack：后进先出与括号匹配

2.11.1 基本操作

```cpp
stack<int> st;
st.push(4); st.push(9);
if (!st.empty()) {
    printf("%d\n", st.top()); // 9
    st.pop();
}
printf("%d\n", (int)st.size()); // 1
```

push 加入栈顶，top 只读取栈顶，pop 只删除栈顶，**pop 不返回被删除的值**。默认底层容器下这些操作为常数时间。stack 不提供下标访问或普通遍历；需要访问所有元素时可改用 vector。普通

数组也能实现栈：用 top 表示已有元素数，入栈 `a[top++]=x`，读栈顶 `a[top-1]`，出栈 `top--`，访问前确认 top 大于 0。

练习与解答：三类括号匹配。输入一行仅由 `()[]{}` 组成的字符串，长度 0 到 10^5；空串合法。若每个右括号能与最近尚未匹配的同类左括号配对，输出 YES，否则 NO。

输入输出样例 1

样例输入

```
([]){}
```

样例输出

```
YES
```

输入输出样例 2

样例输入

```
([)]
```

样例输出

```
NO
```

最近出现的未匹配左括号必须先配对，所以使用栈。扫描右括号时先检查非空，再检查类型；扫描结束还必须检查栈是否为空。每个字符最多入栈出栈各一次，时间和最坏空间均为 $O(n)$。只比较左右括号总数会误判顺序或嵌套类型。

完整参考实现 28_brackets.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <stack>
using namespace std;
char s[100005];
int match(char left, char right) {
    return (left == '(' && right == ')') ||
           (left == '[' && right == ']') ||
           (left == '{' && right == '}');
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    stack<char> st;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty() || !match(st.top(), c)) {
                puts("NO"); return 0;
            }
            st.pop();
        }
    }
    puts(st.empty() ? "YES" : "NO");
    return 0;
}
```

2.12 queue：先进先出与处理顺序

```cpp
queue<int> q;
q.push(4); q.push(9);
if (!q.empty()) {
    printf("%d %d\n", q.front(), q.back()); // 4 9
    q.pop();
}
```

push 在队尾添加，front 读取队首，back 读取队尾，pop 删除队首；访问空队列是错误。默认底层容器下这些操作为常数时间。queue 按进入顺序处理，不按数值大小排序。BFS 按层扩展用 queue；总要取最大值时则应考虑 priority_queue。

普通数组实现队列时，可用 $[head, tail)$ 表示有效区间：入队 `a[tail++]=x`，出队先读 `a[head]` 再 head 加 1，非空条件 head 小于 tail。只增不回绕的写法需要容量覆盖**累计入队次数**；长期复用时可改成循环队列或直接用 queue。

练习与解答：**排队取号**。首行 n，$1 \le n \le 10^5$；随后每行 PUSH x 或 POP，编号 x 为 int 范围内整数。PUSH 入队；POP 输出并删除队首，空队列输出 EMPTY。

输入输出样例

样例输入

```
6
POP
PUSH 5
PUSH 2
POP
POP
POP
```

样例输出

```
EMPTY
5
2
EMPTY
```

维护进入顺序即可。每条命令 $O(1)$，总时间 $O(n)$，最多保存 n 个编号。

完整参考实现 29_queue.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    queue<int> q;
    while (n--) {
        char op[10];
        scanf("%9s", op);
        if (strcmp(op, "PUSH") == 0) {
            int x; scanf("%d", &x); q.push(x);
        } else {
            if (q.empty()) puts("EMPTY");
            else { printf("%d\n", q.front()); q.pop(); }
        }
    }
    return 0;
}
```

2.13 priority_queue：堆与优先处理

2.13.1 大根堆与小根堆

```cpp
priority_queue<int> large;
large.push(4); large.push(9); large.push(2);
printf("%d\n", large.top()); // 9
priority_queue<int, vector<int>, greater<int> > small;
small.push(4); small.push(9); small.push(2);
printf("%d\n", small.top()); // 2
```

第一个模板参数是元素类型，第二个是底层存储，第三个是比较方式。默认大根堆 top 最大；greater 使最小值位于堆顶。top 为 $O(1)$，push/pop 的堆调整为 $O(\log n)$，底层 vector 分配另有均摊成本。堆不是完全排好序的数组，不支持第 k 项查询，也没有 erase(x) 接口。要删除指定值可用 multiset；延迟删除见提高附录。

2.13.2 结构体与普通比较函数

堆的比较函数返回 true，表示**左边对象优先级比右边低**，与 sort 里“应放在前面”的直观含义不同。下面优先取成绩更高者，相同成绩取编号更小者。函数指针类型写法中，Compare 表示接收两个 Item 并返回 bool 的函数指针。

```cpp
struct Item { int score, id; };
bool lower_priority(Item a, Item b) {
    if (a.score != b.score) return a.score < b.score;
    return a.id > b.id;
}
// Inside main:
typedef bool (*Compare)(Item, Item);
priority_queue<Item, vector<Item>, Compare> q(lower_priority);
Item a; a.score = 90; a.id = 2; q.push(a);
a.id = 1; q.push(a);
printf("%d\n", q.top().id); // 1
```

构造时必须传入比较函数，不能让函数指针保持空值。不要直接把 sort 的“高分在前”比较器

原封不动放进堆。本讲义第三篇的任务调度继续使用这种普通函数写法。

练习与解答：保留最大的 k 个数。输入 n、k 及 n 个整数，$1 \le k \le n \le 2 \times 10^5$，数值在 int 范围内。重复值按不同出现计数，按降序输出最大的 k 个数。

输入输出样例

样例输入

```
5 3
4 1 9 9 2
```

样例输出

```
9 9 4
```

维护最多 k 个元素的小根堆。每读入一个数就入堆，超过 k 个时删除最小值；留下的恰好是已读前缀中最大的 k 个。反复弹出得到升序，最后逆转。时间 $O(n \log(k+1) + k)$，额外空间 $O(k)$。若 n 很小且一次读完，直接 sort 也是合理选择。

完整参考实现 `32_heap.cpp`

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;
int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    priority_queue<int, vector<int>, greater<int> > q;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x); q.push(x);
        if ((int)q.size() > k) q.pop();
    }
    vector<int> a;
    while (!q.empty()) { a.push_back(q.top()); q.pop(); }
    reverse(a.begin(), a.end());
    for (int i = 0; i < (int)a.size(); i++)
        printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
    return 0;
}
```

2.14 set：自动去重的有序集合

set 存唯一键，默认按升序排列。insert 重复键不会新增元素；find 失败返回 end；count 在 set 中只能为 0 或 1。查找、插入、按键删除的通常复杂度为 $O(\log n)$，完整遍历为 $O(n)$。

```cpp
set<int> s;
s.insert(5); s.insert(2); s.insert(5);
printf("%d\n", (int)s.size()); // 2
set<int>::iterator it;
for (it = s.begin(); it != s.end(); ++it) printf("%d ", *it);
if (s.find(5) != s.end()) puts("FOUND");
s.erase(2);
```

不能通过 `*it=` 新值修改 set 中的键，否则会破坏有序关系；修改键应先删旧值再插新值。遍历不是按插入顺序，不支持按下标读取。只做一次排序去重时普通数组往往更直接；数据边加入边

查询时 set 更合适。

2.14.1 前驱、后继与安全边界

以下寻找严格小于 x 的前驱，以及不小于 x 的后继。两者定义不同于“严格大于”的后继；严格大于应改用 upper_bound。

```
int x = 6;
set<int>::iterator it = s.lower_bound(x);
if (it != s.end()) printf("next = %d\n", *it);
if (it != s.begin()) {
    --it;
    printf("previous = %d\n", *it);
}
```

空集合 begin 等于 end，两种访问都会被挡住；x 大于所有元素时 it 为 end，可以在集合非空的条件下先减一次取得最大值；x 不大于最小值时不能对 begin 做减法。

2.15 multiset：重复值与删除一个副本

multiset 允许相同值出现多次。`erase(x)` 删除**所有等于 x 的元素**；`erase(it)` 只删除迭代器指向的一个。count 可能返回大于 1，统计大量重复元素的成本也与出现次数有关，不能把它一律算作 $O(\log n)$。

```
multiset<int> s;
s.insert(5); s.insert(5); s.insert(8);
multiset<int>::iterator it = s.find(5);
if (it != s.end()) s.erase(it); // One 5 remains.
printf("%d\n", (int)s.count(5)); // 1
s.erase(5); // Remove all remaining 5s.
```

练习与解答：动态候选值。首行 q，$1 \le q \le 2 \times 10^5$；随后每行 ADD x、DEL x 或 GE x，$|x| \le 10^9$。ADD 加入一个副本，DEL 删除一个副本，不存在则忽略；GE 输出当前最小的、不小于 x 的值，不存在输出 NONE。

输入输出样例

样例输入

```
8
ADD 5
ADD 5
ADD 9
DEL 5
GE 5
DEL 5
GE 5
GE 10
```

样例输出

```
5
9
NONE
```

使用 multiset 保留重复值；DEL 采用 find 加 erase 迭代器；GE 用成员 lower_bound。每条操作至多 $O(\log(q+1))$，空间 $O(q)$。

完整参考实现 30_neighbors.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <set>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    multiset<int> s;
    while (q--) {
        char op[10]; int x;
        scanf("%9s%d", op, &x);
        if (strcmp(op, "ADD") == 0) s.insert(x);
        else if (strcmp(op, "DEL") == 0) {
            multiset<int>::iterator it = s.find(x);
            if (it != s.end()) s.erase(it);
        } else {
            multiset<int>::iterator it = s.lower_bound(x);
            if (it == s.end()) puts("NONE");
            else printf("%d\n", *it);
        }
    }
    return 0;
}
```

2.16 map：键到信息的映射

2.16.1 下标、find 和有序遍历

map 保存唯一键及其对应值，默认按键排序。迭代器的 first 是键、second 是值；键不可直接修改，值可以修改。

例如，string 到 int 的映射可以保存名字及其次数；int 到 vector 的映射可以保存某个编号对应

的一组记录。

```
map<string, int> cnt;
cnt["apple"]++; cnt["pear"] += 2;
map<string, int>::iterator it = cnt.find("orange");
if (it == cnt.end()) puts("NOT FOUND");
for (it = cnt.begin(); it != cnt.end(); ++it)
    printf("%s %d\n", it->first.c_str(), it->second);
```

访问不存在键的 `cnt[key]` 会插入默认值；这里 int 默认为 0，因此计数很方便。但只想查存在性时应使用 find 或 count，避免创建新键。`cnt.count(key)` 表示这个键是否存在，不是该单词的频数。`insert(make_pair(key,value))` 遇到已有键不会覆盖旧值；需要覆盖应使用下标赋值。

比较和树查找通常需要 $O(\log n)$ 次键比较。若键是长度 L 的字符串，一次比较最坏 $O(L)$，整体可估为 $O(L \log n)$。值域很小且连续的计数优先用普通数组，避免不必要的树节点开销。

练习与解答：**单词计数**。输入 n 及 n 个仅含小写字母的单词，每词长度 1 到 30，$1 \le n \le 10^5$。按字典序输出每个不同单词及次数。

输入输出样例

样例输入

```
5
pear apple pear banana apple
```

样例输出

```
apple 2
banana 1
pear 2
```

读入时对键计数；map 已经按键排列，最后顺序遍历即可，不需要再 sort。若有 m 个不同单词，计数约需 $O(30n \log(m+1))$ 时间，存储约 $O(30m)$。

完整参考实现 `31_frequency.cpp`

```
#include <cstdio>
#include <map>
#include <string>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    map<string, int> count;
    for (int i = 0; i < n; i++) {
        char word[31];
        scanf("%30s", word); count[word]++;
    }
    map<string, int>::iterator it;
    for (it = count.begin(); it != count.end(); ++it)
        printf("%s %d\n", it->first.c_str(), it->second);
    return 0;
}
```

2.16.2 遍历时删除与嵌套映射

```
map<string, int>::iterator it = cnt.begin();
while (it != cnt.end()) {
    if (it->second == 0) it = cnt.erase(it);
    else ++it;
}
map<string, set<string> > rights;
rights["alice"].insert("file1");
map<string, set<string> >::iterator p = rights.find("alice");
if (p != rights.end() && p->second.count("file1")) puts("YES");
```

erase 之后继续对旧 it 加 1 会使用失效迭代器。嵌套 map/set 适合权限管理：外层找用户，内层查资源。查询用 find，授权才用下标主动创建，这正是第三篇授权模拟的状态组织方式。

2.17 unordered_map 与 unordered_set：哈希查找

真题练习：相似度计算：第 33 次，统一大小写后记录每个词在哪篇出现；真题速看给完整代码（CSP202403B）。

真题练习：回收站选址：第 18 次，查询坐标是否存在；练习中用 set 的有序查找实现（CSP201912B）。

unordered_map 保存键值对，需要包含 <unordered_map>。unordered_set 只保存不重复的键，需要包含 <unordered_set>。哈希表根据键的哈希值选桶，再在桶内寻找精确相等的键。不同键可能哈希相同，不能把哈希值相同直接当作原对象相同。

与 map、set 相比，哈希容器不维护键的顺序，不能直接求某值的前驱、后继或最小键。插入、查找、删除的**平均**时间为 $O(1)$（字符串还要计算与长度相关的哈希）；严重冲突时单次可能退化为 $O(n)$。不要把平均复杂度写成最坏保证。

```
#include <string>
#include <unordered_map>
std::unordered_map<std::string, int> mask;
mask.max_load_factor(0.7f);
mask.reserve(100000);
mask["apple"] |= 1;
mask["apple"] |= 2; // 3 means present in both articles.
std::unordered_map<std::string, int>::const_iterator it = mask.find("pear");
if (it != mask.end()) printf("%d\n", it->second);
```

reserve 根据预计元素数预留桶，减少插入中的重哈希；不能保证无冲突。负载因子约为“元素数/桶数”，调整时兼顾时间和内存。重哈希会使迭代器失效，所以不要在一次遍历中继续使用重哈希之前的迭代器。operator[] 会插入不存在的键；只想查看存在性用 find。**练习与解答**：若第一篇 apple 出现三次、第二篇出现两次，位标记仍为 3，交集只贡献 1；若要求按词典序输出单词，应另行排序或改用 map。

2.18 常用算法补充与容器选择

```
int a[5] = {1, 2, 3, 4, 5};
reverse(a, a + 5); // 5 4 3 2 1
fill(a, a + 3, 7); // 7 7 7 2 1
int pos = max_element(a, a + 5) - a;
printf("%d %d\n", pos, a[pos]); // 0 7
swap(a[0], a[4]);
```

reverse 和 fill 为线性，max_element 找最大值位置也为线性；空区间会返回右端点，不能解引用。min/max 常用于两个值的比较；数组整体求最小值用 min_element。给普通 int 数组全部置 0 可以 memset；**memset 按字节填充**，`memset(a,1,sizeof(a))` 不是把每项变成整数 1。给每项赋一般数值时用 fill 或 for 循环。

任务特征	首选工具	选择依据
规模已知、频繁下标访问	普通数组	简单直接，容量需提前留够。
末尾追加、撤销，偶尔遍历	vector	连续存储，动态长度。
最近未完成对象先处理	stack	后进先出。
最早进入对象先处理	queue	先进先出。
每次仅取最大或最小	priority_queue	堆顶快，不提供任意删除。
动态查前驱后继、无重复	set	有序唯一键。
动态查前驱后继、有重复	multiset	保留出现次数。
按名字查次数或属性	map	键到值，有序遍历。
静态排序后重复查找	数组加二分	不需维护动态树结构。

unordered_map、unordered_set 不按键排序，常见哈希操作平均较快但最坏可能退化；不能替代需要 lower_bound 或有序输出的 map/set。deque 支持两端操作，list 适合特定位置已知的链表操作，但本讲义常规题型不以它们为主。选择容器应由题目操作决定，不能仅凭“高级”程度选择。

2.19 练习与解答

1. 有序数组 $[2,2,5,7]$ 中，x=2 的 lower 和 upper 下标是多少？x=8 呢？**解答**：分别为 0、2 和 4、4；后者是末尾，不能访问数组第 4 项。

2.vector 只 reserve(20)，能否写 a[0]=3？**解答**：不能，size 仍为 0。push_back(3) 或先 resize 才建立元素。

3.map 中 a 键的频数是 5，`cnt.count("a")` 是多少？**解答**：1；它统计键是否存在。实际频数在 it->second 或 cnt[”a”] 中。

4.multiset 含 $[4,4,8]$，删一个 4 后如何保证仍有一个 4？**解答**：find(4)，检查不为 end，然后 erase 该迭代器；不能直接 erase(4)。

5. 比较器返回 `x.score>=y.score` 是否正确？**解答**：错误，自比较会为 true，破坏严格弱序。相同主关键字再比较次关键字。

6. 栈和队列均依次加入 2、7、3，再依次弹出，输出是什么？**解答**：栈为 3、7、2，队列为 2、7、3；默认大根堆则为 7、3、2。

7. 把坐标 $[1, 100, 101]$ 变成 $[1, 2, 3]$ 后能否用排名差算距离？**解答**：不能。排名只保序，原前两点距离 99 而排名差为 1。

8. 替换题 s=aaa、p=aa、t=aaaa 时，结果和处理次数是多少？**解答**：一次匹配，输出 aaaaa；新插入内容不再匹配，避免无休止增长。

9. 用小根堆保存最大的 3 个数，已读 $[6, 1, 5, 2]$ 时堆中是什么？**解答**：保留 2、5、6，堆顶 2；内部数组次序不必完全升序。

10. 为什么不能在 map 查询缺失用户时一律写 rights[user]？**解答**：该语句会建立空集合；查询越多，状态可能无故增长。使用 find 后再访问已有记录。

接口核对。进一步可查阅 C++ 工作草案中的vector 容量规则、关联容器规则、优先队列与排序算法。上述定义、示例和解答均已写在正文，链接仅用于查阅细节。

3 真题速看

本节五题均选自原资料第二题入口，分别训练双指针、二维前缀和、二分答案、STL 哈希与完全背包。规则按对应做题平台转述；不同平台的数据范围可能不同，本书优先采用曙梦当前范围。

3.1 买菜

第 14 次 CCF-CSP 认证，2018-09，第 2 题。买菜（CSP201809B）。**题目与输入输出**。两个人各有 n 段在广场装车的时间，两组区间分别按起点严格递增，且同组区间互不接触、不重叠。求两人同时在广场装车的总时长。输入 n、第一人的 n 行起止时刻、第二人的 n 行起止时刻；输出总时长。$1 \le n \le 2000$，时刻在 1 到 10^6 之间。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 256MB。**读题思路**。计算的是连续时间长度，端点接触不贡献时长，所以交集长度不加 1。原题每组都已排序；读入后无需重复排序。**做题思路与解答**。用 i、j 指向尚未处理的两段。交集为 $[\max(l_i, l_j), \min(r_i, r_j)]$，右端大于左端时累加其差。结束更早的一段不可能与另一组后面的区间重叠，因此移动它；若同时结束，两个都移动。每次安全丢弃已经算完的一段，既不漏交集，也不会重复计数。

输入输出样例

样例输入

```
4
1 3
5 6
9 13
14 15
2 4
5 7
10 11
13 14
```

样例输出

```
3
```

复杂度与边界。两个指针各前进至多 n 次，时间 $O(n)$，数组空间 $O(n)$。样例中正时长交集是 $[2,3]$、$[5,6]$、$[10,11]$，总长 3。

完整参考实现 `40_intervals.cpp`

```cpp
#include <cstdio>
struct Interval { int l, r; } a[2005], b[2005];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d%d", &a[i].l, &a[i].r);
    for (int i = 0; i < n; i++) scanf("%d%d", &b[i].l, &b[i].r);
    int i = 0, j = 0, answer = 0;
    while (i < n && j < n) {
        int l = a[i].l > b[j].l ? a[i].l : b[j].l;
        int r = a[i].r < b[j].r ? a[i].r : b[j].r;
        if (l < r) answer += r - l;
        if (a[i].r < b[j].r) i++;
        else if (a[i].r > b[j].r) j++;
        else { i++; j++; }
    }
    printf("%d\n", answer);
    return 0;
}
```

3.2 邻域均值

第 22 次 CCF-CSP 认证，2021-04，第 2 题。邻域均值（CSP202104B）。**题目与输入输出**。给出 $n \times n$ 灰度矩阵，各值在 0 到 $L-1$。一个像素的邻域包括与它行、列距离分别不超过 r 的所有合法格子（含自身），越出图像的部分裁掉。统计邻域平均灰度不超过 t 的像素数。输入 n、L、r、t，再输入矩阵；输出计数。$n \le 600$，$1 \le r \le 100$，$2 \le t < L \le 256$。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。每格对应一个裁剪后的矩形查询；边缘像素分母较小，不能固定用 $(2r+1)^2$。条件是“小于等于”。如果每个像素重算整个邻域，工作量最多约 $600^2 \times 201^2$，应预处理。**做题思路与解答**。建二维前缀和，查询每格邻域矩形总和 S，实际格数为 A。用 $S \le tA$ 判定，无需浮点除法。二维容斥保证求出的和恰好是裁剪矩形的所有像素；整数不等式与平均值条件等价。

输入输出样例

样例输入

```
4 16 1 6
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
```

样例输出

```
7
```

复杂度与边界。预处理和全部查询均为 $O(n^2)$，空间 $O(n^2)$。0 行 0 列作为哨兵；r 大于图像边长时，裁剪仍正确。

完整参考实现 42_neighborhood.cpp

```cpp
#include <cstdio>
#include <algorithm>
long long s[605][605];
int main() {
    int n, L, r, t; scanf("%d%d%d%d", &n, &L, &r, &t);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int x; scanf("%d", &x);
            s[i][j] = s[i-1][j]+s[i][j-1]-s[i-1][j-1]+x;
        }
    }
    int answer = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            int x1 = std::max(1, i-r), x2 = std::min(n, i+r);
            int y1 = std::max(1, j-r), y2 = std::min(n, j+r);
            long long sum = s[x2][y2]-s[x1-1][y2]
                            -s[x2][y1-1]+s[x1-1][y1-1];
            long long area = 1LL*(x2-x1+1)*(y2-y1+1);
            if (sum <= area*t) answer++;
        }
    }
    printf("%d\n", answer);
    return 0;
}
```

3.3 垦田计划

第 29 次 CCF-CSP 认证，2023-03，第 2 题。垦田计划（CSP202303B）。**题目与输入输出**。n 块田同时开垦，第 i 块原耗时 t_i；每投入 c_i 资源能缩短 1 天，但最少需要 k 天。共 m 资源，求全部田完成的最少天数。输入 n、m、k，随后 n 行 t_i, c_i。$n, t_i, c_i \le 10^5, m \le 10^9, 1 \le k \le \min t_i$；输出最少天数。**读题思路**。全部田并行，目标是最大单块耗时，不是耗时总和。每块都不能低于 k，资源需求可能达 10^{15}，累加和乘法用 long long。**做题思路与解答**。固定目标天数 T，要让所有 $t_i \le T$，第 i 块至少投入 $\max(0, t_i - T)c_i$，这些资源需求互不替代，和不超过 m 便可行。T 越大成本越低，可行性为“假、假、真、真”；二分 $[k, \max t_i]$ 中第一个可行值，取下中位数，可行令 hi=mid，不可行令 lo=mid+1。每块的必要投入可同时实现，所以判定充分且必要。

输入输出样例

样例输入

```
4 9 2
6 1
5 1
6 2
7 1
```

样例输出

```
5
```

输入输出样例

样例输入

```
4 30 2
6 1
5 1
6 2
7 1
```

样例输出

```
2
```

复杂度与边界。时间 $O(n \log \max t_i)$，空间 $O(n)$。没有资源时答案为最大原耗时；资源很多时也不能小于 k。

完整参考实现 `56_farming.cpp`

```cpp
#include <cstdio>
int t[100005], c[100005], n; long long budget;
bool possible(int days) {
    long long cost = 0;
    for (int i = 0; i < n; i++) {
        if (t[i] > days) cost += 1LL*(t[i]-days)*c[i];
        if (cost > budget) return false;
    }
    return true;
}
int main() {
    int k, hi = 0; scanf("%d%lld%d", &n, &budget, &k);
    for (int i = 0; i < n; i++) {
        scanf("%d%d", &t[i], &c[i]); if (t[i] > hi) hi = t[i];
    }
    int lo = k;
    while (lo < hi) {
        int mid = lo+(hi-lo)/2;
        if (possible(mid)) hi = mid;
        else lo = mid+1;
    }
    printf("%d\n", lo);
    return 0;
}
```

3.4 相似度计算

第 33 次 CCF-CSP 认证，2024-03，第 2 题。相似度计算（CSP202403B）。**题目与输入输出**。输入两篇文章的单词个数 n、m，随后分别输入 n 和 m 个仅含英文字母的单词。忽略大小写并去重，分别输出两个集合的交集大小、并集大小，每个一行。曙梦范围 $n, m \leq 5 \times 10^5$，每词长度不超过 10；原资料 AcWing 页面给 $n, m \leq 10^4$。**读题思路**。题名涉及相似度，但不输出相似度小数；只输出两个计数。同篇重复单词只算一次，大小写必须统一。按较大的曙梦数据范围选择实现。**做题思路与解答**。用哈希表记录每个规范化单词在哪篇出现：位 1 表示第一篇，位 2 表示第二篇。第一篇按位或 1，第二篇按位或 2，重复不会改变状态。表中键数就是并集大小，标记为 3 的键数就是交集大小。哈希表平均查找时间为常数；若用 set 或 map 则有对数因子，适合先练语义后比较效率。

输入输出样例

样例输入

```
3 2
The tHe thE
the THE
```

样例输出

```
1
1
```

输入输出样例

样例输入

```
9 7
Par les soirs bleus dete jirai dans les sentiers
PICOTE PAR LES BLES FOULER LHERBE MENUE
```

样例输出

```
2
13
```

复杂度与边界。设词长上界为 W，平均时间 $O((n+m)W)$，空间与不同词数 u 成正比；哈希冲突严重时性能会退化，不提供最坏常数时间保证。reserve 预留容量降低扩容次数，不能保证没有碰撞。

完整参考实现 57_similarity.cpp

```cpp
#include <cstdio>
#include <string>
#include <unordered_map>
int main() {
    int n, m; scanf("%d%d", &n, &m);
    std::unordered_map<std::string, int> mask;
    mask.max_load_factor(0.7f); mask.reserve(n+m);
    for (int article = 0; article < 2; article++) {
        int count = article == 0 ? n : m;
        for (int i = 0; i < count; i++) {
            char word[11]; scanf("%10s", word);
            for (int j = 0; word[j]; j++)
                if (word[j] >= 'A' && word[j] <= 'Z') word[j] += 'a'-'A';
            mask[std::string(word)] |= article == 0 ? 1 : 2;
        }
    }
    int intersection = 0;
    for (std::unordered_map<std::string, int>::const_iterator it = mask.begin();
         it != mask.end(); ++it) if (it->second == 3) intersection++;
    printf("%d\n%d\n", intersection, (int)mask.size());
    return 0;
}
```

3.5 机器人饲养指南

第 37 次 CCF-CSP 认证，2025-03，第 2 题。机器人饲养指南（CSP202503B）。**题目与输入输出。**有 n 个苹果，每天可喂机器人 i 个苹果（$1 \le i \le m$），当天奖励为 A_i，同一种喂法可重复使用。所有苹果都必须喂完，求总奖励最大值。输入 n、m，再输入 $A_1, \ldots, A_m$；输出最优奖励。$1 \le n \le 10^4$，$1 \le m \le 100$，$0 \le A_i \le 10^5$，$A_0 = 0$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**每天选择可以重复，但资源要求是“恰好用完”。奖励既不保证递增，也不保证单价递增，按单日奖励最大贪心没有依据。$nm \le 10^6$，一维 DP 可行。**做题思路与解答。**令 $f[v]$ 表示恰好吃掉 v 个苹果的最大收益。按最后一天吃 i 个苹果分类，$f[v] = \max_{1 \le i \le \min(m,v)}(f[v-i] + A_i)$。$f[0] = 0$，其余先设为负无穷，按 v 从小到大计算。每个方案都有最后一天，且前面的最优方案可以接上这一天，所以转移不漏方案，并保留最大奖励。

输入输出样例

样例输入

```
10 5
1 3 5 3 1
```

样例输出

```
16
```

输入输出样例

样例输入

```
4 3
1 60 100
```

样例输出

```
120
```

复杂度与边界。时间 $O(nm)$，空间 $O(n+m)$。第二组样例中先吃 3 再吃 1 只有 101，而两次吃 2 得 120，直接说明贪心会失败。

完整参考实现 44_feeding.cpp

```cpp
#include <cstdio>
long long f[10005];
int value[105];
int main() {
    int n, m; scanf("%d%d", &n, &m);
    for (int i = 1; i <= m; i++) scanf("%d", &value[i]);
    const long long NEG = -(1LL << 60);
    for (int v = 1; v <= n; v++) f[v] = NEG;
    f[0] = 0;
    for (int v = 1; v <= n; v++) {
        for (int i = 1; i <= m && i <= v; i++) {
            if (f[v-i] != NEG && f[v-i]+value[i] > f[v])
                f[v] = f[v-i]+value[i];
        }
    }
    printf("%lld\n", f[n]);
    return 0;
}
```

4 考点总结

模型	适用条件	必须检查	对应题目
前缀和	静态多次区域和	0 号哨兵、容斥符号、整数范围	邻域均值
前后缀汇总	单点变化影响一整段	变化前后的分界是否包含当前位置	梦境巡查
差分	修改全部累积后再恢复	r+1 边界、恢复的成本	画图
二分答案	判定沿答案单调	证明单调、哨兵、区间缩小、乘法	机器人宿管
双指针	有序性或窗口单调性	安全移动理由、重复值、总移动次数	买菜、稀疏向量
贪心	有交换论证或最优前缀性质	反例、每次选择能否固定	分蛋糕

模型	适用条件	必须检查	对应题目
质因数分解	试除数量可承受	完整指数、残留质数、n 变小	因子化简
搜索	状态完整、规模允许	去重、撤销、剪枝不能漏解	机器人复健
动态规划	子问题可复用	状态、转移、初值、顺序、答案位置	机器人饲养指南
STL	操作与数据性质吻合	严格比较、空容器、失效、复杂度	数字排序、相似度计算

复盘要求：给每题写三句话：为什么直接枚举会慢（或其实够用）；利用了哪个性质；若输入去掉排序、正数、静态或恰好装满等条件，原算法哪一步会失效。能回答这些问题，比记住代码模板更有用。

5 练习题

先独立完成，再看题后解答。原题单十五道第二题中，五道在真题速看展开，其余十道在本节提供解答与完整实现。先按原题单完成，再做算法专项补充。

5.1 梦境巡查

第 36 次 CCF-CSP 认证，2024-12，第 2 题。梦境巡查（CSP202412B）。**题目与输入输出**。输入 n，接着输入 $a_0,\ldots,a_n$ 及 $b_1,\ldots,b_n$。从 0 号区域出发，走第 i 段消耗 a_i，到区域 i 后得到 b_i；每段出发前能量须够支付。当仅第 i 个补给失效时，求完成整条路线并返回 0 号区域所需的最少初始能量 w_i，输出 n 个结果。$1 \le n \le 10^5$，所有消耗与补给在 0 到 1000 之间。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。n 次逐段模拟为 $O(n^2)$，规模不允许。缺失 b_i 只影响从区域 i 出发及以后的检查；区域 i 之前不受影响。**做题思路与解答**。设 $d_j = \sum_{t=0}^{j} a_t - \sum_{t=1}^{j} b_t$，表示出发走第 j 段前，对初始能量的下界。缺失第 i 个补给后，$j < i$ 仍需 d_j，$j \ge i$ 则需 $d_j + b_i$。分别预处理 d 的前缀最大值 P 和后缀最大值 S，得到 $w_i = \max(P_{i-1}, S_i + b_i)$。因为 $d_0 = a_0 \ge 0$，无需额外把负答案截成 0。每个出发检查都满足当且仅当初始值不低于这些下界的最大值，因此这是最小可行值。

输入输出样例

样例输入

```
3
5 5 5 5
0 100 0
```

样例输出

```
10 20 10
```

输入输出样例

样例输入

```
3
9 4 6 2
9 4 6
```

样例输出

```
15 10 9
```

复杂度与边界。时间 $O(n)$，空间 $O(n)$。检查 b 本来就是 0 和巨额补给位于中间的情况。

完整参考实现 45_dream.cpp

```cpp
#include <cstdio>
long long a[100005], b[100005], need[100005];
long long pre[100005], suf[100007];
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i <= n; i++) scanf("%lld", &a[i]);
    for (int i = 1; i <= n; i++) scanf("%lld", &b[i]);
    need[0] = a[0];
    for (int i = 1; i <= n; i++) need[i] = need[i-1]+a[i]-b[i];
    pre[0] = need[0];
    for (int i = 1; i <= n; i++)
        pre[i] = pre[i-1] > need[i] ? pre[i-1] : need[i];
    suf[n] = need[n];
    for (int i = n-1; i >= 0; i--)
        suf[i] = suf[i+1] > need[i] ? suf[i+1] : need[i];
    for (int i = 1; i <= n; i++) {
        long long answer = pre[i-1] > suf[i]+b[i] ?
                           pre[i-1] : suf[i]+b[i];
        printf("%lld%c", answer, i == n ? '\n' : ' ');
    }
    return 0;
}
```

5.2 机器人宿管指南

第 42 次 CCF-CSP 认证，2026-05，第 2 题。机器人宿管指南（CSP202605B）。**题目与输入输出**。输入初始苹果数 n、损耗百分比 k、天数 m。每天先丢掉 $\lceil tk/100 \rceil$ 个坏苹果（t 是前一天剩余量），再给每个机器人一个苹果。求最多能供养多少机器人，要求第 m 天吃完后剩余非负。$1 \le n \le 10^{18}$，$1 \le k \le 5$，$1 \le m \le 2000$；保证答案为不超过 10^9 的正整数。**评测限制**。曙梦 OJ 当前时限 0.5 秒，内存 512MB。**读题思路**。顺序是先变质后食用，百分比向上取整。n 超过 int 范围，即使 long long

也不宜直接算 $n \times 100$；不能用浮点数近似大整数比例。**做题思路与解答**。人数 x 给定后按天模拟检查。剩余映射为 $\lfloor t(100-k)/100 \rfloor - x$，对 t 单调不减，所以人数越少，全程剩余越多，可行人数形成前缀。二分最后一个可行人数，范围 $[0, \min(10^9, n/m)]$。损耗按 $\lfloor t/100 \rfloor k + \lceil (t \bmod 100)k/100 \rceil$ 计算，各项都在 long long 范围内。

输入输出样例

样例输入

```
1000 1 5
```

样例输出

```
194
```

输入输出样例

样例输入

```
19 5 3
```

样例输出

```
5
```

复杂度与边界。时间 $O(m \log 10^9)$，空间 $O(1)$。0 作为可行哨兵，题面保证最终答案为正。第 m 天可以恰好剩 0，不要求还能吃第 m+1 天。

完整参考实现 46_dorm.cpp

```cpp
#include <cstdio>
long long n; int k, m;
bool enough(long long robots) {
    long long remain = n;
    for (int day = 1; day <= m; day++) {
        // ceil(remain*k/100), without overflowing remain*k.
        long long bad = (remain/100)*k + ((remain%100)*k+99)/100;
        remain -= bad;
        if (remain < robots) return false;
        remain -= robots;
    }
    return true;
}
int main() {
    scanf("%lld%d%d", &n, &k, &m);
    long long lo = 0, hi = n/m;
    if (hi > 1000000000LL) hi = 1000000000LL;
    while (lo < hi) {
        long long mid = lo+(hi-lo+1)/2;
        if (enough(mid)) lo = mid;
        else hi = mid-1;
    }
    printf("%lld\n", lo);
    return 0;
}
```

5.3 字符串变换

第 35 次 CCF-CSP 认证，2024-09，第 2 题。字符串变换（CSP202409B）。**题目与输入输出**。字符集合为大小写字母、数字和空格，共 63 种。输入以 # 包围的初始字符串，再给 n 个形如 #xy# 的字符替换规则，未定义的字符保持不变；规则构成单射。随后输入 m 与 m 个 k，每次独立询问初始串连续变换 k 次的结果，以 # 包围输出。$n \le 63, m \le 1000, k \le 10^9$，初始串长不超过 100。**评测限制**。曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路**。保留空格，不能用%s 读整行；每次查询从原始串开始。每轮替换是同时进行，不能按规则就地多次替换同一个位置。巨大 k 不能逐轮模拟。**做题思路与解答**。预处理 $jump[b][c] = f^{2^b}(c)$，初值为一次变换，递推 $jump[b][c] = jump[b-1][jump[b-1][c]]$。把 k 拆成二进制，对每个字符依次应用对应的跳表即可。每个字符独立变化，函数复合保证跳表恰好等于所需次数。单射还允许按置换环长度取模；这里的倍增实现无需依赖单射，便于统一处理。

输入输出样例

样例输入

```
#Hello World#
6
#HH#
#e #
# r#
#re#
#oa#
#ao#
3
1 2 3
```

样例输出

```
#H llarWaeld#
#HrlloeWo ld#
#Hella Warld#
```

复杂度与边界。字符表用 128 项覆盖 ASCII；时间 $O(128 \log K + m|s| \log K)$，空间 $O(128 \log K + |s|)$。输入的两个 # 是包装符号，不参与变换。

完整参考实现 47_transform.cpp

```cpp
#include <cstdio>
#include <cstring>
char s[105], line[110];
int next_char[128], jump[31][128];
void read_line(char text[], int cap) {
    fgets(text, cap, stdin);
    text[strcspn(text, "\r\n")] = 0;
}
void finish_line() { int c; while ((c=getchar()) != '\n' && c != EOF) {} }
int main() {
    read_line(s, sizeof(s));
    for (int c = 0; c < 128; c++) next_char[c] = c;
    int n; scanf("%d", &n); finish_line();
    for (int i = 0; i < n; i++) {
        read_line(line, sizeof(line));
        next_char[(unsigned char)line[1]] = (unsigned char)line[2];
    }
    for (int c = 0; c < 128; c++) jump[0][c] = next_char[c];
    for (int b = 1; b < 31; b++)
        for (int c = 0; c < 128; c++) jump[b][c] = jump[b-1][jump[b-1][c]];
    int m; scanf("%d", &m);
    while (m--) {
        int k; scanf("%d", &k); putchar('#');
        for (int i = 1; s[i] && s[i] != '#'; i++) {
            int c = (unsigned char)s[i];
            for (int b = 0; b < 31; b++) if ((k >> b) & 1) c = jump[b][c];
            putchar(c);
        }
        puts("#");
    }
    return 0;
}
```

5.4 机器人复健指南

第 38 次 CCF-CSP 认证，2025-06，第 2 题。机器人复健指南（CSP202506B）。**题目与输入输出。**输入 n、k 和起点 x、y。在 $n \times n$ 场地内每次按马步跳跃，即坐标变化为 $(\pm 1, \pm 2)$ 或 $(\pm 2, \pm 1)$，共八个方向，不得跳出场地。统计不超过 k 步可到达的不同格数，包含起点。$1 \le n, k \le 100$。输出格数。**读题思路。**图中的八方向是马步，不是上下左右及四个对角相邻格。要求不超过 k 步，且一个格子多次到达只计一次。**做题思路与解答。**BFS 把每个格子作为状态，首次访问得到最短步数。到达第 k 层就不再扩展；每个首次入队的格子给答案加 1，起点先计入。无权边 BFS 按距离递增发现状态，所以所有且仅有最短距离不超过 k 的格子被统计。也能用“步数、位置”的分层可达 DP，再对各层取并集，但 BFS 不用重复访问位置，复杂度更低。

输入输出样例

样例输入

```
4 1
1 1
```

样例输出

```
3
```

输入输出样例

样例输入

```
4 2
1 1
```

样例输出

```
8
```

复杂度与边界。时间 $O(n^2)$，空间 $O(n^2)$，每格最多一次入队。n=1 或 2 时可能完全没有合法马步；答案仍包含起点。

完整参考实现 48_knight.cpp

```cpp
#include <cstdio>
#include <cstring>
struct Point { int x, y; } q[10005];
int dist[105][105];
int dx[8] = {1,1,-1,-1,2,2,-2,-2};
int dy[8] = {2,-2,2,-2,1,-1,1,-1};
int main() {
    int n, k, sx, sy; scanf("%d%d%d%d", &n, &k, &sx, &sy);
    memset(dist, -1, sizeof(dist));
    int head = 0, tail = 0, answer = 1;
    q[tail++] = {sx, sy}; dist[sx][sy] = 0;
    while (head < tail) {
        Point p = q[head++];
        if (dist[p.x][p.y] == k) continue;
        for (int d = 0; d < 8; d++) {
            int x = p.x+dx[d], y = p.y+dy[d];
            if (x < 1 || x > n || y < 1 || y > n || dist[x][y] != -1)
                continue;
            dist[x][y] = dist[p.x][p.y]+1;
            q[tail++] = {x,y}; answer++;
        }
    }
    printf("%d\n", answer);
    return 0;
}
```

5.5 回收站选址

第 18 次 CCF-CSP 认证，2019-12，第 2 题。回收站选址（CSP201912B）。**题目与输入输出**。输入 n 个两两不同的垃圾点坐标。只有本身有垃圾且上下左右四邻都存在的点可以选址；得分为四个对角邻点中有垃圾的个数。依次输出得分 0 到 4 的选址数量，各一行。$n \le 1000$，坐标绝对值不超过 10^9。**读题思路**。地图坐标巨大且可负，不能按坐标开完整网格。只有已有垃圾点能成为候选；四个正交邻居是资格条件，四个对角邻居只负责评分。**做题思路与解答**。用 set 保存坐标 pair。枚举每个原点，查询四个正交邻点，全部存在后统计四个对角邻点并增加相应频率。set 的键同时包含横纵坐标，查询精确匹配，所以资格和得分都符合定义。n 很小时也可用数组线性查找作为对照。

输入输出样例

样例输入

```
7
1 2
2 1
0 0
1 1
1 0
2 0
0 1
```

样例输出

```
0
0
1
0
0
```

输入输出样例

样例输入

```
2
0 0
-100000 10
```

样例输出

```
0
0
0
0
0
```

复杂度与边界。时间 $O(n \log n)$，空间 $O(n)$。输入坐标绝对值至多 10^9，加减 1 仍在 int 范围内；n 为 1 时输出五个 0。

完整参考实现 58_recycle.cpp

```cpp
#include <cstdio>
#include <set>
#include <utility>
std::set<std::pair<int,int> > points;
int x[1005], y[1005], answer[5];
bool exists(int x, int y) {
    return points.find(std::make_pair(x,y)) != points.end();
}
int main() {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%d%d", &x[i], &y[i]); points.insert(std::make_pair(x[i],y[i]));
    }
    for (int i = 0; i < n; i++) {
        int a = x[i], b = y[i];
        if (!exists(a+1,b) || !exists(a-1,b) ||
            !exists(a,b+1) || !exists(a,b-1)) continue;
        int score = exists(a-1,b-1)+exists(a-1,b+1)
                  +exists(a+1,b-1)+exists(a+1,b+1);
        answer[score]++;
    }
    for (int i = 0; i < 5; i++) printf("%d\n", answer[i]);
    return 0;
}
```

5.6 风险人群筛查

第 20 次 CCF-CSP 认证，2020-09，第 2 题。风险人群筛查（原资料入口）。**题目与输入输出。**输入 n、k、t 及矩形左下 xl、yd、右上 xr、yu；再输入每人的 t 个记录坐标。含边界的矩形内出现过记录即“经过”；有至少 k 条连续记录都在内即“逗留”。输出经过人数和逗留人数，各一行。采用原资料页面较宽的 $n \le 24$，$1 \le k \le t \le 1000$，坐标绝对值不超过 10^6。**读题思路。**只检查采样记录，不插值判断相邻记录间的连线。逗留强调连续；分散的 k 次进入不算。矩形边界也在区域内。**做题思路与解答。**对每人维护曾进入标记、当前连续进入长度 run、曾逗留标记。内点令 run 加 1，外点清 0；run 达到 k 后保留逗留标记，即使后来离开也不取消。每人结束时各标记只加一次计数。

输入输出样例

样例输入

```
1 3 8 0 0 10 10
-1 -1 0 0 0 0 -1 -1 0 0 -1 -1 0 0 0 0
```

样例输出

```
1
0
```

复杂度与边界。时间 $O(nt)$，额外空间 $O(1)$。已经确定逗留后仍须读取剩余坐标，否则下个人输入错位。

完整参考实现 59_risk.cpp

```cpp
#include <cstdio>
int main() {
    int n, k, t, xl, yd, xr, yu;
    scanf("%d%d%d%d%d%d%d", &n, &k, &t, &xl, &yd, &xr, &yu);
    int pass = 0, stay = 0;
    for (int person = 0; person < n; person++) {
        bool touched = false, stayed = false; int run = 0;
        for (int j = 0; j < t; j++) {
            int x, y; scanf("%d%d", &x, &y);
            bool inside = xl <= x && x <= xr && yd <= y && y <= yu;
            if (inside) { touched = true; run++; }
            else run = 0;
            if (run >= k) stayed = true;
        }
        pass += touched; stay += stayed;
    }
    printf("%d\n%d\n", pass, stay);
    return 0;
}
```

5.7 非零段划分

第 23 次 CCF-CSP 认证，2021-09，第 2 题。非零段划分（原资料入口）。**题目与输入输出。**输入 n 项非负整数 A，选择正整数 p，把所有小于 p 的值变 0，求剩余连续非零段数量的最大值。一个非零段必须不能再向左或向右扩展。$n \le 5 \times 10^5, A_i \le 10^4$。输出最大段数。**读题思路。**阈值变化时段数不单调，不能二分“段数达到某值”。直接枚举每个 p 并扫描全数组约 5×10^9 次，需利用值域。**做题思路与解答。**每段恰好由一个开头贡献。设 $A_0 = 0$，位置 i 成为开头当且仅当 $A_{i-1} < p \le A_i$。因此当 $A_{i-1} < A_i$ 时，给阈值区间 $[A_{i-1}+1, A_i]$ 贡献 1，用差分记录；遍历阈值恢复段数并取最大值。每个真实段只有一个开头，条件与开头定义等价，所以统计不重不漏。

输入输出样例

样例输入

```
11
3 1 2 0 0 2 0 4 5 0 2
```

样例输出

```
5
```

输入输出样例

样例输入

```
14
5 1 20 10 10 10 10 15 10 20 1 5 10 15
```

样例输出

```
4
```

复杂度与边界。时间 $O(n+V)$，V=10^4，空间 $O(V)$。数组全 0 时答案 0；为 A_i+1 处预留容量。

完整参考实现 60_nonzero.cpp

```cpp
#include <cstdio>
int delta[10005];
int main() {
    int n, previous = 0; scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int a; scanf("%d", &a);
        // Position i starts a surviving segment iff previous < p <= a.
        if (previous < a) { delta[previous+1]++; delta[a+1]--; }
        previous = a;
    }
    int segments = 0, answer = 0;
    for (int p = 1; p <= 10000; p++) {
        segments += delta[p]; if (segments > answer) answer = segments;
    }
    printf("%d\n", answer);
    return 0;
}
```

5.8 出行计划

第 25 次 CCF-CSP 认证，2022-03，第 2 题。出行计划（原资料入口）。**题目与输入输出。**输入计划数 n、查询数 m、等待结果时长 k；随后 n 行出行时刻 t_i 和有效时长 c_i，最后 m 个检测时刻 q。q 检测的结果可从 $q+k$ 到 $q+k+c_i-1$ 进入对应场所。每查询输出符合条件的计划数。$n, m, k \le 10^5$，$t_i, c_i, q \le 2 \times 10^5$，均为正整数。**读题思路。**有效时间包括刚获得结果的时刻，末端是有效时长减 1。把“这次检测满足哪些计划”反过来，求“每项计划允许在哪些时刻检测”。**做题思路与解答。**条件 $q+k \le t_i \le q+k+c_i-1$ 等价于 $t_i - c_i + 1 - k \le q \le t_i - k$。每项计划给这个整数区间加 1，按 q 正数范围裁剪后做差分；一次前缀累加得到每个 q 的答案。等价变形保证区间内恰好计入满足要求的计划。

输入输出样例

样例输入

```
6 2 10
5 24
10 24
11 24
34 24
35 24
35 48
1
2
```

样例输出

```
3
3
```

复杂度与边界。时间 $O(n+m+V)$，V=2×10^5，空间 $O(V)$。右端为负或裁剪后左端大于右端时跳过；r+1 必须有空间。

完整参考实现 61_travel.cpp

```cpp
#include <cstdio>
int d[200005];
int main() {
    int n, m, k; scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < n; i++) {
        int t, c; scanf("%d%d", &t, &c);
        int l = t-c+1-k, r = t-k;
        if (l < 1) l = 1;
        if (l <= r) { d[l]++; d[r+1]--; }
    }
    for (int q = 1; q <= 200000; q++) d[q] += d[q-1];
    while (m--) { int q; scanf("%d", &q); printf("%d\n", d[q]); }
    return 0;
}
```

5.9 寻宝！大冒险！

第 26 次 CCF-CSP 认证，2022-06，第 2 题。寻宝！大冒险！（CSP202206B）。**题目与输入输出。**地图是坐标 0 到 L 的方格点，共 n 棵树，输入 n、L、S 及 n 个树坐标，再输入 $(S+1)\times(S+1)$ 的 0/1 藏宝图。图的第一维按 S 到 0 输入，第二维按 0 到 S 输入。藏宝图左下格 $B[0][0]=1$。求所有与藏宝图完全匹配且不越地图边界的左下位置数。$n\le 1000, L\le 10^9, S\le 50$，树坐标不重复。**读题思路。**不能按 L 开地图，也不能枚举所有坐标。左下格保证有树，只需枚举 n 棵树为候选。匹配要求 1 与 0 都一致，不能只找到规定的树就忽略多余树。输入首行对应 B[S]，不要按 B[0] 读。**做题思路与解答。**先数藏宝图中的树数 need。对每个候选，拒绝越界窗口；扫描全部地图树，若落入窗口却对应藏宝图 0，立即拒绝，否则统计窗口树数。没有多余树且 count=need 时，一一对应保证所有藏宝图 1 也都有树，其余格为 0，故整个窗口完全匹配。

输入输出样例

样例输入

```
5 100 2
0 0
1 1
2 2
3 3
4 4
0 0 1
0 1 0
1 0 0
```

样例输出

```
3
```

输入输出样例

样例输入

```
5 4 2
0 0
1 1
2 2
3 3
4 4
0 0 0
0 1 0
1 0 0
```

样例输出

```
0
```

复杂度与边界。时间 $O(n^2+S^2)$，空间 $O(n+S^2)$。代码严格按题面 $B[S-i][j]$ 的输入下标存储；不额外转置。

完整参考实现 62_treasure.cpp

```cpp
#include <cstdio>
int x[1005], y[1005], pattern[55][55];
int main() {
    int n, L, S; scanf("%d%d%d", &n, &L, &S);
    for (int i = 0; i < n; i++) scanf("%d%d", &x[i], &y[i]);
    int need = 0;
    for (int row = S; row >= 0; row--)
        for (int col = 0; col <= S; col++) {
            scanf("%d", &pattern[row][col]); need += pattern[row][col];
        }
    int answer = 0;
    for (int i = 0; i < n; i++) {
        if (x[i]+S > L || y[i]+S > L) continue;
        bool ok = true; int count = 0;
        for (int j = 0; j < n; j++) {
            int a = x[j]-x[i], b = y[j]-y[i];
            if (a < 0 || a > S || b < 0 || b > S) continue;
            if (!pattern[a][b]) { ok = false; break; }
            count++;
        }
        if (ok && count == need) answer++;
    }
    printf("%d\n", answer);
    return 0;
}
```

5.10 何以包邮？

第 27 次 CCF-CSP 认证，2022-09，第 2 题。何以包邮？（原资料入口）。**题目与输入输出**。输入 n、包邮阈值 x，随后 n 本书的正整数价格，每本只能选一次。求总价至少 x 的最小选购总价，保证全部价格和不小于 x。$n\le 30, a_i\le 10^4$，故总价 S 不超过 3×10^5。输出最小总价。**读题思路**。不是花费不超过 x 的最大价值，而是总价至少 x 且最小。2^{30} 枚举太大；利用价格和上界做 01 可达性背包。**做题思路与解答**。reachable[s] 表示用已处理书本能否恰好买到 s 元。初值仅 0 为真，每

本价格 a 加入时从 S 向 a 倒序更新 $reachable[s] \mid= reachable[s-a]$，保证当前书只使用一次。全部处理后从 x 向上找首个可达 s，就是满足包邮的最小价格。

输入输出样例

样例输入

```
4 100
20
90
60
60
```

样例输出

```
110
```

输入输出样例

样例输入

```
3 30
15
40
30
```

样例输出

```
30
```

复杂度与边界。时间 $O(nS)$，空间 $O(S)$。重复价格仍代表不同书，不能去重；全部购买可行保证扫描能找到答案。

完整参考实现 63_shipping.cpp

```cpp
#include <cstdio>
int a[35]; bool reachable[300005];
int main() {
    int n, x, total = 0; scanf("%d%d", &n, &x);
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); total += a[i]; }
    reachable[0] = true;
    for (int i = 0; i < n; i++)
        for (int s = total; s >= a[i]; s--)
            if (reachable[s-a[i]]) reachable[s] = true;
    for (int s = x; s <= total; s++)
        if (reachable[s]) { printf("%d\n", s); break; }
    return 0;
}
```

5.11 机器人饲养指南：完全背包复做

第 37 次，2025-03，第 2 题。机器人饲养指南（CSP202503B）。题面、样例和完整程序见本篇第三节。遮住答案重新写出 $f[v]$，并回答：如果允许剩苹果，初始化和答案位置会有什么变化？**解答**：当前题要求恰好用完，不能把所有状态初始化为 0。若改为最多吃 n 个且价值非负，可用“不超过容量”的全 0 初值，也可保留恰好状态并取 $\max_{0\le v\le n} f[v]$。两种状态解释要与实现一致。

5.12 数字排序：STL 比较规则复做

第 4 次，2015-03，第 2 题。数字排序（CSP201503B）。**任务。**输入 n 个整数，统计每个不同值的出现次数，按次数从大到小输出“值次数”；次数相同按值从小到大输出。**解答与关键代码。**值域小时先用桶计数，再把非零记录存入结构体数组排序。值域大时可用 map 计数后生成记录。比较函数只表达严格顺序，不能用大于等于。

```
struct Record { int value, count; };
bool less_record(const Record &a, const Record &b) {
    if (a.count != b.count) return a.count > b.count;
    return a.value < b.value;
}
// std::sort(records, records+total, less_record);
```

排序时间 $O(u \log u)$，u 是不同值个数；还需加上计数过程的成本。与第一篇众数题区别在于，这里输出全部不同值，并有两层排序键。

5.13 其他算法专项训练

以下题按本篇算法顺序安排，关键方法在相应小节已经解释；洛谷仅用于原资料中没有已核对曙梦入口的专项训练。

做题链接	训练点	解答要点与边界
画图（第 2 次）（CSP201409B）	二维差分	每块矩形标记四角，恢复后统计覆盖数大于 0 的单位格，重叠只计一次。
相似度计算（第 33 次）（CSP202403B）	string、set	先统一大小写，每篇各自去重，统计交集与并集；同篇重复出现不能反复计数。
矩阵运算（第 30 次）（CSP202305B）	结合律、复杂度	对 QK^TV 先算 K^TV，避免构造 $n \times n$ 中间矩阵；输出还需按题意乘对应行权重。
P2678 跳石头	二分、贪心	固定距离后从左到右判断最少移走数量，终点不可移走。
P1706 全排列	DFS	选未使用元素，递归后撤销；原题要求每数宽度 5。
P1443 马的遍历	BFS	八个马步方向，首次入队记录最短距离，不可达输出-1。
B3637 最长上升子序列	序列 DP	严格上升用 $a_j < a_i$；大范围时对 tails 使用 lower_bound。
P1115 最大子段和	递推 DP	$f_i = \max(a_i, f_{i-1} + a_i)$；全负时不能把答案设成 0。

做题链接	训练点	解答要点与边界
P1048 采药	01 背包	每件一次，容量倒序；容量含义为最多使用。
P1616 疯狂的采药	完全背包	每件不限次数，容量正序；价值总和与时限重新估算。
P1757 通天之分组背包	分组背包	每组至多一件，不能让同组候选相互接续。

5.14 算法补充真题：稀疏存储与基础数学

以下两题来自历年考点补充，均在算法讲解旁给出做题入口。

5.15 稀疏向量

第 19 次 CCF-CSP 认证，2020-06，第 2 题。稀疏向量（CSP202006B）。**题目与输入输出。**长度为 n 的两个向量，只有 a、b 项非零。每个向量的非零项按下标严格递增输入，每项为下标、数值；求内积 $\sum_{i=1}^{n} u_i v_i$。首行 n、a、b，随后 a 行 u 和 b 行 v。$n \le 10^9$，$0 < a, b < n$，$a, b \le 5 \times 10^5$，非零数值绝对值不超过 100。输出内积。**评测限制。**曙梦 OJ 当前时限 2 秒，内存 512MB。**读题思路。**n 很大，但非零项数较小，按 n 开数组会浪费数 GiB。只有相同下标的两项才产生非零乘积，排序信息已经给出。最多约 5×10^9 的绝对值累加，需要 long long。**做题思路与解答。**将每组存成下标和值的结构体数组。下标较小的那项在另一组当前及后续项中不可能匹配，可以丢弃；下标相同则乘积累加并同时前进。每个共同非零位置只遇到一次，其他位置贡献为 0，所以扫描值等于原内积。

输入输出样例

样例输入

```
10 3 4
4 5
7 -3
10 1
1 10
4 20
5 30
7 40
```

样例输出

```
-20
```

复杂度与边界。时间 $O(a+b)$，空间 $O(a+b)$，与完整向量长度 n 无关。若改成映射也能做，但已有有序性时双指针更简单。

完整参考实现 41_sparse.cpp

```cpp
#include <cstdio>
struct Item { int index, value; } a[500005], b[500005];
int main() {
    int n, ca, cb; scanf("%d%d%d", &n, &ca, &cb);
    for (int i = 0; i < ca; i++) scanf("%d%d", &a[i].index, &a[i].value);
    for (int j = 0; j < cb; j++) scanf("%d%d", &b[j].index, &b[j].value);
    long long answer = 0;
    int i = 0, j = 0;
    while (i < ca && j < cb) {
        if (a[i].index < b[j].index) i++;
        else if (a[i].index > b[j].index) j++;
        else { answer += 1LL*a[i].value*b[j].value; i++; j++; }
    }
    printf("%lld\n", answer);
    return 0;
}
```

5.16 因子化简

第 32 次 CCF-CSP 认证，2023-12，第 2 题。因子化简（CSP202312B）。**题目与输入输出。**对整数 n 进行质因数分解 $n = \prod p_i^{e_i}$，删除所有指数 $e_i < k$ 的整个因子 $p_i^{e_i}$，保留其余因子，输出结果；全部删去时为 1。输入 q，随后 q 行 n、k；每个结果一行。$1 \le q \le 10$，$1 < n \le 10^{10}$，$1 < k \le 10$。**评测限制。**曙梦 OJ 当前时限 1 秒，内存 512MB。**读题思路。**“删除指数不足的整个质因子幂”不是把每个指数减少 k，也不是只保留 p_i^k。例如 $360 = 2^3 \cdot 3^2 \cdot 5$，k=3 时答案为 8。n 超过 int 上界，读入必须用 long long。**做题思路与解答。**从 p=2 开始试除，每找到一个质因子就反复整除，记录完整指数以及对应幂。指数达标才把该幂乘入答案。条件 $p \le n/p$ 避免 p^2 溢出，也利用不断缩小的 n 减少试除。结束后残留大于 1 的数是指数为 1 的质因子；由于本题 k 至少为 2，它不保留。所有质因子都被处理且筛选严格按定义，得到所求乘积。

输入输出样例

样例输入

```
3
2155895064 3
2 2
10000000000 10
```

样例输出

```
2238728
1
10000000000
```

复杂度与边界。最坏时间 $O(q\sqrt{\max n})$，空间 $O(1)$。答案初值为 1，覆盖所有因子被删除的情况。

完整参考实现 43_factor.cpp

```cpp
#include <cstdio>
int main() {
    int q; scanf("%d", &q);
    while (q--) {
        long long n, answer = 1; int k;
        scanf("%lld%d", &n, &k);
        for (long long p = 2; p <= n/p; p++) {
            if (n % p != 0) continue;
            int count = 0; long long power = 1;
            do { n /= p; power *= p; count++; } while (n % p == 0);
            if (count >= k) answer *= power;
        }
        // Remaining prime has exponent 1; k is at least 2.
        printf("%lld\n", answer);
    }
    return 0;
}
```

第三部分　第三题：字符串解析与综合模拟

本篇围绕长题意的实现展开：先解析输入，再组织状态，最后执行规则并检查边界。遇到陌生背景时，先列出输入格式、状态定义、修改规则和查询规则，再写程序。

1　字符串处理与解析基础

1.1　常用操作与区间含义

真题练习：字符串匹配：第 2 次认证，按是否区分大小写查找子串，训练扫描与比较（CSP201409C）。

真题练习：字符串变换：第 35 次认证，按规则逐轮替换，完整解答见第二篇练习题（CSP202409B）。

string 的下标从 0 开始。`s.substr(pos,len)` 的第二个参数是长度；`s.find(t)` 返回首次匹配位置，找不到时返回 `string::npos`。不要把 npos 随意存进 int 后再与某个猜测值比较。字符串比较是字典序，例如 `"10" < "2"`，与整数大小不同。

扫描字符串时最好约定："指针 p 总是指向下一个尚未处理的字符"。每个分支要么推进 p，要么返回。若某个分支既不推进也不退出，就容易死循环。处理完整段落时，通常比反复在字符串中间 erase 更清楚、也更高效。

1.2　例题带引号的单行分词

题目。一行文本由普通字符、空格和英文双引号组成。引号外的连续空格分隔单词；引号内的空格属于单词；双引号本身不输出。相邻的普通片段和引号片段属于同一个单词。空引号表示一个空单词。本题不支持转义字符。若引号不匹配，输出 ERROR。行长不超过 10^5。

输入。一整行文本，可为空。**输出**。合法时先输出单词数，随后每个单词单独一行，用方括号包住，便于看见空单词和空格。

输入输出样例

样例输入

```
add "hello world" "" x
```

样例输出

```
4
[add]
[hello world]
[]
[x]
```

状态设计。quoted 表示当前是否在引号内；word[0..len-1] 保存当前单词；active 表示当前单词已经开始。active 不能由 len 是否为 0 代替，否则空引号会丢失。遇到引号切换 quoted；遇到引号外空格时结束当前单词；其余字符追加到 word 并增加 len。扫描结束后检查引号是否闭合，再处理最后一个单词。

正确性。每个字符恰好进入一种分支。引号仅改变状态，分隔空格仅结束一个已开始的单词，因此保留的字符、单词边界和空单词均符合规则。时间 $O(L)$，输出存储 $O(L)$。

完整参考实现 04_tokens.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 100005;
char s[N], word[N];
int main() {
    if (fgets(s, N, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    vector<string> words;
    int len = 0, quoted = 0, active = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '"') { quoted = !quoted; active = 1; }
        else if (c == ' ' && !quoted) {
            if (active) {
                word[len] = '\0'; words.push_back(word);
                len = 0; active = 0;
            }
        } else { word[len++] = c; active = 1; }
    }
    if (quoted) { puts("ERROR"); return 0; }
    if (active) { word[len] = '\0'; words.push_back(word); }
    printf("%d\n", (int)words.size());
    for (int i = 0; i < (int)words.size(); i++)
        printf("[%s]\n", words[i].c_str());
    return 0;
}
```

1.3 C 字符数组与 string 的区别

C 字符串以终止字符 '\0' 结尾，数组需要额外空间存放它。strlen 统计终止符前的字符数。定义后的字符数组不能用赋值表达式整体赋值；string 可以直接赋值。strcmp 返回值的符号表示字典序关系，不保证只返回 1 或-1。字符分类函数如 isdigit 接收字符时，可先转为 unsigned char，避免负 char 值带来的未定义行为。

1.4 解析与业务处理分开

例如命令 GRANT alice file1，解析层先产生三个字符串，再由执行层决定如何修改权限。这样出现错误时，可以先打印解析结果，判断错在读入还是状态更新。本篇下一节会进一步用函数处理路径和表达式。

1.5 练习与解答

练习 1。输入 a"b c"d，分词结果是什么？**解答**。只有一个单词 ab cd。相邻片段没有引号外空格，不应拆开。

练习 2。输入只有三个空格，输出什么？**解答**。单词数为 0。输入 "" 时则是一个空单词，输出 1 和 []。

练习 3。若题目新增反斜杠转义，当前程序能直接使用吗？**解答**。不能。需要新增 escape 状态：读取反斜杠后，下一个字符按普通字符处理，不切换引号状态。还要定义行末孤立反斜杠是否非法。规则改变后必须重新列出状态转移。

2 第三题综合模拟专题

2.1 长题意的实现流程

先写一张规则表：输入是什么、状态存在哪里、操作改变什么、查询输出什么、异常如何处理。再按职责拆成解析层、状态层和执行层。只有确认每层的输入输出后，再连成完整程序。下面五个案例规则完整，分别训练路径、映射状态、递归解析、树形结构和依赖展开，可以脱离外部题面学习。

2.2 例题路径规范化

真题练习：路径解析：第 7 次认证，处理绝对路径、相对路径以及点号路径段（CSP201604C）。

题目。给定固定的当前目录 cwd 和 q 条路径。以斜杠开头的是绝对路径，否则相对于 cwd。连续斜杠视为一个；点号目录表示当前目录；两个点号表示父目录，在根目录继续上移仍为根。普通目录名只含字母、数字、下划线。路径不含空格且非空。

输出规范绝对路径：以一个斜杠开头，不含点号、两个点号或重复斜杠；除根目录外不以斜杠结尾。**每条查询独立使用原来的 cwd，不改变当前目录**。路径长度不超过 10^4，全部输入路径总长度不超过 2×10^5，$1 \le q \le 10^4$。第一行 cwd，第二行 q，随后 q 行各一条路径。cwd 保证为绝对路径，也允许特殊片段。

输入输出样例

样例输入

```
/a/b
5
../c
/x//./y/../
../../../
.
tmp/../../z
```

样例输出

```
/a/c
/x
/
/a/b
/a/z
```

解答。用 vector 保存从根向下的目录名。普通片段入栈，两个点号在非空时弹栈，空片段和点号忽略。绝对路径从空栈开始，相对路径从 cwd 的副本开始。

最后一条查询从 $[a,b]$ 出发，读 tmp 得到 $[a,b,tmp]$，两次父目录操作后为 $[a]$，加入 z 后得到 $[a,z]$。不变量是：栈中的目录始终表示已经处理的路径前缀的规范形式。

设 cwd 长度为 B，第 i 个查询长度为 L_i。复制基目录的总成本最坏为 $O(qB)$，加上扫描与输出为 $O(qB+\sum L_i)$。不能漏算复制和输出中重复的 cwd 长度。

完整参考实现 19_path.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 10005;
char cwd[N], path[N];
void append_parts(const char s[], vector<string> &parts) {
    char item[N];
    int i = 0;
    while (s[i] != '\0') {
        while (s[i] == '/') i++;
        int len = 0;
        while (s[i] != '\0' && s[i] != '/') item[len++] = s[i++];
        item[len] = '\0';
        if (len == 0 || strcmp(item, ".") == 0) continue;
        if (strcmp(item, "..") == 0) {
            if (!parts.empty()) parts.pop_back();
        } else parts.push_back(item);
    }
}
int main() {
    int q;
    scanf("%10004s%d", cwd, &q);
    vector<string> base;
    append_parts(cwd, base);
    while (q--) {
        scanf("%10004s", path);
        vector<string> result;
        if (path[0] != '/') result = base;
        append_parts(path, result);
        if (result.empty()) printf("/");
        for (int i = 0; i < (int)result.size(); i++)
            printf("/%s", result[i].c_str());
        printf("\n");
    }
    return 0;
}
```

调试。先测根目录与普通绝对路径，再测相对路径和多次上移，最后检查是否错误地修改了共享 cwd。若后续查询受前面查询影响，优先检查是否复用了同一可变目录数组。

2.3 例题授权和撤销的状态管理

真题练习：角色授权：第 26 次认证；下面教学例题先练习较小的授权模型，原题还需处理角色关联与权限匹配（CSP202206C）。

题目。执行 q 条命令：`GRANT user resource` 授予权限；`REVOKE user resource` 撤销；`CHECK user resource` 查询并输出 YES 或 NO。重复授予不叠加；撤销不存在的权限不报错；未出现用户默认无权限。操作依次执行，只有 CHECK 输出。$1 \le q \le 2\times 10^5$，名称为 1 到 20 个字母或数字。本题不含角色继承、通配符或优先级。

输入输出样例

样例输入

```
8
CHECK alice file1
GRANT alice file1
GRANT alice file1
CHECK alice file1
REVOKE alice file1
CHECK alice file1
REVOKE bob file2
CHECK bob file2
```

样例输出

```
NO
YES
NO
NO
```

解答。外层 map 按用户名定位，内层 set 保存资源。set 处理重复授权。查询和撤销用 find，避免无意创建空记录。所有操作依次执行，因此 CHECK 反映此前的状态。

设用户数 U，每位用户最多持有 R 项权限，单次比较次数为 $O(\log U + \log R)$，还需考虑字符串比较长度。总存储不超过授予过的记录规模。权限是用户与资源的二元关系，不能只判断用户是否出现过；撤销 file1 不能影响 file2。

完整参考实现 20_permissions.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <map>
#include <set>
#include <string>
using namespace std;
map<string, set<string> > rights;
int main() {
    int q;
    scanf("%d", &q);
    while (q--) {
        char op[10], user[21], resource[21];
        scanf("%9s%20s%20s", op, user, resource);
        if (strcmp(op, "GRANT") == 0) rights[user].insert(resource);
        else {
            map<string, set<string> >::iterator it = rights.find(user);
            if (strcmp(op, "REVOKE") == 0) {
                if (it != rights.end()) it->second.erase(resource);
            } else {
                int allowed = it != rights.end() && it->second.count(resource);
                puts(allowed ? "YES" : "NO");
            }
        }
    }
    return 0;
}
```

2.4 例题带括号的整数表达式

题目。输入一行表达式，支持非负整数字面量、加、减、乘、括号和一元正负号。乘法优先于加减，同级二元运算从左到右；一元符号优先于乘法；只忽略普通空格。合法时输出结果，语法错误输出 ERROR。不支持除法、变量和隐式乘法，例如 `2(3+4)` 非法。

行长不超过 1000，括号与连续一元符号嵌套不超过 100 层，字面量不超过 10^6；保证求值过程中涉及的数值运算的中间结果绝对值不超过 10^{12}，本题只讨论语法错误，不要求处理溢出。

输入输出样例 1

样例输入

```
2 * (3 + 4) - -5
```

样例输出

```
19
```

输入输出样例 2

样例输入

```
1 + (2 * 3
```

样例输出

```
ERROR
```

按语法拆函数。

语法规则

```
expression = term { ('+' | '-') term }
term       = factor { '*' factor }
factor     = number | '(' expression ')' | '+' factor | '-' factor
```

expression 负责加减，term 负责乘法，factor 负责括号、数字与一元符号。共享指针 p 指向下一个未处理字符，每个函数返回其负责部分的值。term 先完整处理乘法，从而保证优先级；循环保证减法左结合，例如 $8-3-2=3$。

解析前面一个合法表达式不代表整行合法。例如 `1 2`，解析 1 后还剩 2，必须输出 ERROR。最后检查是否读到行尾。时间 $O(L)$，递归空间与嵌套深度有关。

完整参考实现 21_expression.cpp

```cpp
#include <cstdio>
#include <cstring>
char s[1005];
int p = 0, valid = 1;
void skip_space() { while (s[p] == ' ') p++; }
int take(char c) {
    skip_space();
    if (s[p] == c) { p++; return 1; }
    return 0;
}
long long expression();
long long factor() {
    if (take('+')) return factor();
    if (take('-')) return -factor();
    if (take('(')) {
        long long x = expression();
        if (!take(')')) valid = 0;
        return x;
    }
    skip_space();
    if (s[p] < '0' || s[p] > '9') { valid = 0; return 0; }
    long long x = 0;
    while (s[p] >= '0' && s[p] <= '9') x = x * 10 + s[p++] - '0';
    return x;
}
long long term() {
    long long x = factor();
    while (valid && take('*')) x *= factor();
    return x;
}
long long expression() {
    long long x = term();
    while (valid) {
        if (take('+')) x += term();
        else if (take('-')) x -= term();
        else break;
    }
    return x;
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    long long answer = expression();
    skip_space();
    if (!valid || s[p] != '\0') puts("ERROR");
    else printf("%lld\n", answer);
    return 0;
}
```

调试。依次验证单数字、一元负号、连续加减、乘法优先级、嵌套括号，再测空串、缺右括号、多右括号和多余字符。查看每个函数消费到哪里，比只看最终错误数值更有帮助。

2.5 例题树形文档的后代选择查询

真题练习：元素选择器：第 14 次认证；下面先用显式父编号练习祖先链，原题还需解析缩进与选择器（CSP201809C）。

题目。给出按先序排列的一片有根森林，共 n 个节点。每行记录深度、标签和标识。根深度 0，孩子比父亲深 1；输入结构合法，每次深度最多增加 1。标识为减号表示无标识，其余标识互不相

同；标签和标识只含字母数字，长度 1 到 20。

查询由空格分隔的选择符组成。普通选择符匹配标签，以井号开头的选择符匹配标识。`div p` 表示匹配 p 节点，且它的某个**严格祖先**是 div；`div div p` 需要两个不同的 div 祖先按顺序出现，允许隔着其他节点。节点不能把自己当祖先。

$1 \le n \le 500$，$1 \le q \le 50$，最大深度 100，每条查询 1 到 10 个选择符，查询行总长度不超过 1000。第一行 n、q，随后 n 行节点，最后 q 行查询。每次输出匹配数和按输入顺序排列的节点行号，编号从 1 开始。

输入输出样例

样例输入

```
7 5
0 html -
1 body -
2 div main
3 p one
2 div side
3 p two
2 p three
p
div p
#main p
div div p
#side
```

样例输出

```
3 4 6 7
2 4 6
1 4
0
1 5
```

建树。用祖先栈。读取深度 d 的节点时，把栈截断到长度 d；栈顶就是父亲，空栈则为根，然后压入自己。读取同深度兄弟时，旧兄弟会被弹出。

匹配。从查询末尾开始，先检查候选节点本身，再沿父链逆序匹配剩余选择符。找到一个就继续寻找更高祖先。选择最近的匹配祖先不会损失可行解，因为它给剩余前缀保留最多的上方祖先。找齐全部选择符则接受。

候选检查最多走树高 H 条父边并处理 K 个选择符，整体为 $O(qn(H+K))$，本题规模下足够。节点 7 虽是 p，但没有 div 祖先；若程序把它算入 `div p`，通常是把“之前出现的节点”误认为“当前祖先”。

完整参考实现 22_selector.cpp

```cpp
#include <cstdio>
#include <cstring>
struct Node { char tag[21], id[21]; int parent; };
Node a[505];
int ancestors[505], answer[505];
char line[1005], tokens[10][22];
int matches(int u, const char token[]) {
    if (token[0] == '#')
        return strcmp(a[u].id, "-") != 0 && strcmp(a[u].id, token + 1) == 0;
    return strcmp(a[u].tag, token) == 0;
}
int main() {
    int n, q, top = 0;
    scanf("%d%d", &n, &q);
    for (int i = 0; i < n; i++) {
        int depth;
        scanf("%d%20s%20s", &depth, a[i].tag, a[i].id);
        top = depth;
        a[i].parent = top == 0 ? -1 : ancestors[top - 1];
        ancestors[top++] = i;
    }
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) {}
    while (q--) {
        if (fgets(line, sizeof(line), stdin) == NULL) break;
        int count = 0;
        char *s = strtok(line, " \t\r\n");
        while (s != NULL) {
            strcpy(tokens[count++], s);
            s = strtok(NULL, " \t\r\n");
        }
        int total = 0;
        for (int u = 0; u < n; u++) {
            int k = count - 1;
            if (k < 0 || !matches(u, tokens[k])) continue;
            k--;
            int v = a[u].parent;
            while (v != -1 && k >= 0) {
                if (matches(v, tokens[k])) k--;
                v = a[v].parent;
            }
            if (k < 0) answer[total++] = u + 1;
        }
        printf("%d", total);
        for (int i = 0; i < total; i++) printf(" %d", answer[i]);
        printf("\n");
    }
    return 0;
}
```

2.6 例题模板展开与循环依赖

真题练习：模板生成系统：第 5 次认证；下面教学题额外加入递归依赖与环检测，原题按其自身替换规则实现（CSP201509C）。

题目。有 n 个命名模板，每个由若干以空白分隔的词组成。以 at 符号开头的词引用另一个模板，其余词直接输出。定义全部读完后，展开指定起始模板，以单个空格连接所有普通词。从起始模板可达的引用若出现未定义名称或环，输出 ERROR；不可达模板的错误不影响本次输出。

模板名互不重复，只含字母数字，长度不超过 20；引用符号后必须有名称。允许空模板。$1 \le$

$n \le 100$, 每个定义最多 100 个词, 每词长度最多 20。保证无环情况下每个模板完整展开不超过 2000 个词。第一行 n，随后每个定义包含名称、词数 k 及 k 个词，最后给出起始名称。

输入输出样例 1

样例输入

```
3
title 1 world
line 3 hello @title !
all 2 @line @line
all
```

样例输出

```
hello world ! hello world !
```

输入输出样例 2

样例输入

```
2
a 1 @b
b 1 @a
a
```

样例输出

```
ERROR
```

三色状态。0 表示未算，1 表示正在递归，2 表示已经完成并缓存。访问状态 1 说明当前依赖链回到了自己；访问状态 2 直接复用。第二次引用同一模板不必然成环，关键看第一次展开是否结束。

解答。先读入全部定义以允许前向引用。按词序展开，普通词加入结果，引用词接上被引用模板的结果。记忆化避免重算，但每次拼接仍复制相应词。时间取决于查找及所有实际复制的词数；本题每个模板最多缓存 2000 词，总缓存约不超过 100×2000 词，不能忽略输出规模而声称总是 O(n)。

完整参考实现 `23_templates.cpp`

```cpp
#include <cstdio>
#include <map>
#include <string>
#include <vector>
using namespace std;
map<string, int> number;
vector<string> definition[105], memo[105];
int state[105];
int expand(int u) {
    if (state[u] == 1) return 0;
    if (state[u] == 2) return 1;
    state[u] = 1;
    for (int i = 0; i < (int)definition[u].size(); i++) {
        string token = definition[u][i];
        if (token[0] != '@') memo[u].push_back(token);
        else {
            map<string, int>::iterator it = number.find(token.substr(1));
            if (it == number.end() || !expand(it->second)) return 0;
            int v = it->second;
            for (int j = 0; j < (int)memo[v].size(); j++)
                memo[u].push_back(memo[v][j]);
        }
    }
    state[u] = 2;
    return 1;
}
int main() {
    int n;
    scanf("%d", &n);
    char name[21], token[21];
    for (int i = 0; i < n; i++) {
        int k;
        scanf("%20s%d", name, &k); number[name] = i;
        for (int j = 0; j < k; j++) {
            scanf("%20s", token); definition[i].push_back(token);
        }
    }
    scanf("%20s", name);
    map<string, int>::iterator it = number.find(name);
    if (it == number.end() || !expand(it->second)) puts("ERROR");
    else {
        int u = it->second;
        for (int i = 0; i < (int)memo[u].size(); i++)
            printf("%s%s", i == 0 ? "" : " ", memo[u][i].c_str());
        printf("\n");
    }
    return 0;
}
```

2.7 练习与解答

练习 1。cwd 为根，路径为 `../../a`，结果是什么？**解答**。`/a`，超出根的上移被忽略。

练习 2。alice 有 A 和 B，撤销 A 后查询 B 输出什么？**解答**。YES，资源权限分别维护。

练习 3。`-2*-3+4` 的值是多少？**解答**。10。一元负号属于 factor，乘法先得 6，再加 4。

练习 4。一个 div 节点能否独自匹配 `div div`？**解答**。不能，两个选择符必须匹配不同节点，前一个位于严格祖先链。

练习 5。A 引用 B 两次，B 只含 x，是否有环？**解答**。没有。第一次结束后 B 为状态 2，第二

次复用，输出 `x x`。

3 综合训练与复盘

3.1 练习方式

本章三道题均为教学训练题，不对应某次认证的原版试卷。建议先不看解答，给自己安排 30、50、90 分钟的独立尝试，再用解答检查建模过程。用时只是训练建议，不是官方考试分配。

3.2 例题训练 A 差值数对

给 n 个整数和非负整数 c，求满足 $i<j$ 且 $|a_i-a_j|=c$ 的位置对数量。$1\le n\le 2\times 10^5$，$|a_i|\le 10^9$，$0\le c\le 2\times 10^9$。第一行 n、c，第二行数组，输出一个整数。

输入输出样例

样例输入

```
5 2
1 3 1 3 3
```

样例输出

```
6
```

解答。先统计每个值的频次。c>0 时，只从较小值 x 寻找 x+c，不会重复统计同一无序数对；贡献是两组频次乘积。c=0 时，频次 k 的组中任选两个位置，贡献 $k(k-1)/2$。答案最多为 $n(n-1)/2$，需要 long long。时间 $O(n\log n)$，空间 $O(n)$。

完整参考实现 `24_pairs.cpp`

```cpp
#include <cstdio>
#include <map>
using namespace std;
int main() {
    int n; long long c;
    scanf("%d%lld", &n, &c);
    map<long long, long long> count;
    for (int i = 0; i < n; i++) {
        long long x; scanf("%lld", &x); count[x]++;
    }
    long long answer = 0;
    map<long long, long long>::iterator it;
    for (it = count.begin(); it != count.end(); ++it) {
        long long x = it->first, k = it->second;
        if (c == 0) answer += k * (k - 1) / 2;
        else {
            map<long long, long long>::iterator p = count.find(x + c);
            if (p != count.end()) answer += k * p->second;
        }
    }
    printf("%lld\n", answer);
    return 0;
}
```

检查用例。数组 $[7,7,7]$、c=0 应输出 3；单元素时应输出 0。若输出 9，说明把相同位置配对或对称重复都算了进去。

3.3 例题训练 B 矩形增量的最大值

初始有 $n \times m$ 全 0 矩阵，执行 q 次操作，每次给闭矩形左上 (x_1, y_1) 到右下 (x_2, y_2) 加 v。最终输出最大元素值，以及等于最大值的格子数。$1 \le n, m \le 1000$，$0 \le q \le 2 \times 10^5$，$|v| \le 10^6$，坐标合法。第一行 n、m、q，随后每行五个数。

输入输出样例

样例输入

```
3 3 2
1 1 2 2 2
2 2 3 3 3
```

样例输出

```
5 1
```

解答。把每次矩形更新记录在二维差分四角，最后恢复每格真实值，同时维护最大值和次数。只统计 1 到 n 行、1 到 m 列，不能把辅助边界算进去。允许负更新，最大值要从负无穷开始，不能默认 0。时间 $O(nm+q)$，空间 $O(nm)$。

完整参考实现 25_rectangle.cpp

```cpp
#include <cstdio>
#include <climits>
const int N = 1005;
long long d[N][N];
int main() {
    int n, m, q;
    scanf("%d%d%d", &n, &m, &q);
    while (q--) {
        int x1, y1, x2, y2; long long v;
        scanf("%d%d%d%d%lld", &x1, &y1, &x2, &y2, &v);
        d[x1][y1] += v; d[x2 + 1][y1] -= v;
        d[x1][y2 + 1] -= v; d[x2 + 1][y2 + 1] += v;
    }
    long long best = LLONG_MIN, count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1];
            if (d[i][j] > best) { best = d[i][j]; count = 1; }
            else if (d[i][j] == best) count++;
        }
    }
    printf("%lld %lld\n", best, count);
    return 0;
}
```

检查用例。q=0 时输出 0 n*m，第二项应计算成实际格子数；若整个矩阵一次减 5，则最大值为-5 而不是 0。

3.4 例题训练 C 非抢占任务调度

有一台处理器和 n 个任务，每个任务给定到达时刻、优先级和执行时间，输入顺序就是任务编号。处理器空闲时，在所有已经到达但尚未执行的任务中选择优先级最高者；并列时选到达更早者，再并列选编号更小者。一旦开始执行就**不能被抢占**。

一个任务完成时，必须先把恰好在该时刻到达的任务加入候选，再选择下一项。若没有已到达任务，时钟直接跳到下一次到达。输入不保证按到达时刻有序。$1 \le n \le 2 \times 10^5$，到达时刻和优先级在 0 到 10^9 之间，执行时间在 1 到 10^9 之间。

第一行 n，随后每行依次给到达时刻、优先级、执行时间。按输入编号输出每个任务的开始和结束时刻，最后一行输出最大等待时间，等待时间为开始减到达。

输入输出样例

样例输入

```
4
0 1 5
1 1 2
2 3 1
5 2 2
```

样例输出

```
0 5
8 10
5 6
6 8
7
```

推演。任务 1 在 0 到 5 执行，中途到达的高优先级任务不能打断它。时刻 5，先加入任务 4；待选任务 2、3、4 中任务 3 优先级最高，所以执行顺序为 1、3、4、2。任务 2 等到 8 开始，等待 7。

状态划分。排序数组保存还未到达的任务，指针 i 指向下一项；优先队列保存已经到达的任务；now 保存处理器可再次分配工作的时刻。每轮先补全所有到达时间不大于 now 的任务，再选堆顶。

正确性。选择前，堆恰好包含全部可选任务；比较器严格实现三级优先顺序。执行结束后更新 now，不处理中途抢占，符合规则。每个任务入堆出堆各一次，排序与堆操作总时间 $O(n \log n)$，空间 $O(n)$。结束时刻可能达到 10^{14} 量级，应使用 long long。

完整参考实现 26_scheduler.cpp

```cpp
#include <cstdio>
#include <algorithm>
#include <queue>
#include <vector>
using namespace std;
const int N = 200005;
struct Task { long long arrival, priority, duration; int id; };
Task tasks[N];
long long start_time[N], finish_time[N];
bool by_arrival(Task a, Task b) {
    if (a.arrival != b.arrival) return a.arrival < b.arrival;
    return a.id < b.id;
}
bool lower_priority(Task a, Task b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    if (a.arrival != b.arrival) return a.arrival > b.arrival;
    return a.id > b.id;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%lld%lld%lld", &tasks[i].arrival,
              &tasks[i].priority, &tasks[i].duration);
        tasks[i].id = i;
    }
    sort(tasks, tasks + n, by_arrival);
    typedef bool (*Compare)(Task, Task);
    priority_queue<Task, vector<Task>, Compare> ready(lower_priority);
    long long now = 0, max_wait = 0;
    int i = 0;
    while (i < n || !ready.empty()) {
        if (ready.empty()) now = max(now, tasks[i].arrival);
        while (i < n && tasks[i].arrival <= now) ready.push(tasks[i++]);
        Task t = ready.top(); ready.pop();
        start_time[t.id] = now; finish_time[t.id] = now + t.duration;
        max_wait = max(max_wait, now - t.arrival);
        now += t.duration;
    }
    for (int j = 0; j < n; j++)
        printf("%lld %lld\n", start_time[j], finish_time[j]);
    printf("%lld\n", max_wait);
    return 0;
}
```

3.5 如何复盘一次错误

先保存最小失败输入，写出期望输出，再归类原因。输入解析错误要检查分词和换行；状态错误要明确变量对应时刻；算法错误要重新验证单调性、转移或贪心理由；复杂度错误要估计最坏规模。修复后还要补一个能够稳定复现原错误的测试。

例如调度程序把新任务加入堆的条件写成“到达时刻小于 now”，就会漏掉恰好在任务结束时到达的任务。用“任务 1 在 0 开始、执行 2；任务 2 在 2 到达”即可定位边界。改成小于等于后，再检查已有等待任务与同时到达任务的优先顺序。

3.6 提交前检查表

1. 输出内容、顺序和格式是否与题意一致，有没有调试输出残留。
2. 最小输入、全相等、全负数、空结果、不可达等合法边界是否处理。
3. 数值和数组范围是否足够，r+1、i-1 等辅助位置是否分配。
4. 每个循环能否终止；每个递归状态是否完整；是否可能爆栈。
5. 每个容器访问前是否检查为空或 end；比较器是否严格。
6. 程序复杂度是否覆盖最大数据，而不仅是样例。

A 提高专题与模型扩展

本附录承接原资料中的提高内容。建议在完成正文例题后选择学习。每次变换模型时，先检查状态语义是否改变，不要仅凭代码形式相似就替换循环。

A.1 多重背包与二进制拆分

同类练习（在线做题）：P1776 宝物筛选。

每种物品有数量上限，拆成 01 物品包；输入字段次序以原题为准。

第 i 种物品最多有 s_i 件。朴素转移枚举件数 k：

$$f[i][v]=\max_{0\le k\le s_i,\ kc_i\le v}\{f[i-1][v-kc_i]+kw_i\}.$$

当数量较大时，把 s_i 件拆成若干只能选一次的物品包。例如 13 拆成 1、2、4、6，每包体积和价值乘相应件数。选择这些包可以表示 0 到 13 的任意件数，转化为 01 背包。拆包数量为 $O(\log(s_i+1))$，总时间为 $O(V\sum_i\log(s_i+1))$。

```
int remaining = count;
for (int batch = 1; remaining > 0; batch *= 2) {
    int take = min(batch, remaining);
    remaining -= take;
    long long cc = 1LL * take * cost;
    long long ww = 1LL * take * value;
    if (cc <= capacity)
        for (int v = capacity; v >= cc; --v)
            f[v] = max(f[v], f[v-(int)cc] + ww);
}
```

此片段假设 count 不超过 10^6、体积为正，避免 batch 翻倍溢出。若采用更大数量范围，应使用更宽类型并检查翻倍边界。

为什么能覆盖。若已有包可以组成 0 到 t，再加入大小不超过 t+1 的新包，就可以连续覆盖到 t 加新包大小。按 1、2、4 增长并保留余数满足这个条件。

计数陷阱。这个拆分通常用于最优值或可达性，不能直接拿来统计原问题的方案数。例如拆成 1、2、4、6 后，6 可由 6 或 2+4 表示，形成不同拆包选择却代表同样件数。计数背包应另行设计不重不漏的转移。

A.2 混合背包与多维费用

同类练习（在线做题）：P1507 NASA 的食物计划。

对应本节二维费用模型，两种资源容量都必须满足。

混合背包对每种物品分别判断：限一次用逆序，无限次用正序，有限次可枚举或拆包；组内互斥则按整组处理，不能把同组物品当普通独立物品。不同模型混在一起时，应确保每轮转移的“一个决策单位”仍然清晰。

若每件物品同时占体积 c 和重量 d，容量限制为 V 和 W，定义 $f[v][w]$ 为两种资源限制下的最大价值。对于 01 物品，两个容量维度都逆序：

```
for (int v = V; v >= cost; --v)
    for (int w = W; w >= weight; --w)
        f[v][w] = max(f[v][w], f[v-cost][w-weight]+value);
```

时间 $O(nVW)$，空间 $O(VW)$。增加状态维度会迅速增加计算量。应选择取值范围较小的量作维度，必要时考虑按价值记录最小成本，但也要重新定义答案和不可达值。

A.3 方案恢复

同类练习（在线做题）：P1064 金明的预算方案。

这是依赖背包的拓展题：附件必须依附主件。先枚举每个主件的合法购买组合，再作分组背包；不是普通独立物品模型。

若要求输出选了哪些物品，仅得到最大值还不够。对于二维 01 背包，可在每个状态记录是否选择第 i 件，或保留整张 f 表，从 (n, V) 逆推：若最优值来自不选则 i 减 1；若来自选择，则记录 i 并把容量减去体积。

一维压缩后直接保存“当前容量的父指针”容易被后续物品覆盖，造成恢复结果重复选同一物品。为了初学者的可靠实现，先使用二维记录恢复，再考虑持久化等高级优化。并列最优解允许任选时，固定一种规则即可；要求字典序最小时必须把并列规则也纳入设计。

A.4 计数背包的撤销

同类练习（在线做题）：P4141 消失之物。

删除的是一个指定物品，输出恰好装满的方案数末位数字；使用本节计数撤销公式。

设 F[s] 是所有物品组成体积 s 的 01 选择方案数，删除一个体积为 w 的**指定物品**后，G[s] 是剩余方案数。按是否选该物品分类：

$$F[s] = G[s] + G[s-w], \qquad G[s] = F[s] - G[s-w].$$

负下标项为 0。因 G[s] 依赖更小的 G，按 s 从小到大计算。若只输出末位数字，应在模 10 下减法并把负数归一化：

```
for (int s = 0; s <= capacity; s++) g[s] = all[s];
for (int s = w; s <= capacity; ++s)
    g[s] = (all[s] - g[s-w] + 10) % 10;
```

例如两个体积都为 1 的不同物品，完整方案数为 $F[0..2] = [1, 2, 1]$；删去其中一件，得到 $G =$

$[1,1,0]$。物品即使体积相同也按身份区分。该线性计数关系可撤销，但最大值背包的 max 会丢失候选信息，不能简单相减恢复。

A.5 LIS 优化与排列 LCS

同类练习（在线做题）：P1439 两个排列的最长公共子序列。

利用元素各出现一次的性质映射成 LIS；二次 LCS 不能覆盖大规模数据。

定义 tail[len] 为所有长度 len 的严格上升子序列中最小的末尾值。这个数组严格递增。处理 x 时，用 lower_bound 找到第一个不小于 x 的位置并替换；若不存在就把 x 接到末尾。替换更小的末尾不会改变已获得的长度，却给后续元素留下更多延伸可能。

```
int length = 0; // tail[] and a[] have capacity at least n.
for (int i = 0; i < n; i++) {
    int pos = lower_bound(tail, tail + length, a[i]) - tail;
    tail[pos] = a[i];
    if (pos == length) length++;
}
printf("%d\n", length);
```

时间 $O(n \log n)$。tail 的内容不一定是一条真实的 LIS；若要求恢复路径，还需要记录位置和前驱。不下降子序列对应 upper_bound，而不是 lower_bound。

若两个序列都是 1 到 n 的排列，把第一序列每个值的位置记为 pos，再将第二序列映射成位置序列。共同子序列需要在第一序列位置上递增，因此转成 LIS。这个转换利用了元素唯一的性质，不能直接用于有任意重复值的一般 LCS。

A.6 实数二分与三分的前提

实数二分可固定迭代次数，避免直接比较浮点相等。例如连续函数在区间两端异号，可以保留仍有变号的半区间来寻找一个根。端点恰好为根时先处理；两个端点同号不代表没有根，也不能保证可以用这一规则寻找全部根。三次方程若有多个根，应先按题目给定性质隔离各根。

三分用于已知单峰或单谷的目标函数。对单峰求最大值，比较两个三等分点，舍弃不可能包含峰值的一侧；若函数存在多个峰，这个依据就不成立。分数规划、爬山和模拟退火还各有额外假设，不能用“都在搜索答案”把它们当作二分的替代模板。基础前三题训练应先把可证明的单调模型掌握牢固。

A.7 可删堆的延迟删除

优先队列通常只支持删除堆顶。若外部能保证删除请求对应一个当前存在的元素，可维护两个同类型堆：A 保存所有插入值，B 保存待删除值。每次取有效堆顶前，若两堆顶相等就同时弹出，直到不同或有堆为空。

重复值应按次数插入删除堆，不是用布尔标记；还要维护逻辑元素数，以区分真实为空与仍有待清理的节点。延迟删除让每个入堆元素最多被清理一次，但实现前应先判断 multiset 是否已经足够简单可靠。

A.8 字符串匹配的线性预处理

直接在每个起点比较模式串的复杂度最坏为 $O(nm)$，在小规模下完全可用。规模较大时，可用 KMP 的前缀函数维护“已匹配前缀还能退到多长”。设 pi[i] 为模式串前缀 0 到 i 的最长相等真前后缀长度：

```
void prefix_function(const char s[], int pi[]) {
    int n = strlen(s);
    if (n == 0) return;
    pi[0] = 0;
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
}
```

对非空模式 p 扫描文本时，用同样的回退规则更新匹配长度 j；当 j 等于模式长度，记录一次匹配并回退到 pi[j-1]，从而允许重叠匹配。由于指针前进与回退的总量受线性界约束，预处理和匹配合计 $O(n+m)$。空模式的匹配规则应另行规定，不能直接访问 p[0]。

A.9 附录练习与解答

练习 1。数量 10 如何拆包？**解答**。1、2、4、3，和为 10，能表示 0 到 10 的每个件数。

练习 2。两个排列 $[3,1,2]$ 与 $[1,3,2]$ 的 LCS 长度是多少？**解答**。第一排列位置为 pos[3]=1、pos[1]=2、pos[2]=3；第二个映射为 $[2,1,3]$，LIS 长度 2，所以 LCS 长度 2。

练习 3。模式 aaa 在文本 aaaa 中出现几次？**解答**。2 次，起点 0 和 1。记录第一次后应保留可重叠的前后缀，而不是把匹配长度清成 0。

B 资料来源与继续练习

本讲义依据以下原始材料重新编排知识体系，补充了自包含教学例题、解答和参考程序。原课件中的重点概念问题已在正文采用修正后的表述，包括 unique 语义、DP 状态信息、背包初始化、BFS 归类和计数背包撤销。

原始文件	页数	本讲义对应内容
历年前三题考点梳理.pdf	5	题型安排、基础模拟、综合模拟选题方向。
练习题与真题.pdf	2	继续训练入口，仓库中另存链接索引。
stl.pptx	42	第二篇 STL 专章、第三篇解析与可删堆。
前缀和、差分、二分、双指针.pptx	34	第二篇前缀和、二分、双指针。

原始文件	页数	本讲义对应内容
搜索、剪枝、暴力.pptx	13	第二篇搜索与第三篇模板展开。
dp 入门.pptx	18	第二篇 DP、LIS 与 LCS 扩展。
动态规划之背包问题.pptx	53	第二篇背包与附录提高专题。

优先做题入口：曙梦 OJ 题库和历次认证训练。已展开的认证真题及解答见前两篇，各算法旁附直达链接。以下为原课件的专项补充：排序 P1177、去重 P1059、前缀和 P8218、差分 P2367、二分 P2249、切木条 P2440、全排列 P1706、马的遍历 P1443、LIS B3637、最大子段和 P1115、01 背包 P1048、完全背包 P1616、分组背包 P1757。这些题可能有与教学题不同的输入、限制和输出要求，应重新阅读原题。

提高题可选：P2678、P1219、P3956、P1776、P1507、P1064、P1439、P4141。其中 P4141 的目标是删除指定物品后的恰好装满方案数末位数字，不是最大价值；P1439 的较大数据需要利用排列性质，普通二维 LCS 不能覆盖全部规模。

接口语义可查阅 C++ 工作草案的sort 条目和unique 条目。平台原题和标准文档是进一步核对的依据，本讲义正文的教学例题与解答已经提供独立学习所需的规则和算法。

B.1 团队题单与专题入口

练习总入口：洛谷团队 113242 的训练题单。以下 5 个专题地址来自原资料。团队页与专题题单目前需要访问权限，整理时未能核对其中的完整题目列表；正文中的公开题目链接不据此认定为团队题单成员。

专题	打开题单
二分、前缀和、差分、双指针	1048062
搜索与剪枝	1048981
STL	1055276
简单动态规划	1052248
贪心	1052246

B.2 原资料真题做题入口

以下完整覆盖原资料的 23 个真题入口。认证次数、题名和原资料跨平台对应关系已于 2026 年 10 月 11 日核对；正文十道真题速看与十三道剩余真题都给出解答与完整实现。已有曙梦页面优先链接曙梦，未读取到对应题目的旧题保留原资料入口。具体输入与范围按正文所用平台说明。原始链接仍保存在仓库索引，便于追溯。

题号	真题	认证次数与年月	主要做题入口
1	打酱油	第 11 次，2017-09	CSP201709A（原入口）
1	跳一跳	第 13 次，2018-03	CSP201803A（原入口）
1	小中大	第 16 次，2019-03	CSP201903A（原入口）
1	线性分类器	第 19 次，2020-06	CSP202006A（曙梦）
1	检测点查询	第 20 次，2020-09	CSP202009A（原入口）
1	数组推导	第 23 次，2021-09	CSP202109A（原入口）
1	如此编码	第 27 次，2022-09	CSP202209A（原入口）
1	田地丈量	第 29 次，2023-03	CSP202303A（曙梦）
2	买菜	第 14 次，2018-09	CSP201809B（曙梦）
2	回收站选址	第 18 次，2019-12	CSP201912B（曙梦）
2	风险人群筛查	第 20 次，2020-09	CSP202009B（原入口）
2	邻域均值	第 22 次，2021-04	CSP202104B（曙梦）
2	非零段划分	第 23 次，2021-09	CSP202109B（原入口）
2	出行计划	第 25 次，2022-03	CSP202203B（原入口）
2	寻宝！大冒险！	第 26 次，2022-06	CSP202206B（曙梦）
2	何以包邮？	第 27 次，2022-09	CSP202209B（原入口）
2	垦田计划	第 29 次，2023-03	CSP202303B（曙梦）

题号	真题	认证次数与年月	主要做题入口
2	相似度计算	第 33 次，2024-03	CSP202403B（曙梦）
2	机器人宿管指南	第 42 次，2026-05	CSP202605B（曙梦）
2	字符串变换	第 35 次，2024-09	CSP202409B（曙梦）
2	梦境巡查	第 36 次，2024-12	CSP202412B（曙梦）
2	机器人饲养指南	第 37 次，2025-03	CSP202503B（曙梦）
2	机器人复健指南	第 38 次，2025-06	CSP202506B（曙梦）